OSCILLA

User Manual

Animated Graphic Score Framework

Rob Canning

oscilla.cc

2026

Contents

Preface

This manual documents Oscilla, an open-source browser-based framework for creating animated, cue-driven graphic scores. It is generated from the same source files as the online documentation at oscilla.kompot.si/docs.

The manual covers installation, score authoring, the cue DSL, animation, control systems, session features, and developer reference. For the most up-to-date version, consult the online documentation.

Chapter 1

Getting Started

Oscilla is an open-source framework for creating animated, cue-driven graphic scores that run in the browser. Scores are authored as SVG documents in Inkscape, with performance behaviour embedded directly into the graphic elements using a lightweight cue syntax. The result is a single document that is simultaneously a visual composition and an executable performance environment.

This section covers what Oscilla is, how to install it, and how to move from a blank SVG to a working score. Whether you are a composer exploring graphic notation, an improviser looking for networked coordination tools, or a performer encountering an Oscilla score for the first time, these pages will orient you within the system.

At a Glance

Oscilla is a framework for creating, performing, and sharing animated, cuedriven graphic scores in the web browser.

It sits between **notation**, **performance**, and **live electronic control**, allowing composers to design scores that move, react, and synchronize across performers devices while remaining readable, editable, and shareable.

If you can draw it in SVG, Oscilla can turn it into a timebased, interactive score.

What Problems Does Oscilla Solve?

Many contemporary scores:

- rely on **static PDFs** that cannot express temporal or interactive processes
- require **custom software** or fragile patches to realise
- separate **notation**, **rehearsal coordination**, and **electronic control** into different tools

Oscilla addresses this by providing:

- **notationcentric** workflow (the score is the interface)
- **browserbased performance** (no installation for performers)
- **explicit time, cue, and process control** embedded directly in the score

---

What Can You Actually Do With It?

1. Create Animated Graphic Scores

- Draw notation in Inkscape (or any SVG editor)
- Attach behaviours to graphical elements using simple, readable IDs
- Animate rotation, scaling, traversal, and visibility over time

The score becomes **dynamic**, without becoming opaque or procedural.

---

2. Coordinate Ensemble Performance

- Synchronise scores across multiple devices via a local server
- Use shared playheads, rehearsal marks, and cues
- Support fixed form, open form, or hybrid structures

Each performer sees the same temporal structure, while still interpreting locally.

---

3. Work With Composed Improvisation

Oscilla is particularly suited to:

- controlled improvisation
- modular or sectional forms
- scores that evolve differently on each run

Cues can:

- branch
- repeat
- pause
- trigger other events

This allows the score to **guide behaviour** rather than prescribe outcomes.

---

4. Control Electronics (Without Becoming a DAW)

Oscilla allows graphical elements in the score to function directly as **control structures** for electronics.

In practice this means:

- paths, shapes, and points drawn in Inkscape can be interpreted as **trajectories**, **timelines**, or **control curves**
- these structures can drive time, density, position, or other parameters
- the same graphical material serves both as **notation** and **control logic**

This creates a direct visual link between what performers see and how electronics behave.

Oscillas built-in audio system is intentionally simple:

- basic sample playback
- simple synthesis
- time-aligned audio cues placed directly in the score

This makes it possible to realise works for **instruments and fixed media ("tape") entirely in the browser**, without external software or complex setup.

Where more advanced processing is required, Oscilla can:

- send OSC messages to SuperCollider, Pure Data, Max, lighting systems, or custom software
- act as a **notational and organisational layer** for complex electronic setups

Oscilla is not a replacement for audio environments it is the **score that coordinates them**.

---

5. Support Rehearsal and Collaboration

- Performers can add private or shared annotations
- Scores can be shared via URL
- No special software is required beyond a modern browser

This lowers barriers in rehearsal contexts and supports distributed ensembles.

Where Does Oscilla Sit in the Bigger Picture?

Oscilla belongs to a lineage of systems exploring the space between:

- graphic notation
- live electronics
- networked performance

What distinguishes Oscilla is that:

- the **score itself** is the primary interface
- everything runs using **open web technologies**

- composers work visually, not programmatically

Rather than inventing a new notation language, Oscilla **extends existing graphic practices into time, motion, and interaction**.

Who Is Oscilla For?

Oscilla is designed for:

- composers working with graphic or hybrid notation
- performers comfortable with nontraditional scores
- ensembles using electronics, live processing, or networked coordination
- researchers and educators exploring new scoreperformance relationships

It is not aimed at replacing traditional notation tools, but at **augmenting what scores can do**.

Oscilla Installation Guide

Oscilla is a browserbased system for creating and performing synchronized graphic scores. It can be run **either as a standalone desktop application** (recommended for most users) **or** via **Node.js** for development and advanced workflows.

Option 1 Standalone Application (Recommended)

Standalone builds are available for **Linux**, **macOS (Intel & Apple Silicon)**, and **Windows**.

No Node.js, npm, or Git required.

Download

Download the latest release from:

https://github.com/robcanning/oscilla/releases

Choose the build for your operating system:

- **Linux** AppImage
- **macOS** `.app` (Intel or Apple Silicon)
- **Windows** `.exe`

Run

- **Linux**: Make the AppImage executable and run it
  ```
  chmod +x Oscilla-*.AppImage
  ./Oscilla-*.AppImage
  ```
- **macOS**: Open the `.app`
  - You may need to rightclick **Open** the first time
- **Windows**: Doubleclick the `.exe`

Oscilla will start its local server automatically and open in your browser.

Open Oscilla

If it does not open automatically, visit:

`http://localhost:8001`

Requirements for Standalone Use

Tool	Purpose
Inkscape	Create and edit SVGbased score projects
Modern web browser	Chrome, Firefox, Safari, or Edge

Option 2 Node.js / npm Installation (Advanced / Development)

This option is intended for: - contributors and developers - users modifying Oscilla source code - custom server integrations

Requirements

Tool	Purpose
Node.js + npm	Run the Oscilla local server
Inkscape	Create and edit SVG score projects
Modern web browser	View and perform scores
Git (optional)	Clone and update the repository

Windows (Node.js Route)

1. Install Inkscape

https://inkscape.org/release/windows/

2. Install Node.js

https://nodejs.org/en/download

Choose the **Windows Installer (.msi)** and ensure: - **Add to PATH** is enabled

Verify:

```
node -v
npm -v
```

3. Get Oscilla

Option A Git

```
git clone https://github.com/robcanning/oscilla.git
cd oscilla
```

Option B ZIP 1. Download from https://github.com/robcanning/oscilla/releases 2. Extract the ZIP 3. Open PowerShell in the extracted folder

4. Install & Run

```
npm install
npm start
```

Open:

```
http://localhost:8001
```

Linux (Node.js Route)

1. Install Inkscape

```
sudo apt update
sudo apt install inkscape
```

2. Install Node.js (20.x recommended)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

Verify:

```
node -v
npm -v
```

3. Get Oscilla

```
git clone https://github.com/robcanning/oscilla.git
cd oscilla
```

Or download and extract the ZIP from the releases page.

4. Install & Run

```
npm install
npm start
```

Open:

```
http://localhost:8001
```

---

macOS (Node.js Route)

1. Install Inkscape

https://inkscape.org/release/macos/

2. Install Node.js

https://nodejs.org/en/download

Verify:

```
node -v
npm -v
```

3. Get Oscilla

```
git clone https://github.com/robcanning/oscilla.git
cd oscilla
```

Or download and extract the ZIP from the releases page.

4. Install & Run

```
npm install
npm start
```

Open:

```
http://localhost:8001
```

Test the Demo Project

Once Oscilla is running:

```
http://localhost:8001/?project=helper-score
```

You should see the demo score **helperscore**.

Inkscape Extension (Optional)

Oscilla includes an optional **Inkscape extension** to assist with: - creating score templates - inserting cue elements - managing IDs and layers

See the documentation for installation and usage details.

Troubleshooting

Problem	Solution
App does not start	Ensure port **8001** is free
Browser shows blank screen	Check browser console for errors
`npm` not found	Reinstall Node.js
Inkscape SVG not loading	Ensure SVG is saved as **Plain SVG**
Port conflict	Change port in `server.js`

Quick Start

Download the latest release from:

https://github.com/robcanning/oscilla/releases

OR

Clone the repository and start the local server:

```
git clone https://github.com/robcanning/oscilla.git
cd oscilla
npm install
npm start
```

Then open your browser at:

```
http://localhost:8001
```

The demo project can be launched directly from the splash screen.

For full setup instructions (including Inkscape, Node.js, and platform-specific installation), see the Full Installation Guide.

---

title: workflow layout: docs_layout.njk

Workflow

This page describes the current OscillaScore workflow, including **creating, loading, saving, and sharing projects**. It reflects the newer project lifecycle tools available from the UI (New Project, Save As, Import / Export).

The core idea remains the same: **draw in Inkscape, perform in the browser**, but project management is now handled explicitly by Oscilla.

---

1. Projects and Project Structure

All Oscilla projects live inside:

```
public/scores/
```

Each project is a self-contained folder. At minimum, a project contains:

```
myProject/
└── score.svg
```

This is enough for a project to load and run.

A typical project created by Oscilla looks like:

```
myProject/
├── score.svg               # main scrolling or hybrid score
├── preferences.json        # project preferences (auto-generated)
├── pages/                  # optional page-mode SVGs
├── audio/                  # optional local audio files
├── texts/                  # optional external text cues
└── videos/                 # optional local video files
```

You normally do **not** create this structure by hand anymore Oscilla does it for you.

---

2. Creating a New Project

Recommended method (UI)

1. Open Oscilla in your browser:

```
http://localhost:8001
```

2. Open the **hamburger menu** (top-right)
3. Choose:

```
File → New Project…
```

4. Enter a project name

Oscilla will:

- create a new project folder inside `public/scores/`
- copy the **project template** (including `score.svg` and helper files)
- load the project automatically

After creation, Oscilla shows a short hint explaining where the score file lives on disk and that it should be edited in Inkscape.

---

3. Editing the Score in Inkscape

Oscilla does not provide a built-in score editor. **Inkscape is the primary authoring tool.**

1. Open Inkscape
2. Open:

```
public/scores/myProject/score.svg
```

3. Draw shapes, text, paths, and layout the score visually
4. Save the file

Refresh the browser and the changes appear immediately.

Page size conventions

Oscilla expects specific page dimensions depending on score type:

Score Type	Width	Height	Use Case
Scrolling	~40000px	1024px	Continuous horizontal scores
Paged	1366px	1024px	Discrete pages (tablet-friendly)

Set this in Inkscape via:

File Document Properties Page Size

Notes:

- 1024px height is optimized for tablet landscape display
- For scrolling scores, width directly affects playback duration
- Playback speed can be adjusted in `preferences.json`

---

4. Adding Behaviour Using IDs

Behaviour is encoded by editing SVG element IDs (via the XML Editor):

- **Ctrl + Shift + X** (Inkscape)

Examples:

- `pause(dur:12, count:true)`
- `audio(file:intro_theme.wav)`
- `o2p(path:p01)`
- `scale([1,1.3,1])`

Using the Oscilla Inkscape Extension (recommended)

Editing complex IDs manually can be errorprone. Oscilla therefore provides an **Inkscape extension** that makes working with cue and animation IDs significantly easier.

The extension:

- provides structured input fields instead of raw text editing
- helps avoid syntax errors
- exposes common cue and animation parameters
- speeds up authoring for larger or more complex scores

The extension is **optional**, but strongly recommended for regular use.

Documentation and installation instructions:

https://robcanning.github.io/oscilla/docs/inkscape_extension/

You can freely mix workflows:

- use the extension for most edits
- finetune or experiment directly in the XML editor when needed

---

5. Opening and Switching Projects

You can open projects in several ways.

From the loader

1. Visit:

`http://localhost:8001`

2. Select a project from the loader dialog

Direct URL

`http://localhost:8001/?project=myProject`

From the menu

Use:

`File → Projects`

to switch between existing projects without restarting the server.

---

6. Saving and Duplicating Projects

Save As

Use:

`File → Save Project As…`

This:

- duplicates the current project
- assigns a new project name
- preserves all SVGs, media, and preferences

The new project is loaded automatically.

This is the recommended way to create variations or rehearsal versions.

---

7. Exporting and Importing Projects (.oscilla)

Oscilla supports bundling a complete project into a single file for sharing.

Export

`File → Export Project (.oscilla)`

This creates a `.oscilla` file containing:

- `score.svg`
- `preferences.json`
- pages, audio, texts, and videos (if present)

You can send this file to collaborators or archive it.

Import

`File → Import Project (.oscilla)`

When importing:

- you choose a **new project name**
- the project is unpacked into `public/scores/`
- the project is loaded automatically

This allows projects to be shared without manually copying folders.

---

8. Iteration Loop (Typical Use)

In practice, work looks like this:

1. Edit `score.svg` in Inkscape
2. Save
3. Refresh the browser
4. Test cues, timing, and layout
5. Repeat

Oscilla never locks files the filesystem is always the source of truth.

---

9. Rehearsal and Performance

For rehearsal and performance:

- open the same project on all devices
- use cues for timing and navigation
- optionally synchronize multiple clients

Each performer reads from the same authored score, rendered locally in their browser.

Summary

- Projects are **folders**, not opaque files
- Inkscape is the **authoring environment**
- Oscilla handles **loading, duplication, import/export, and playback**
- `.oscilla` files are for **sharing and archiving**, not daily editing

This separation keeps the system transparent, hackable, and robust.

Working with Parts

Oscilla supports two approaches to per-performer parts: **separate score files** and **stacked layers within a single SVG**. Both are configured in the **Parts** tab of the preferences dialog and stored per-browser, so each performer on a shared network can independently choose their own view.

The two approaches can be combined. A project might use separate score files for radically different parts (e.g.a conductor score and a performer score) while also using layers within each file for finer distinctions (e.g.violin I and violin II sharing a string parts score).

Approach 1: Separate Score Files

Each part is authored as an independent scrolling SVG. All files live in the project root alongside `score.svg`:

```
myProject/
├── score.svg                 # full score (always present)
├── score-part-violin.svg     # violin part
├── score-part-viola.svg      # viola part
├── score-part-cello.svg      # cello part
├── score-conductor.svg       # conductor view
├── preferences.json
└── pages/
```

Naming Convention

Files must match the pattern `score*.svg`:

- `score.svg` the main or full score (always listed first)
- `score-part-violin.svg` a part score
- `score-conductor.svg` a conductor score
- `score-electronics.svg` an electronics part

The prefix `score` is required. Everything after it becomes the display label in the preferences dialog. Hyphens become spaces: `score-part-violin.svg` appears as **part violin**.

When to Use Separate Files

Separate files are appropriate when parts differ substantially in content, layout, or length:

- different notation for different instruments
- a conductor score with rehearsal annotations that players should not see
- an electronics part showing control data rather than musical notation
- parts at different zoom levels or page sizes
- simplified parts for less experienced performers

Authoring

Each file is a standard Inkscape SVG. All the usual cue syntax works identically cues are embedded in element IDs just as in `score.svg`. The scrolling playhead, transport, timing, and synchronisation all work the same regardless of which score file is loaded.

One important consideration: if you use cue IDs that reference specific elements (e.g.`nav(scroll@A)`), the rehearsal marks referenced must exist in the currently loaded score file for that performer. If mark **A** exists in `score.svg` but not in `score-part-violin.svg`, navigation cues targeting it will not work for a performer who has selected the violin part.

Performer Selection

Each performer selects their score file in **Preferences > Parts > Score**. The selection is stored in the browsers `localStorage` and takes effect on the next project load. This means each performer on a different device (or browser) can independently choose which score file they see, while all remaining synchronised via the shared transport.

Approach 2: Stacked Layers

A single `score.svg` contains all parts as Inkscape layers. Each performer can highlight their own layer and dim the others.

```
score.svg
├── [layer] part-violin
├── [layer] part-viola
├── [layer] part-cello
├── [layer] shared-notation      ← always visible (no "part" in name)
└── [layer] background           ← always visible (no "part" in name)
```

Naming Convention

Oscilla scans Inkscape layers (groups with `inkscape:groupmode="layer"`) and includes only those whose label contains the word **part** (case-insensitive). Layers without “part” in their name are always visible to all performers and are not affected by the filter.

This means you can freely mix filterable and non-filterable layers:

Layer Name	Filterable?	Why
part-violin	Yes	contains “part”
part-cello	Yes	contains “part”
conductor-part	Yes	contains “part”
shared-notation	No	no “part” in name
background	No	no “part” in name
grid	No	no “part” in name

Creating Layers in Inkscape

1. Open `score.svg` in Inkscape
2. Open the Layers dialog: **Layer > Layers** (or Shift+Ctrl+L)
3. Create a layer for each part, naming it with “part” in the name

4. Draw each instruments notation on its own layer
5. Put shared elements (barlines, rehearsal marks, time signatures, background) on a layer without "part" in the name
6. Save

When to Use Layers

Layers work well when parts share the same horizontal timeline and general layout but differ in vertical content:

- ensemble scores where all parts scroll at the same speed
- scores where performers benefit from seeing each others parts in context
- pieces where the spatial relationship between parts is meaningful
- situations where you want a single file to maintain

Performer Selection

Each performer selects their layer in **Preferences > Parts > My Part**. The **Other Parts** slider controls how visible the remaining part layers are (0% = invisible, 100% = fully visible). The default is 15% enough to see the shape of other parts without them dominating the view.

Like score file selection, layer preferences are stored per-browser and apply immediately.

Combining Both Approaches

A project can use both methods simultaneously. For example:

```
myProject/
├── score.svg                # full score with all layers
│   ├── [layer] part-violin-I
│   ├── [layer] part-violin-II
│   ├── [layer] part-viola
│   └── [layer] shared
├── score-part-cello.svg     # cello has its own score entirely
├── score-conductor.svg      # conductor view with annotations
└── preferences.json
```

Here the string players share a layered score (each highlighting their own part), while the cellist and conductor each have dedicated score files.

The Parts tab in preferences adapts to what is available: if multiple score files exist, it shows the Score dropdown. If the currently loaded score contains layers with "part" in the name, it shows the layer filter controls. If both are present, both controls appear.

View Modes

The **Default View** setting in **Preferences > Settings** interacts with parts:

Mode	Behaviour
hybrid (default)	Loads the selected score file for scrolling. Page navigation also available via cues or the mode toggle.
scroll	Same as hybrid. Score file is required.
page	Starts in page mode. If a score file exists, the mode toggle allows switching to scroll view. If no `score*.svg` files exist, pure page mode with no scroll option.

In hybrid and scroll modes, the selected score file determines what scrolls. In page mode, score file selection is irrelevant unless the performer switches to scroll view.

Project Structure Summary

```
myProject/
├── score.svg                # main scrolling score (required for scroll/hybrid)
├── score-part-*.svg         # optional per-performer score files
├── preferences.json         # project settings
├── pages/                   # page-mode SVGs (optional)
│   ├── home.svg
│   └── section-a.svg
├── audio/                   # audio files (optional)
├── texts/                   # text cue content (optional)
└── videos/                  # video files (optional)
```

Quick Reference

What you want	What to do
Different notation per instrument	Create separate `score-part-*.svg` files
Same notation, highlight own part	Use Inkscape layers with “part” in the name
Both	Combine: layers inside score files, plus separate files for very different parts
Shared elements visible to all	Put them on a layer without “part” in the name
Performer chooses their part	Each performer opens Preferences > Parts
Settings persist across sessions	Automatic stored in browser localStorage
Settings sync across devices	They do not each browser has its own selection, which is the intended behaviour

Chapter 2

Session Layer

The session layer encompasses features that operate on top of the score during rehearsal and performance. These are per-performer tools that require no SVG authoring they work with any score, or even with no score at all.

The playhead provides a configurable look-ahead window that performers can adjust based on notation density and personal preference. Instrumental parts allow performers to highlight their own material while keeping the ensemble context visible. Annotations provide a text-and-audio compositional layer created directly in the browser, from simple rehearsal notes to executable audio triggers. Freehand drawing, drop markers, and sequence timers offer further ways to sketch structure onto a shared timeline without interrupting the flow of a rehearsal or performance.

Together, these tools form a flexible interaction surface that sits between the authored score and the live performance moment.

Playhead & Playzone

The playhead is the vertical line that marks the current reading position in a scrolling score. As playback advances, the score moves beneath the playhead and cues are triggered when their bounding box crosses the playhead line.

The playzone is a translucent highlight region centred on the playhead. It gives performers a wider visual context of "where we are now" and can be styled per project.

Adjustable Playhead Position

By default, the playhead sits at the horizontal centre of the screen. This means roughly equal amounts of upcoming and past notation are visible at any moment. But different musical situations call for different reading positions, and performers often have strong preferences about how much lead time they need.

Why Move the Playhead?

Preparation time. In a densely notated score with precise rhythmic cues or complex extended technique instructions, performers need to see material well in advance. Shifting the playhead to the left say 25% or 30% from the screen edge gives significantly more screen space to the right, meaning upcoming notation stays visible for longer before it reaches the intersection point. This extra read-ahead time can be the difference between a confident entrance and a scrambled reaction.

Loose timing and sustained material. In freer passages where timing is approximate and material might be held, stretched, or revisited, a centred or even right-of-centre playhead keeps the current material on screen longer. Performers who are sustaining a texture or interpreting a passage freely may not want the notation theyre working with to disappear off the left edge too quickly. Keeping the playhead further right means the "now" zone lingers.

Ensemble coordination. In networked performance where multiple performers share the same scrolling score, different instrumentalists may need different amounts of preparation depending on their part. A percussionist reading rapid triggering cues might want maximum look-ahead, while a vocalist sustaining long tones might prefer a centred view. Since the playhead offset is a per-device setting, each performer can adjust independently without affecting synchronisation the underlying score position stays locked across all clients.

Audience projection. When a separate display shows the score to an audience, a centred playhead often makes the most visual sense, giving equal context on either side. Meanwhile the performers tablets might

have the playhead pulled left for more look-ahead. This is a natural configuration in live performance setups.

Rehearsal vs.performance. During rehearsal, a performer might want a centred playhead for a balanced overview while learning the piece. In performance, they might shift it left once they know the material and need that extra preparation window for confident execution. The drag handle makes this a quick adjustment between runs.

Using the Drag Handle

1. **Hover** near the playhead line. A small grip handle and a lock icon appear at the top of the line.
2. **Click the lock icon** to unlock the playhead position.
3. **Drag the grip handle** left or right to reposition. The playzone follows automatically.
4. **Click the lock icon** again to lock the position and prevent accidental movement during performance.

The offset is saved per device and persists across page reloads.

Using Preferences

Open the project preferences dialog (hamburger menu) and look for **Playhead Position %** in the Appearance section. The slider ranges from 10% (far left) to 90% (far right), with 50% being the default centre position.

The preferences value is saved per project on the server, while the drag handle saves per device in the browser. The drag handle setting takes priority on load, since screen size and performer preference are inherently local.

Visibility Toggle

The playhead and playzone can be cycled through four visibility states using the toggle button in the transport controls (bottom-right grid):

- **Both** playhead line and playzone visible (default)
- **Playhead only** just the line
- **Playzone only** just the highlight region
- **None** both hidden

Appearance

Playhead and playzone colours and the playhead line width can be configured in the project preferences dialog under **Appearance**:

- **Playhead Color** colour of the vertical line
- **Playzone Color** colour of the highlight region (supports transparency via hex-with-alpha)
- **Playhead Width** thickness of the line in pixels

Cue Interaction

Cues are triggered when the playhead crosses their left edge during forward playback. The collision system reads the playhead elements actual screen position, so adjusting the playhead offset does not break cue triggering cues always fire at the playhead line, wherever it is on screen.

This applies to all cue types: navigation, audio, synth, OSC, animation triggers, pause, stop, and continuous control lanes (`oscCtrl`).

Repeat Indicator

When a `cueRepeat` region is active, the playhead line turns red and a count box appears near the bottom showing the current repeat iteration. Clicking the count box cancels the repeat and resumes normal playback.

Edge Behaviour

At the very start and end of the score, the playhead departs from its configured screen position to avoid showing empty space beyond the score boundaries:

- **Near the start**: the scores left edge stays at the screens left edge and the playhead moves rightward from the left toward its configured offset.
- **Near the end**: the scores right edge stays at the screens right edge and the playhead moves rightward from its configured offset toward the right.

During the main body of the score, the playhead stays fixed at its offset position and the score scrolls beneath it.

Parts & Layers per-performer layer filtering

Allows each performer to select their own instrumental part from the Inkscape layers in a score, dimming other parts to a configurable opacity. This is a client-side, per-browser preference each device remembers its own part selection independently.

Authoring in Inkscape

Organise your score using Inkscapes layer system. Each layer represents one instrumental part or section of the ensemble. Layer names become the labels shown in the Oscilla preferences dialog.

```
Layer panel in Inkscape:


  Part: Violin        <-- contains "part", detected
  Part: Viola         <-- contains "part", detected
  Part: Electronics   <-- contains "part", detected
  Background          <-- no "part", always visible
  Shared              <-- no "part", always visible
```

Each layer corresponds to a `<g>` element in the SVG with Inkscapes layer attributes:

```
<g inkscape:groupmode="layer"
   inkscape:label="Part: Violin"
   id="layer1">
  <!-- score content for violin -->
</g>
```

The only naming requirement is that the layer label contains the word “part” (case-insensitive). Everything else about the name is freeform. Layers without “part” in their name are never filtered, making them suitable for shared notation, grids, or background elements.

Layer detection

When a score is loaded, Oscilla scans the SVG for all `<g>` elements with `inkscape:groupmode="layer"` whose label contains the word “part” (case-insensitive). Layers without “part” in their name are left untouched by the filter and always render at their authored visibility.

This means layers like “Part 1 Violin”, “viola part”, or “PART: Electronics” are all detected, while layers named “Background”, “Grid”, or “Shared” are excluded and remain fully visible regardless of the performers selection.

Preferences UI

The “Parts” section appears automatically in the Preferences dialog when a score contains Inkscape layers. It provides two controls:

Control	Description
My Part	Dropdown listing all detected layers, plus “All (no filter)”
Other Parts	Opacity slider (0100%) for non-selected layers

Changes apply immediately as a live preview. The setting is also saved when the Preferences dialog Save button is pressed.

Behavior

- Selecting a part sets that layer to full opacity and all other layers to the configured “Other Parts” opacity.
- When a part is selected, all layers are forced to `display:inline`, overriding any Inkscape visibility state. This ensures hidden layers become visible (at reduced opacity) so the performer sees the full score context.
- Selecting “All (no filter)” restores all layers to their authored visibility and clears any opacity overrides.
- The opacity slider at 0% fully hides other parts. At 100% all parts are equally visible (useful for rehearsal or conducting).

Storage

Layer filter preferences are stored in `localStorage`, keyed per project:

`oscilla_layerFilter_<projectName>`

This means each browser or device maintains its own part selection. A violinists tablet remembers “Part: Violin” while the cellists tablet remembers “Part: Cello”, even when viewing the same score.

The preference is not saved to `preferences.json` on the server and does not affect other connected clients.

Authoring tips

- Include the word “part” in any layer name that should appear in the performers dropdown (e.g.“Part: Violin I”, “Electronics Part”, “Percussion part”). The match is case-insensitive.
- Place shared notation (rehearsal marks, structural cues, tempo markings) on a layer whose name does not contain “part” it will always remain visible regardless of the performers selection.
- Layer ordering in Inkscape determines rendering order in the SVG. Place shared/background layers below part layers.
- Cue elements (animations, navigation, audio triggers) work normally regardless of layer filtering opacity changes are purely visual.
- If a layer is hidden in Inkscape (`display:none`), it will remain hidden until a performer actively selects a part, at which point all layers become visible at their respective opacities.

Technical details

The feature is implemented in `layerFilter.js` with integration points in `projectLoader.js` (layer scanning on score load) and `oscillaPreferences.js` (UI section in preferences dialog).

Layer detection uses both namespace-aware attribute access (`getAttributeNS`) and a plain attribute fallback (`getAttribute("inkscape:groupmode")`) to handle differences in how browsers resolve XML namespaces when SVG is embedded in an HTML document.

Notes

- Only Inkscape layers whose label contains “part” (case-insensitive) are detected. Other layers and plain SVG `<g>` groups are not affected by the filter.
- The filter applies to scroll mode. Page mode loads individual SVG files which may have their own layer structure.
- All animations, cue triggers, and OSC output continue to function normally on filtered layers the filter only affects visual opacity.

Performer Annotations

Overview

Performer Annotations allow performers to add **personal notes directly onto the score view** while working with Oscilla.

Annotations are intended for: - reminders - listening cues - coordination notes - interpretive decisions - rehearsal observations

Annotations **do not change the score** and **do not affect playback or cues**.
They exist as a separate, optional layer that can be shown or hidden at any time.

> ***Note:*** *Because annotations can be placed freely in time and space, Oscilla can be used in a* ***minimal, annotation-only mode*** *(without an SVG score). In this case, annotations function as a very lightweight textual score. This mode is intentionally limited, but can be useful for sketches, instructions, or rehearsal frameworks.*

Entering Annotation Mode

1. Click the **pen icon** in the toolbar
2. The cursor changes to indicate annotation mode
3. Click anywhere on the score area to add a new annotation

Annotation mode automatically exits after saving a note.

Creating an Annotation

When you click the score area in annotation mode:

1. A text editor appears
2. Enter your note (multi-line text is supported)
3. Choose whether the note is:
   - **Local** visible only on your device
   - **Shared** visible to other connected clients
4. Click **Save**

The annotation appears as: - a small dot (anchor) - a text label displaying the note

---

Editing an Annotation

- Click **either the dot or the text label** to open the editor
- You can:
  - edit the text
  - change Local / Shared status
  - adjust font size
  - delete the annotation

Click **Save** to apply changes.

Moving an Annotation

- Click and **drag the text label** to reposition the annotation
- Release to confirm the new position
- The position is saved automatically

The dot remains visually tied to the text.

Keyboard Behaviour While Editing

While typing in the annotation editor: - All global keyboard shortcuts are temporarily disabled - This prevents accidental playback or navigation - Press **Escape** to close the editor

Normal keyboard controls resume once the editor is closed.

Local vs Shared Annotations

Local

- Stored only in your browser
- Not visible to others
- Suitable for personal rehearsal notes

Shared

- Broadcast to other connected clients
- Read-only for other performers
- Useful for ensemble coordination

Shared annotations do not override local ones.

Visibility

Annotations: - can be globally shown or hidden - move with the score when scrolling or paging - remain aligned to the score content

They never affect timing, cues, or navigation.

What Annotations Do *Not* Do

Annotations: - do not trigger playback - do not alter the score - do not affect synchronization - do not replace cues or instructions

They are purely informational.

Notes on Use

Annotations are best used for: - marking sections to listen closely - noting transitions or handovers - tracking rehearsal decisions - personal memory aids

They are intentionally lightweight and non-authoritative.

Future Changes

This feature is evolving. Possible future additions include: - filtering annotations by author - export / import of annotation files - visibility modes (rehearsal vs performance)

Freehand Annotation

Overview

The freehand annotation layer allows performers and composers to mark up the score directly in the browser using finger, stylus, or mouse. Strokes are captured as vector paths anchored in score coordinates, persisted locally, and optionally synchronised across all connected clients via WebSocket.

Freehand annotations serve the same role as ink on a printed part: rehearsal marks, breath indicators, dynamic contours, circled passages, conductor gestures, or any other visual markup a musician might add during preparation or performance. They coexist with the SVG score and the structured annotation layer without affecting playback, cues, or timing.

Strokes drawn in a single session are automatically grouped. Groups can be selected, repositioned, deleted, and have their network scope toggled between local and shared.

Three Coexisting Layers

SVG Score Layer	Annotation Layer	Freehand Layer
Authored in Inkscape	Text + audio triggers	Freehand strokes
Embedded DSL cues	Click-to-execute	Visual markup only
Fixed at authoring time	Editable in browser	Editable in browser
Playhead-driven	Performer-driven	Performer-driven

All three layers move together with the score during scrolling and playback. None interfere with each other.

Entering Draw Mode

1. Click the **paintbrush icon** in the top bar (next to the annotation pen icon)
2. The button highlights to indicate draw mode is active
3. A **toolbar** appears at the bottom of the screen with colour, width, select, eraser, and share controls
4. Draw directly on the score with finger, stylus, or mouse

Draw mode captures all pointer input on the score area. Score dragging is disabled while drawing. During playback the score continues to auto-scroll normally.

> ***Tip:*** *On a tablet with a stylus, draw mode gives you a natural ink-on-paper feel. Pressure data is captured for future variable-width rendering.*

Drawing

When draw mode is active:

- **Touch down / mouse down** on the score begins a stroke
- **Move** to extend the stroke it renders live as you draw
- **Lift / release** to finish the stroke
- The stroke is **saved automatically** to localStorage (and broadcast to other clients if sharing is enabled)

Very short taps (accidental touches) are discarded.

Strokes are rendered as smooth curves using quadratic bezier interpolation. Points are thinned during capture to keep data compact.

Session Grouping

All strokes drawn in a single draw-mode session belong to the same **group**. A new group is created each time draw mode is activated (clicking the paintbrush icon or pressing D). When you exit draw mode and re-enter, a new group begins.

This means drawing a word, symbol, or phrase produces a single group that can later be selected and moved as a unit.

To split work into separate groups, toggle draw mode off and on again between drawing sessions.

Toolbar

The floating toolbar appears when any drawing sub-mode (draw, select, or erase) is active. It can be **dragged** to a different position by the grip handle on its right edge.

Colour

A row of colour swatches. Click a swatch to change the stroke colour and return to draw mode. The active colour is indicated by a white border.

Default colours: white, red, blue, green, yellow, pink, grey.

Width

A slider controlling stroke width (120 units). Adjusts the thickness of subsequent strokes. Existing strokes are not affected.

Select / Move

A toggle button for select mode (see below). Highlighted in blue when active.

Eraser

A toggle button for eraser mode (see below). Highlighted in red when active.

Share

A toggle button controlling whether new strokes are shared with all connected clients or kept local. Highlighted in green when sharing is enabled (the default).

Drag Handle

The small grip bar on the right edge of the toolbar. Drag it to reposition the toolbar anywhere on screen.

Select and Move

Click the **move icon** in the toolbar (or press **V** while in draw or eraser mode) to enter select mode.

In select mode:

- Hover over any stroke to highlight its entire group (blue glow)
- **Click** a stroke to select its group a dashed blue bounding box appears
- **Shift-click** another group to add it to the selection (or remove it if already selected)
- **Drag** to reposition all selected groups together
- **Click empty space** to deselect

The bounding box encompasses all selected groups. On drag release, all stroke coordinates are updated and saved.

Keyboard shortcuts in select mode

Key	Action
Delete / Backspace	Delete all selected groups
S	Toggle scope of selected groups between local and shared
D	Return to draw mode
E	Switch to eraser
Escape	Exit select mode

Eraser

Click the **eraser button** in the toolbar (or press **E** while in draw or select mode) to enter erase mode.

In erase mode:

- Hover over a stroke to highlight it in red
- Click the stroke to delete it
- The deletion is saved immediately

Erasing works at the stroke level each drawn line is a separate object that can be individually removed. To delete an entire group at once, use select mode and press Delete.

Press **D** to return to draw mode.

Undo

Press **Ctrl+Z** (or Cmd+Z on Mac) while in draw or erase mode to undo the most recent stroke by the current author. Repeat to undo further strokes.

Exiting Draw Mode

Any of:

- Click the **paintbrush icon** again to toggle off
- Press **Escape** from any sub-mode
- The toolbar disappears and normal score interaction resumes

---

Keyboard Shortcuts

Key	Action	Context
Escape	Exit all drawing modes	Any drawing mode
D	Switch to draw mode	Select or eraser mode
V	Switch to select mode	Draw or eraser mode
E	Toggle eraser	Draw or select mode
S	Toggle local/shared scope on selection	Select mode with group(s) selected
Delete / Backspace	Delete selected group(s)	Select mode with group(s) selected
Ctrl+Z / Cmd+Z	Undo last stroke	Draw or erase mode

All shortcuts are suppressed when a text input is active (e.g.the annotation editor).

Local vs Shared

Shared (default)

- Broadcast to all connected clients via WebSocket
- Visible to everyone viewing the same project
- Stored on the server for late-joining clients
- Useful for conductor markings, collaborative annotation, or ensemble rehearsal notes

Local

- Stored only in the current browser
- Not visible to other performers
- Suitable for personal rehearsal markup

The share toggle on the toolbar controls the default scope for new strokes. The scope of existing strokes can be changed after the fact: select a group and press **S** to toggle all its strokes between local and shared.

Shared strokes are transmitted as compact point-array data, not as images. All clients render strokes locally from the same world coordinates, so they align correctly regardless of screen size or resolution.

Persistence

Freehand annotations are stored as part of the annotation data in localStorage, keyed by project name. They persist across browser sessions and page refreshes.

They are included in annotation import/export (JSON). When exporting annotations, strokes are exported alongside text annotations, markers, and triggers in the same file.

Coordinates and Scaling

Strokes are stored in **world coordinates** the same coordinate space as the SVG scores viewBox. This means:

- Strokes remain anchored to the correct position in the score regardless of screen size or zoom level
- A stroke drawn on a tablet appears in the same score position on a laptop
- Shared strokes align correctly across devices with different resolutions

The drawing overlay uses an SVG element with a matching viewBox, so coordinate conversion is handled automatically by the browsers SVG rendering.

Stroke Appearance

By default, strokes use `vector-effect: non-scaling-stroke` in CSS. This means the visual thickness of a line stays constant regardless of how the score is scaled the same way a pen mark on paper looks the same width whether you hold the page close or far away.

To make strokes scale with the score instead (thicker when zoomed in, thinner when zoomed out), remove the `vector-effect` rule from `oscillaDrawing.css`.

What Freehand Annotations Do *Not* Do

Freehand annotations:

- do not trigger playback or cues
- do not alter the SVG score
- do not affect synchronisation or timing
- do not carry executable behaviour

They are purely visual markup. For executable score elements, use structured annotations with triggers.

Technical Notes

Data Model

Strokes are stored as annotation items with `kind: "stroke"`:

```
{
  "id": "ann_m3k...",
  "kind": "stroke",
  "scope": "shared",
  "anchor": { "mode": "scroll" },
  "placement": { "space": "score" },
  "stroke": {
    "points": [
      { "x": 1234.5, "y": 456.2, "p": 0.72 },
      { "x": 1238.1, "y": 458.0, "p": 0.68 }
    ],
    "color": "#ffffff",
    "width": 3,
    "opacity": 1,
    "groupId": "grp_01JK..."
  }
}
```

The `p` field records pointer pressure (01) from stylus input. This data is captured but not yet used for variable-width rendering.

The `groupId` field links strokes drawn in the same session. All strokes sharing a `groupId` are selected and moved together.

Module Location

`public/js/interaction/drawing.js`

Integrated via `interactionSurface.js` alongside the annotation editor, markers, and trigger system.

Window API

```
// Mode control
window.oscillaDrawing.toggleDrawMode()
window.oscillaDrawing.setDrawMode(true)
window.oscillaDrawing.setEraserMode(true)
window.oscillaDrawing.setSelectMode(true)
window.oscillaDrawing.toggleSelectMode()

// Stroke settings
window.oscillaDrawing.setStrokeColor("#ff4444")
window.oscillaDrawing.setStrokeWidth(5)

// Sharing
window.oscillaDrawing.toggleShareDrawing()       // toggle default scope
window.oscillaDrawing.toggleSelectedScope()       // toggle selected groups

// Edit operations
window.oscillaDrawing.undoLastStroke()
window.oscillaDrawing.clearLocalStrokes()
```

```
window.oscillaDrawing.deleteSelectedGroup()       // delete all selected groups

// State queries
window.oscillaDrawing.isDrawMode()
window.oscillaDrawing.isEraserMode()
window.oscillaDrawing.isSelectMode()
window.oscillaDrawing.isShareDrawing()
window.oscillaDrawing.getStrokeColor()
window.oscillaDrawing.getStrokeWidth()
```

Also accessible via the annotation API:

```
window.oscillaAnnotations.toggleDrawMode()
window.oscillaAnnotations.setDrawMode(true)
```

Future Directions

- **Pressure-sensitive rendering**: variable-width strokes using captured pressure data
- **Page mode support**: freehand annotation in page view (currently scroll mode only)
- **Stroke simplification**: Ramer-Douglas-Peucker path reduction for long sessions
- **Drawing layers**: multiple named layers (e.g.“rehearsal 1”, “rehearsal 2”) with independent visibility
- **Full colour picker**: integration with the existing `colorPicker.js` component

Markers

Markers are visual waypoints that performers and composers can drop onto the score during playback, rehearsal, or pre-performance planning. Unlike rehearsal marks (which are authored into the score itself), markers are created in the moment by anyone in the ensemblea lightweight way to sketch structure onto a timeline without interrupting the flow.

No SVG authoring required

Markers are part of Oscillas **interaction layer** features that work on top of any score without requiring you to edit SVG files or learn cue syntax. You can:

- Load any image or simple graphic as a score
- Drop markers to create structure
- Navigate between markers with arrow keys and transport buttons
- Use the timer system for durational cues, with automatic jumps to markers on completion
- Annotate with pins and text

This makes markers an accessible entry point for ensembles who want networked coordination and shared visual structure without diving into Oscillas more advanced SVG-based cue system. The score becomes a backdrop; markers become the notation. Combined with countdown timer sequences and `onComplete` actions, markers can form a complete temporal framework a **session layer** built entirely from the interaction tools.

For groups working with graphic scores, text scores, or fully improvised sets, this approach lets you use Oscilla as a **shared timeline and coordination tool** rather than a notation rendering engine. Wire markers to timer sequences and you have automated section navigation without writing a single line of SVG.

What markers are for

Markers serve as **compositional scaffolding** for loosely-defined works. In graphic notation, open scores, and improvised music, the timeline is often a container rather than a prescription. Markers let you populate that container with signposts:

- **Structural landmarks** “climax here”, “transition begins”, “return to opening texture”
- **Solo assignments** designate who plays when across a 20-minute set
- **Moments to revisit** flag a passage during a run-through without stopping
- **Cues for coordination** visible anchors that all performers can see and anticipate
- **Navigation targets** named markers can be jumped to with arrow keys, transport buttons, `nav()` cues, or timer completion actions

Think of markers as sticky notes on a shared timeline. Theyre fast to place, easy to move, and visible to everyone (or just yourself, if you prefer). Unlike plain sticky notes, though, theyre wired into the transport you can jump between them, and other systems can target them.

Pre-performance planning

Before a performance, markers can sketch out a flexible roadmap:

1. **Set duration** You have a 20-minute set. The score provides material but not strict form.
2. **Drop structural markers** Place markers for major sections: intro, development, climax, wind-down, ending.
3. **Assign solos** Each performer drops markers in their own color indicating where theyll take the lead.
4. **Mark transitions** Add markers where the ensemble should shift texture, density, or energy.
5. **Size for visibility** Drag important markers to the center of the score and increase their font size so theyre visible from across the stage.

This creates a shared map thats visible on every connected client. During performance, the playhead moves through the score and performers see their markers approachinggentle reminders rather than rigid cues.

During rehearsal

Markers shine as a **non-interruptive annotation tool**:

- Spot something interesting or problematic? Drop a marker without stopping.
- After the run-through, use Arrow Up/Down to jump between markers, or click any marker label to jump back and discuss.
- The marker label (editable) can hold a quick note: “bass too loud”, “nice texture”, “try again”.

This keeps rehearsals flowing. Instead of “stop, lets go back to where was it?”, you have a trail of breadcrumbs to follow and keyboard shortcuts to follow them instantly.

Shared and local markers

By default, markers are **shared** they appear on all connected clients scores in real-time. This supports collaborative planning and ensemble-wide visibility.

Toggle the network icon in the marker tools to switch to **local** mode. Local markers are private to your screen, useful for personal notes you dont need to broadcast.

Individual markers can also be toggled between shared and local in the marker editor (click any marker label to open it).

Colors

Each connected client is assigned a unique color, visible in the client list in the top bar. When you drop a marker, it inherits your client color by defaultmaking it easy to see at a glance who marked what.

Colors can be changed per-marker in the editor. Use them however makes sense for your ensemble:

- Per-performer (each player has their color)
- Per-function (red = problem, green = landmark, blue = solo)
- Per-section (color-code movements or scenes)

The system doesnt impose meaning on colorstheyre a visual vocabulary for you to define.

Marker controls

The marker tools appear in the top bar:

Button	Function
(arrow)	**Drop marker** at current playhead position
M (marker icon)	**Toggle visibility** of all markers on/off
(network icon)	**Toggle sharing** when active (blue), new markers sync to all clients

Button	Function
/ (transport)	**Fast forward / rewind** between markers and rehearsal marks
Arrow Up / Down	**Keyboard navigation** between markers and rehearsal marks

Dragging markers

Markers can be **dragged in two dimensions**:

- **Horizontal drag** moves the marker along the timeline
- **Vertical drag** slides the label up or down along the marker line

This lets you position a label anywhere on the score at the top for unobtrusive markers, or in the center of the page for prominent waypoints visible from across the room.

Editing markers

Click any marker label to open the **marker editor**:

- **Label** edit the text
- **Show vertical line** toggle the full-height line on/off
- **Share with all clients** switch between shared and local
- **Size** adjust font size (1032px) for prominence
- **Color** pick from presets or choose a custom color
- **Delete** remove the marker

Large font sizes combined with central vertical positioning create bold structural landmarks that performers can see at a glance during performance.

Relationship to other timing tools

Markers work alongside Oscillas other timing features:

- **Rehearsal marks** Navigation points defined in the SVG score using Oscillas cue syntax. These require authoring `cue_rehearsal(...)` IDs in your SVG file. Markers offer similar landmark functionality without any SVG editing they live in the interaction layer, not the score file. Since both rehearsal marks and markers share the same navigation system (arrow keys, fast forward/rewind, rehearsal popup), they function interchangeably as structural waypoints the difference is only in how theyre created.
- **Countdown timer sequences** For non-timeline-based structures, the timer system can trigger sequences of countdowns (e.g., “5 minutes 3 minutes 2 minutes done”). Each timer slot has an optional **onComplete** field that accepts any cue expression. Setting `nav(scroll@intro)` or `nav(mark@bridge)` as a slots onComplete action causes the playhead to jump to that marker when the countdown finishes. This means you can build an entire performance structure from markers and timers alone no SVG authoring required.
- **`nav()` cues** Named markers can be targeted from SVG cue syntax or from timer onComplete fields. See *Navigation targets* below.

Navigation targets

Named markers (any marker whose label has been edited from the default “m”) can be jumped to programmatically using Oscillas cue syntax:

Syntax	Behavior
`nav(scroll@myMarker)`	Jumps to rehearsal mark “myMarker” first; if not found, jumps to the user marker with that name. Playback auto-resumes.
`nav(mark@myMarker)`	Jumps directly to the user marker (skips rehearsal marks). Playback auto-resumes.

Syntax	Behavior
`nav(markPaused@myMarker)`	Jumps to the user marker and stays paused.
`nav(scrollPaused@myMarker)`	Jumps to rehearsal mark or marker and stays paused.

These expressions work anywhere a cue string is accepted:

- **In SVG cue IDs** embed navigation in the score itself
- **In timer onComplete fields** jump to a marker when a countdown slot finishes
- **In button triggers** wire a `button()` cue to jump to a specific marker

When multiple markers share the same name, the leftmost one (by score position) is used. A console warning is logged when duplicates are found.

Keyboard and transport navigation

Markers integrate into Oscillas transport navigation alongside rehearsal marks:

Control	Action
Arrow Up	Jump to the next navigation point (marker or rehearsal mark) after the current playhead position
Arrow Down	Jump to the previous navigation point before the current playhead position
Fast Forward button	Same as Arrow Up
Fast Rewind button	Same as Arrow Down
Rehearsal popup	Shows all rehearsal marks and named markers; click any to jump

All navigation points rehearsal marks and markers are sorted by their position on the score. Arrow keys and transport buttons walk this unified list based on where the playhead currently is, not on a stored index. This means navigation stays correct regardless of how the playhead got to its current position (manual seeking, cue jumps, timer actions, etc.).

Default unnamed markers (“m”) are included in keyboard navigation. All markers with a valid position are navigable.

The session layer

Markers, timers, and the `nav()` cue together form what could be called a **session layer** a way to construct temporal structure on top of any score without editing SVG files.

A typical session-layer workflow:

1. **Load any image or graphic as a score** it doesnt need cue IDs, rehearsal marks, or any Oscilla-specific markup.
2. **Drop and name markers** create structural waypoints: “intro”, “bridge”, “climax”, “ending”.
3. **Build a countdown sequence** create timer slots matching your planned section durations.
4. **Wire onComplete actions** set each slots onComplete to `nav(scroll@nextSection)` so the playhead automatically jumps forward when each timer finishes.
5. **Optionally add `ui()` calls** use `ui(#layerName, visible:true)` in onComplete fields to show/hide visual layers at section boundaries.

The result is a fully navigable, timed performance structure built entirely from the interaction layer. The score provides visual material; markers and timers provide the temporal framework.

Typical workflow

Before rehearsal: 1. Load the score 2. Set the duration/tempo 3. Drop markers to outline the set structure 4. Assign colors or labels as needed 5. Optionally: build a countdown sequence with onComplete actions targeting your markers

During rehearsal: 1. Start playback 2. Drop markers on-the-fly when something notable happens 3. Use Arrow Up/Down to jump between markers without stopping 4. Dont stop keep playing

After the run-through: 1. Click markers (or use arrow keys) to jump back to flagged moments 2. Discuss, adjust, make notes 3. Delete or reposition markers as the interpretation evolves

Before performance: 1. Clear rehearsal markers (or hide them) 2. Keep only the structural landmarks you want visible 3. Perform with a shared map that guides without dictating

---

Markers are intentionally simple to create drop, name, drag, done. But they connect into Oscillas deeper systems: theyre navigable by keyboard and transport controls, targetable by `nav()` cues, and triggerable from timer completion actions. This makes them a bridge between casual annotation and structured performance start with sticky notes on a timeline, and wire them into a full temporal framework when youre ready.

Timers

Overview

Oscillas Timer system provides flexible time-tracking tools for performers, conductors, and ensembles. At its core is the **Auxwatch** an auxiliary timer that can function as a simple stopwatch, countdown timer, or sequenced section timer for structuring musical time.

The Timer system is designed for: - **Solo performers** tracking practice or performance duration - **Ensembles** synchronizing timed sections across multiple devices - **Improvisers** using time-based structures without traditional notation - **Conductors** managing sectional rehearsals or timed cues - **Session-layer workflows** where timer slots trigger navigation to markers, creating automated performance structures

---

Accessing the Timer

Click the **stopwatch display** in the top bar to open the fullscreen Timer view.

Fullscreen Timer Display

The fullscreen view shows three columns at the bottom:

Element	Description
Mini Timer	Swappable display (Performance Time or Timer)
Clock	Current time of day
Countdown Controls	Play button and sequence selector with red ring indicator

The large central display shows elapsed time or active countdown.

View Modes

Click the **button** (top left) to cycle through display modes:

1. **Solid** Dark background, light text. Clean, high-contrast display.
2. **Blur** Score visible but blurred behind timer. Maintains context.
3. **Transparent** Timer overlays score directly. Ideal for minimal cue scores.

Press **Escape** or click to exit fullscreen.

Countdown Controls

The rightmost column in the fullscreen view contains the countdown controls:

Element	Description
/	Play/Stop button for the selected sequence or cue
Selection text	Shows currently selected sequence or cue name (click to change)
Red ring	Click to open the Countdown Sequences editor

Quick Playback

1. Click the **selection text** to open the dropdown
2. Choose a sequence or individual cue
3. Click to start

The selection remembers your last choice between sessions.

Playlist Behavior

The countdown controls work like a playlist: - After a countdown completes, the selection automatically advances to the next item - When reaching the end of the list, it wraps back to the beginning - This makes it easy to step through multiple sequences or cues manually

---

Swapping Timers

Click the **Mini Timer** box (left column) to swap positions: - Performance Time and Timer swap between main and mini display - Choose which time reference dominates the display

---

Countdown Sequences

Click the **red ring** to open the Countdown Sequences editor.

Creating Sequences

A **sequence** is a named group of timed sections that play consecutively. Each section can optionally trigger an action when it finishes.

Example Sonata Form:

```
Sequence: "Sonata"
├── Exposition      120s  onComplete: nav(scroll@development)
├── Development     180s  onComplete: nav(scroll@recap)
└── Recapitulation   90s
```

Editor Controls

Control	Function
+ Add Sequence	Create a new sequence group
Sequence Name	Text field to name the sequence
(on sequence)	Play entire sequence from start
	Delete sequence
Loop	Number of times to repeat (0 = infinite)
Then	Chain to another sequence when complete
+ Add Cue	Add a timed section to sequence
Cue Name	Name displayed during countdown
Duration (s)	Length in seconds
onComplete	Cue expression to trigger when this slot finishes (e.g.`nav(scroll@B)`)
(on cue)	Play single cue only
	Delete cue

Looping

The **Loop** control determines how many times a sequence repeats:

Value	Behavior
1	Play once (default)
2, 3,	Repeat that many times
0	Loop infinitely until manually stopped

Example: A 3-cue sequence with Loop = 2 plays:

```
Cue 1 → Cue 2 → Cue 3 → Cue 1 → Cue 2 → Cue 3 → stop
```

Chaining (Sequence of Sequences)

The **Then** dropdown lets you chain sequences together:

```
"Warm-up" (Loop: 1, Then: "Main Set")
    → plays once, then automatically starts "Main Set"

"Main Set" (Loop: 3, Then: "Cool-down")
    → plays 3 times, then starts "Cool-down"

"Cool-down" (Loop: 1, Then: — stop —)
    → plays once, then stops
```

This creates a **playlist** of sequences without manual intervention.

Per-Cue Completion Actions (onComplete)

Each cue slot has an optional **onComplete** field a cue expression that fires when that slots countdown reaches zero. This connects the timer system to Oscillas navigation and UI systems.

onComplete expression	What happens
`nav(scroll@B)`	Jump to rehearsal mark or marker “B”, auto-resume playback
`nav(mark@bridge)`	Jump to user marker “bridge” (skips rehearsal marks)
`nav(markPaused@ending)`	Jump to marker “ending”, stay paused
`ui(#brass, visible:true)`	Show a visual layer
`page(C)`	Switch to page C
`audio(src:gong.wav)`	Play a sound

Any valid Oscilla cue expression works in the onComplete field. The action fires on **all connected clients** simultaneously (since the server owns the timer).

Example timed sections with automatic navigation:

```
Sequence: "Performance"
├── Intro        120s  onComplete: nav(scroll@development)
├── Development  180s  onComplete: nav(scroll@climax)
└── Climax        90s  onComplete: nav(scroll@ending)
```

When each sections countdown finishes, the playhead automatically jumps to the next marker. The score scrolls itself through the performance structure.

onComplete actions also work with single cues (not just sequences). If you start a standalone countdown that has an onComplete expression, it fires when that countdown reaches zero.

Playback Behavior

When a countdown runs: - Main display shows **section name** as label - Time counts down in **MM:SS** format - Sequences auto-advance to next section - **onComplete** action (if set) fires when each section finishes - Single cues return to normal timer when complete - Play button changes to (stop)

Network Synchronization

The Timer system automatically synchronizes across all connected clients.

How It Works

- When any client starts a countdown, **all clients** see the same countdown
- Sequences are shared automatically when clients connect
- Time is synchronized using server timestamps all displays show the same remaining time

What Syncs

Event	Behavior
Client connects	Receives current sequences from server
Sequence created/edited	Changes broadcast to all clients (including onComplete fields)
Countdown started	All clients show synchronized countdown
Cue slot completes	Server broadcasts onComplete action; all clients execute it simultaneously
Countdown stopped	All clients return to normal display

Server-Owned Timer

The countdown timer is **server-owned**, meaning: - The server maintains the authoritative timer state - Clients send commands (start/stop) to the server - Server broadcasts state to all clients - Late-joining clients receive current countdown state immediately

This ensures all performers see exactly the same time, regardless of when they connected.

Import & Export

Sequences can be saved and loaded as JSON files.

Export

Click **Export** to download `countdown-sequences.json` containing all sequences.

Example JSON:

```
[
  {
    "name": "Warm-up",
    "loop": 1,
    "chain": 1,
    "cues": [
      { "name": "Stretch", "seconds": 120, "onComplete": null }
    ]
  },
  {
    "name": "Sonata",
    "loop": 1,
    "chain": null,
    "cues": [
      { "name": "Exposition", "seconds": 120, "onComplete": "nav(scroll@development)" },
      { "name": "Development", "seconds": 180, "onComplete": "nav(scroll@recap)" },
      { "name": "Recapitulation", "seconds": 90, "onComplete": null }
    ]
  }
]
```

Import

Click **Import** and select a JSON file to load sequences.

Note: Import merges with existing sequences by ID. Sequences are automatically shared to other connected clients after import.

Use Cases

1. Structured Improvisation

Create a sequence of timed sections:

```
"Free Improv Structure"
├── Sparse Texture     60s
├── Build              90s
├── Peak               45s
└── Decay             120s
```

All performers see the same countdown, enabling synchronized structural changes without traditional cues.

2. Session Layer Markers + Timers

Use markers and timer onComplete actions to build a complete performance structure without editing SVG:

1. Load any image as a score
2. Drop and name markers at key positions: “intro”, “development”, “climax”, “ending”
3. Create a countdown sequence with onComplete actions:

```
"Performance"
├── Intro          120s  onComplete: nav(scroll@development)
├── Development    180s  onComplete: ui(#overlay, visible:true)
├── Climax          90s  onComplete: nav(scroll@ending)
└── Ending          60s
```

The playhead jumps between markers automatically as each section finishes. Layers can appear/disappear at section boundaries. No SVG cue authoring needed.

3. Rehearsal Sectionals

Time individual sections during rehearsal:

```
"Rehearsal Plan"
├── Warm-up            300s
├── Exposition work    600s
├── Break              300s
└── Development work   600s
```

4. Performance Timing

For pieces with strict durational requirements:

```
"Competition Piece"
├── Movement I    480s
├── Pause          30s
└── Movement II   360s
```

5. Minimal Cue Scores

Use **Transparent mode** with countdown sequences as a time-based score: - Score contains only markers - Timer overlays showing current section - onComplete actions handle navigation between sections - Performers follow time structure, not traditional notation

---

Technical Notes

Storage

- Sequences stored in `localStorage` as `oscilla.countdownSequences`
- Each cue object contains `name`, `seconds`, and optional `onComplete` (cue expression string or null)
- Selected sequence/cue stored as `oscilla.countdownControls.type` and `.index`
- Persists between sessions

Timer Precision

- Display updates every 250ms (synced with server broadcast)
- Network sync uses `Date.now()` timestamps
- Typical sync accuracy: 50-200ms depending on network

Performance Time vs Timer

Timer	Behavior
Performance Time	Linked to transport. Pauses on manual stop, continues during musical pauses (cue-triggered).
Timer (Auxwatch)	Independent stopwatch. Can be used for countdowns. Swappable with Performance Time.

Keyboard Shortcuts

Key	Action
Escape	Exit fullscreen timer
Click stopwatch	Enter fullscreen
Click mini timer	Swap timer positions
Click red ring	Open countdown editor
Click selection text	Open sequence/cue dropdown

Summary

The Timer system transforms Oscilla into a flexible time-structuring tool:

- **Simple**: Click stopwatch fullscreen timer
- **Quick access**: Play button and dropdown right in the fullscreen view
- **Sequenced**: Create named countdown sections with looping and chaining
- **Actionable**: Per-cue onComplete fields trigger navigation, UI changes, or any cue expression
- **Synchronized**: All clients automatically stay aligned
- **Portable**: Import/export sequences as JSON

Combined with markers, the Timer system enables a **session layer** approach building complete performance structures from the interaction layer without editing score files. Whether for strict durational works, timed improvisations, or rehearsal management, the Timer provides a unified interface for musical time.

drag() Moveable Score Elements

Make any SVG group draggable in the browser. Author a palette of visual components in Inkscape, then arrange and rearrange them during rehearsal or performance. Positions persist and sync across all connected clients.

Quick Start

In Inkscape, give a group the ID `drag(1)`:

```
<g id="drag(1)">
  <circle cx="100" cy="100" r="30" fill="red" />
</g>
```

Open in browser click and drag the group anywhere on the score.

Syntax

```
drag(1)                          // basic draggable
drag(1, osc:1)                   // + send position via OSC
drag(1, osc:/my/custom/addr)     // + custom OSC address
```

Parameter	Description
`1`	Enable drag (required)
`osc:1`	Send x,y position via OSC to `/oscilla/drag/{id}`
`osc:/addr`	Send to a custom OSC address

Combining with Animations

Drag works alongside any cue or animation. Use a **compound ID** (space-separated) to combine drag with scale, rotate, o2p, etc:

```
// drag + scale
<g id="scale(values:[1,1.5,1], dur:2) drag(1)">

// drag + rotate + OSC
<g id="rotate(dir:1, dur:4) drag(1, osc:1)">

// drag + slow rotate with uid
<g id="rotate(dir:1, dur:30, uid:myThing, osc:1) drag(1)">
```

The animation plays on the element while you can still grab and reposition the whole group. They dont interfere with each other.

Nesting

Use the documented nesting pattern to keep cues separate:

```
<g id="drag(1)">
  <g id="scale(values:[1,2,1], dur:3)">
    <rect ... />
  </g>
</g>
```

Behaviour stacks outer inner. The outer group is draggable, the inner group scales. Shapes inherit all enclosing cues.

With propagate

Each child in a propagated group gets its own independent drag handler:

```
<g id="propagate(rotate(dir:1, dur:rnd(2,8)) drag(1))">
  <circle cx="50" cy="50" r="20" />
  <circle cx="150" cy="50" r="20" />
  <circle cx="250" cy="50" r="20" />
</g>
```

Each circle rotates independently AND can be dragged independently.

Use Cases

Modular score layout Author a collection of musical fragments as separate groups in Inkscape. Add `drag(1)` to each. In the browser, arrange them spatially to create the performance layout. Positions are saved close the browser and reopen, everything is where you left it.

Performer interaction Performers drag their assigned elements during performance, creating a collaboratively arranged visual score in real time. All clients see the changes immediately.

Spatial control via OSC Use `drag(1, osc:1)` and route x,y position to synthesis parameters drag a visual element to pan a sound source, control filter frequency by vertical position, etc.

Rehearsal tool Quickly rearrange sections of a score during rehearsal without going back to Inkscape. Reset all positions when youre done: open the browser console and type `resetDragPositions()`.

Persistence

Drag positions are automatically saved and shared:

- **Across page reloads** positions survive browser refresh
- **Across clients** drag on one screen, all connected screens update
- **Across server restarts** positions saved to JSON in your project folder
- **Per project** each project has its own saved positions

Positions are stored in `scores/myProject/drag-positions.json` alongside your other project files. This file is auto-generated you dont need to create or edit it.

Resetting Positions

To move everything back to the original authored positions:

```
// Browser console
resetDragPositions()
```

This clears all drag positions for the current project on all connected clients and deletes the saved JSON file. Elements return to where they were placed in Inkscape.

Workflow

```
1. Author shapes & groups in Inkscape
2. Add drag(1) to group IDs (XML Editor: Ctrl+Shift+X)
3. Optionally combine with animations:
     rotate(dir:1, dur:4) drag(1)
4. Save SVG → Open in browser
5. Drag elements to arrange the score
6. Positions sync to all clients + saved to disk
7. resetDragPositions() to start fresh
```

Live Console Live Coding & Inspection

The Live Console lets you type and execute Oscilla DSL expressions in real time, directly against elements in the running score. It also provides a signal monitor showing all active control-plane values.

Use it for:

- **live coding** type DSL, execute it, hear/see the result
- **testing** try parameter changes without editing the SVG
- **debugging** inspect signal flow and animation state
- **performance** build and modify animations during a show

Opening the Console

Click the **>_** button in the top bar. The console panel opens on the right side of the screen. Click the button again (or the X) to close it.

Panel Layout

The panel has four sections, top to bottom:

Target

Shows which SVG element your DSL will be applied to. You can set the target by:

- **Picking** click "pick", then click any element in the score. The element is highlighted and its uid populates the target field.
- **Typing** enter a uid or element id in the target field and press Enter. Partial matches against the animation registry are supported.

The info line below the field shows the elements tag, id, uid, kind (rotate/scale/o2p/), and whether it is currently running.

Animation cues (`rotate`, `scale`, `o2p`, `color`, `fade`) require a target element. Other cues (`synth`, `audio`, `speed`, `nav`, `osc`, `stop`, `pause`) do not they execute without a target.

Editor

A text area where you type DSL expressions. Execute with:

- **Ctrl+Enter** run the current line (where the cursor is)
- **Ctrl+Shift+Enter** run all lines

When you pick an element, its existing DSL expression (from its SVG id) is pre-filled in the editor so you can modify and re-apply.

Output

Shows the result of each execution: green for success, red for parse errors or missing targets.

Signals

A live-updating view of all values on the ParamBus (the control plane). Updated at 5 fps. Use the filter field to narrow by path prefix e.g.type `rotate` to see only rotation signals, or a uid to see a specific elements outputs.

Usage Examples

Modify a running rotation

1. Click **pick**, click the rotating element in the score
2. The editor fills with the current DSL, e.g. `rotate(dir:1 dur:30 uid:spinner1 osc:1)`
3. Change `dur:30` to `dur:5` and press **Ctrl+Enter**
4. The element now rotates at the new speed

Start a synth (no target needed)

Type directly in the editor:

```
synth(freq:440 amp:0.3 wave:sine)
```

Press Ctrl+Enter. No target element required.

Change playback speed

```
speed(0.5 dur:4)
```

Ramps playback to half speed over 4 seconds.

Navigate to a rehearsal mark

```
nav(B)
```

Jumps to rehearsal mark B.

Trigger an audio cue

```
audio(clicks.wav vol:0.6 loop:1)
```

Layer multiple cues

Write multiple lines and run all with Ctrl+Shift+Enter:

```
rotate(dir:-1 dur:8 uid:orb1)
synth(freq:orb1.norm[200,800] amp:0.4)
speed(1.5)
```

Keyboard Behaviour

While the cursor is in any field or the editor inside the live console panel, all keyboard events are captured by the panel. Arrow keys, spacebar, and other keys will not trigger transport controls or score navigation. Keyboard shortcuts resume normal behaviour when focus leaves the panel.

Signal Monitor

The signal monitor displays all values currently published to the ParamBus. Signals are published by running animations:

```
o2p:slider1.t       0.4523
rotate:spinner.norm  0.7210
synth:drone1.freq    442.00
```

Use the filter to focus on signals you care about. This is useful for verifying that cross-cue modulation sources are producing expected values before wiring them to targets.

Tips

- You can re-pick a different element at any time without closing the panel.
- The DSL you type follows exactly the same syntax used in SVG element IDs in Inkscape the same parser handles both.
- Animation cues applied via the console replace the previous animation on that element, just as re-triggering from the playhead would.
- The console does not modify the SVG file. Changes exist only in the running session.

Related

- Control & Modulation
- Cue System
- rotate(), scale(), o2p(), synth()

Chapter 3

Cues

Cues are the core mechanism through which an Oscilla score becomes executable. A cue is a short instruction embedded in the ID attribute of an SVG element. The most common trigger is the scrolling playhead when it crosses an element, the cue fires. But cues can also be triggered by interactive buttons embedded in the score, by timer sequences on completion of each unit, or typed directly into the live console during performance.

The cue syntax is designed to be readable and compact a minimal, accessible form of text-based authoring rather than traditional programming. A single element can carry multiple cues, and cues can reference other elements, enabling navigation, conditional branching, and structural control within the score. Cues are typed into Inkscapes Object Properties dialog, or built using the Oscilla Inkscape extension which provides a graphical interface for assembling cue expressions.

This section documents each cue type with its syntax, parameters, and usage examples.

`stop(...)` Toggle Playback (On / Off)

The `stop(...)` cue toggles the transport:

- if playback is running it stops
- if playback is already stopped it starts again

It does not end the score and it does not freeze the system.

Syntax

```
stop()
stop(uid:NAME)
stop(next:CUEID)
```

Parameters

param	required	description
`uid`		Only needed if multiple identical `stop(...)` cues appear
`next`		Trigger another cue immediately after `stop(...)` runs

Behaviour

- toggles playback (stop start)
- briefly ignores sync echoes
- `next(...)` triggers another cue **immediately** (does not wait for resume / spacebar)

Examples

```
stop()
stop(uid:s1)
stop(next:nav(End))
```

Related

- `pause(...)`
- `nav(...)`

pause() Temporarily Halt Playback

The `pause(...)` cue temporarily stops playback for a fixed duration. It may optionally display a countdown overlay and can automatically trigger another cue when finished.

Playback and synchronization are completely frozen during a pause.

Syntax

```
pause(dur:SECONDS, count:BOOL, next:CUEID, uid:NAME)
```

Daisy-chaining with `next(...)`

`next(...)` does **not** make the pause jump anywhere by itself.

Instead, it tells Oscilla:

> *"When the pause finishes, trigger this other cue."*

```
pause(dur:4, next:sectionB)
```

What happens:

1. pause
2. countdown (if enabled)
3. the cue named `sectionB` is triggered

If `sectionB` happens to be a navigation cue, the score may jump but it is the **navigation cue** doing the jump, not the pause system.

speed()

Controls the playback scroll speed of the score.

The `speed()` cue sets the playback speed multiplier. Speed values are **relative to the base speed** set in project preferences. Speed changes may be applied instantly or gradually over time.

Syntax

```
speed(
  value:<multiplier>,
  add:<offset>,
  dur:<seconds>,
  ease:<easing>,
  uid:<id>
)
```

Shorthand (positional):

```
speed(<multiplier>)
```

Parameters

- **value**
  Speed multiplier relative to the base speed.
  `1` = base speed, `0.5` = half base speed, `2` = double base speed.
- **add**
  Relative adjustment applied to the current speed multiplier.
- **dur**
  Duration of the speed change in seconds.
  If omitted, the speed change is applied immediately.

- **ease**
  Easing curve used during a timed speed change.
  Defaults to linear.
- **uid**
  Optional unique identifier for synchronization.

Base Speed Multiplier (Preferences)

The **base speed multiplier** is set in your projects `preferences.json`:

```
{
  "defaultPlaybackSpeed": 0.3
}
```

All `speed()` cues multiply against this base value:

Preference	Cue	Actual Speed
`0.3`	`speed(1)`	0.3 (1 0.3)
`0.3`	`speed(2)`	0.6 (2 0.3)
`0.3`	`speed(3)`	0.9 (3 0.3)
`1.0`	`speed(0.5)`	0.5 (0.5 1.0)

This allows you to: - Set a **global tempo** for the entire piece in preferences - Use `speed()` cues for **relative tempo changes** within the score - Adjust overall tempo without editing individual cues

---

Two Types of Speed Cues

1. Position-Based Speed Markers

Simple speed markers embedded in the SVG that apply **instantly** when the playhead crosses them:

```
speed(1)
speed(2)
speed(0.5)
```

These are extracted from SVG elements with IDs starting with `speed(`. They: - Apply immediately when crossed - Are position-aware (rewinding past a marker reverts to the previous speed) - Automatically sync with the server - Multiply against `baseSpeedMultiplier`

2. Triggered Speed Cues (cueSpeed)

Full cue syntax with ramping support:

```
cueSpeed(value:2, dur:3)
```

These support gradual transitions and are triggered like other cues.

Position Awareness

When you **rewind or seek** to a position before a speed marker, the system automatically applies the correct speed for that position:

```
Position:    0 -------- 5000 -------- 10000 -------- 15000
Markers:     speed(1)   speed(3)      speed(1)

Playhead at 7000 → speed = 3 × base
Rewind to 3000   → speed = 1 × base (automatically updated)
```

This ensures consistent playback regardless of navigation direction.

Behaviour

- Speed values multiply against `baseSpeedMultiplier` from preferences
- Position-based markers (`speed(x)`) apply instantly when crossed
- Triggered cues (`cueSpeed`) support ramping with `dur` parameter
- Rewinding past a speed marker reverts to the previous speed
- Speed changes sync to all connected clients via the server
- A new speed cue replaces any previously active speed change
- Speed changes affect scrolling, timing, and all time-based cues

---

Examples

Simple Position Markers (in SVG)

`speed(1)`

Base playback speed.

`speed(0.5)`

Half of base speed.

`speed(2)`

Double base speed.

Triggered Cues with Ramping

`cueSpeed(value:1.25, dur:3)`

Gradually change to 1.25 base speed over 3 seconds.

`cueSpeed(add:-0.5, dur:2)`

Gradually reduce speed multiplier by 0.5 over 2 seconds.

`cueSpeed(value:2, dur:6, ease:inOutQuad)`

Smooth, eased speed transition to 2 base speed.

Project Setup

preferences.json

```
{
  "defaultPlaybackSpeed": 0.3,
  "duration_minutes": 10,
  "darkMode": false
}
```

SVG Score

Place speed markers as elements with appropriate IDs:

```
<circle id="speed(1)" cx="100" cy="50" r="5" fill="blue"/>
<circle id="speed(3)" cx="5000" cy="50" r="5" fill="red"/>
<circle id="speed(1)" cx="10000" cy="50" r="5" fill="blue"/>
```

The visual element (circle, rect, path, etc.) marks the position; only the `id` matters for the cue system.

Server Synchronization

Speed changes are broadcast to all connected clients:

```
{
  type: "set_speed_multiplier",
  multiplier: 0.9,        // actual speed (cue × base)
  cueSpeed: 3,            // raw cue value
  baseMultiplier: 0.3,    // from preferences
  source: "position_watch"
}
```

The server maintains the authoritative speed state and syncs it at ~4Hz.

Debugging

Check speed state in browser console:

```
console.log("baseSpeedMultiplier:", window.baseSpeedMultiplier);
console.log("speedMultiplier:", window.speedMultiplier);
console.log("speedCueMap:", window.speedCueMap);  // position-based markers
```

Watch speed changes in real-time:

```
[speedWatch] pos=5000, cue=3, base=0.3, actual=0.90, current=0.30
[speedWatch] □ SPEED CHANGE: 0.30 → 0.90 (cue=3 × base=0.3)
```

Notes

- Speed cues modify global playback behaviour
- Only one speed is active at a time
- The performer sees only a cue symbol in the score
- The microsyntax appears only in the SVG `id` field
- Position-based speed is automatically restored when seeking/rewinding
- Base speed from preferences allows global tempo control

stopwatch *(with autostart support)*

Displays a stopwatch overlay above the score. Stopwatch overlays may show the **main stopwatch time** or create their own **independent stopwatch**.

This cue now supports **autostart**, meaning it can begin running automatically when the SVG is loaded in both page and scroll modes without requiring the playhead to cross.

Syntax

```
stopwatch(source:<main|new>, hold:<seconds>, scroll:<true|false>,
          offsetX:<pixels>, style:"<css rules>", trig:<auto|edge>)
```

Arguments

Arg	Values	Default	Behaviour
source	`"main"` / `"new"`	`"main"`	Use global clock or create/reset local stopwatch
hold	seconds	`0`	When >0, overlay fades+removes at timeout
scroll	`true` / `false`	`false`	Follows scrolling score or fixed overlay
offsetX	px	`0`	X-offset from cue location
style	CSS		Inline CSS, use `;` separated
trig	`"auto"` / `"edge"`	`"edge"`	`auto` means autostart on load

Behaviour

- `source:main` displays the **global** stopwatch time
- `source:new` starts (or resets) a **private timer**
- If `hold > 0`, the stopwatch fades after expiry
- If `hold = 0`, click dismisses it
- Multiple independent stopwatches can run concurrently
- Autostart requires no playhead crossing

Autostart (NEW)

Autostart activates when:

`trig:auto`

Example:

```
stopwatch(source:new,style:"font-size:3em;color:red",trig:auto)
```

Autostart behaviour:

- Works in **page overlay** and **scroll** modes
- Executed **after SVG and DOM are fully initialized**
- Overlay appears immediately
- Does not require playhead position
- Does not use trigger dedupe

Autostart Examples

Main stopwatch, always visible

```
stopwatch(source:main,style:"font-size:2.5em;",trig:auto)
```

Independent timer with 10s hold

```
stopwatch(source:new,hold:10,style:"font-size:2em;",trig:auto)
```

Scrolling stopwatch

```
stopwatch(source:new,scroll:true,trig:auto)
```

Stylised overlay

```
stopwatch(source:new,
          style:"font-size:4em;color:#0ff;text-shadow:0 0 5px #000;",
          trig:auto)
```

Edge-triggered Examples (non-autostart)

```
stopwatch(source:new)
stopwatch(source:main,scroll:true)
stopwatch(source:new,hold:8,offsetX:-30)
```

Developer Notes

- Autostart runs **once**, after initialization
- Stopwatch overlay is a **DOM absolute positioned element**
- High stacking ensured using:
  `div.style.zIndex = "99999"`
- Autostart does not use `triggeredCues`

Known Limitations

- Overlap of multiple autostarts is a user-design choice
- Very large fonts may clip on small screens

metronome *(with autostart support)*

Triggers a visual and/or audible metronome marker at the cues location or a specified `target:` element. The metronome can follow scrolling objects, stay fixed to the viewport, and optionally send OSC messages.

The metronome **can also autostart**, meaning it runs automatically when the SVG is loaded in either page mode or scroll mode without needing a cue trigger crossing.

Syntax

```
metro(bpm:<number>,beats:<number>,visual:<type>,audio:<0|1>,
          position:<fixed|scrolling>,osc:<0|1>,uid:<string>,
          target:<uid>,hideTarget:<0|1>,showcount:<0|1>,hold:<seconds>,
          colour:<csscolour>,size:<pixels>,trig:<auto|edge>)
```

Parameters

Parameter	Type	Default	Description
bpm	number	`120`	Beats per minute. Controls metronome speed.
beats	number	`4`	Number of beats per cycle.
visual	string	`"circle"`	Visual style (`circle`, `hex`, future shapes).
audio	`0` / `1`	`0`	Enables click sound.
position	`"fixed"` / `"scrolling"`	`"fixed"`	Follow-scrolling or fixed overlay.
osc	`0` / `1`	`0`	Publishes OSC beat messages.
uid	string	`"default"`	Unique metronome identifier.
target	uid		Anchor element (scrolling mode only).
hideTarget	`0` / `1`	`1`	Hide target element when starting.
showcount	`0` / `1`	`1`	Show count number inside shape.
hold	seconds	`0`	Stop after defined duration.
colour	CSS color	`"red"`	Circle colour.
size	pixels	`50`	Circle diameter.
trig	`"auto"` / `"edge"`	`"edge"`	Whether to autostart or trigger when playhead crosses.

Behaviour

- Runs as **overlay DOM** element (not part of the SVG).
- Follows scrolling or stays screen-fixed depending on `position:`.
- Supports **audio click** and **OSC output**.
- Supports **multiple concurrent** metronomes (UID required).
- Can target another SVG element and hide it on activation.

Autostart Mode (NEW)

When is a metronome autostarted?

Whenever the cue contains:

```
trig:auto
```

Example:

```
cue:metro(bpm:90,visual:hex,uid:m21,trig:auto)
```

When does autostart happen?

- In **page overlay mode** (during PDF-style view)
- In **scroll mode** (timeline view)
- **After the SVG and DOM are initialized**

- **Before** the animation clock starts

What does autostart actually do?

Autostart metronomes fully initialise as if they had been triggered by the scrolling playhead:

- Overlay DOM is created
- Beat scheduler starts
- UI is visible
- OSC messages begin
- Audio (if enabled) clicks on beat 1,2,3,4

Multiple autostarts supported

Each metronome UID becomes its **own independent instance**.

Example cues

Description	Example
Autostart metronome at load	`metro(bpm:90,visual:hex,trig:auto,uid:m1)`
Two autostarts	`metro(bpm:70,trig:auto,uid:A) + cue:metro(bpm:110,trig:auto,uid:B)`
Standard (on crossing), no autostart	`metro(bpm:100,visual:circle,uid:M)`
Scroll-following autostart	`metro(bpm:120,position:scrolling,target:x1,trig:auto`

OSC Message Example

```
/oscilla/metro
{
  "uid": "m21",
  "client": "rob",
  "bpm": 90,
  "beat": 3
}
```

Implementation Notes (for developers)

- Autostart hooks run **after** DOM insertion and SVG setup.
- Autostart routines are separate from cue crossing logic:
  - No tracking in `triggeredCues`
  - No playhead position checks required
- Overlay visibility ensured via:
  `div.style.zIndex = "99999";`

Known Limitations

- Multiple autostarts in high BPM (>200) may need scheduler improvements.
- OSC feedback not currently aware of phase differences between UIDs.
- Appearance position is based on CSS absolute, not SVG hitbox.

page() navigate pages or scrolling positions in the score

Triggers navigation between SVG score pages or scroll positions.
The cue can load a single page, sequence multiple pages, loop sections,
choose randomly from options, randomize order, or define composite playlists using pattern-based syntax. It forms the structural backbone for cue-based navigation and timeline control in Oscilla scores.

Syntax

page() page(Pseq([:,:],)) page(Pxrand([„],)) page(Pshuf([„],)) page(Prand([„],)) page(Pchoose([„])) page(Pseq([page1:3, page2:3],1), after:mode(scroll@F)) page(Pseq([Pseq([page1:2,page2:2],2),page3:4],1))

Arguments

Argument	Description
page	ID of a single page or scroll section to load
Pseq	sequential playback of listed pages or subpatterns
Prand	random selection with replacement
Pxrand	random order, reshuffled each repeat cycle
Pshuf	random order, shuffled once and repeated as fixed order
Pchoose	choose one random page from a list (per trigger)
repeats	number of cycles for the pattern (`inf` = infinite)
:	optional duration for each page (in seconds)
after:	optional post-action (e.g.`mode(scroll@F)`)
mode:	switch playback display mode (`scroll` or `page`)
@uid	rehearsal mark or cue ID to jump to after sequence completes

Behavior

- `page(page1)` jumps immediately to the given page or scroll position.
- `page(Pseq([page1:2,page2:2],3))` loops between pages 1 and 2 three times.
- `page(Prand([page1,page2,page3],5))` selects five random pages with replacement.
- `page(Pshuf([page1,page2,page3],2))` shuffles once and repeats the same order twice.
- `page(Pxrand([page1,page2,page3],2))` reshuffles the list for each repetition.
- `page(Pchoose([page1,page2,page3]))` chooses one random page each time the cue is triggered.
- Durations can be specified using `pageId:seconds` (e.g.`page1:5`).
- The `after:` clause defines what happens when the sequence ends (e.g.returning to scroll mode).
- `after:mode(scroll@F)` jumps to scroll mode at rehearsal mark F and resumes playback.
- `after:mode(scrollPaused@F)` jumps to scroll mode at mark F and stays paused.
- All navigation is synchronized across connected clients.

Examples

page(page1) page(Pseq([page1:2,page2:2],3)) page(Prand([page1,page2,page3],4)) page(Pxrand([pageA,pageB,pageC] page(Pshuf([pageIntro,pageMid,pageEnd],1)) page(Pchoose([pageA,pageB,pageC])) page(Pseq([page1:3],1), after:mode(scroll@F)) page(Pseq([page1:3],1), after:mode(scrollPaused@F)) page(Pseq([Pseq([page1:2,page2:2],2),pa

Notes

- Pattern parentheses `()` are required for lists and sequences.
- Repetition counts follow the pattern syntax (`Pseq([...],<count>)`).

- Durations follow the page name with a colon (`page1:4` = 4 s).
- The `@` suffix defines a rehearsal mark or cue ID to jump to after completion.
- `Pshuf` and `Pxrand` follow SuperCollider semantics:
  - **Pshuf**: shuffled once, same order repeated.
  - **Pxrand**: reshuffled on each repeat.
- `after:` supports `scroll` and `scrollPaused` modes; `:hold(n)` extension planned.
- All page and mode changes are fully synchronized across connected clients.

nav() Navigation and Structural Movement

Navigates to a page, rehearsal mark, or scroll mode location.

Syntax

```
nav(<actionOrPage>[ @ <target> ], repeats:<N>?, uid:<id>?)
```

The first argument determines the navigation behavior. It may be: - `scroll` = switch to scrolling mode and continue playback - `scrollPaused` = switch to scrolling mode but do not resume playback - `<pagename>` = jump directly to a page or rehearsal mark

The optional `@<target>` is used when scroll mode needs to jump to a specific place.

Parameters

Parameter	Description
`actionOrPage`	`"scroll"`, `"scrollPaused"`, or a page/rehearsal name (e.g., `page3`, `Coda`)
`target`	Optional rehearsal/page anchor used only with scroll modes
`repeats`	Optional number of times to re-trigger a scroll jump before traversal continues
`uid`	Unique cue identifier required for repeats; recommended whenever nav() may re-trigger

Examples

```
// direct page navigation
nav(page3)
nav(page3, uid:p3)
nav(Coda)

// scroll navigation
nav(scroll@A)
nav(scrollPaused@B)
nav(scroll)
nav(scrollPaused)

// repeatable scroll navigation
nav(scroll@G, repeats:3, uid:g1)
// jump to G three times; afterwards traversal continues normally
```

UID rule

UID is optional unless repeats are used or multiple identical `nav()` expressions appear. It uniquely identifies navigation state across jump cycles.

Notes

- The performer sees only the visual cue symbol in the score.
- The microsyntax remains in the SVG id and is not shown in the visible notation.

`button()` Clickable Buttons Inside the Score

The `button()` helper lets you place **HTML buttons** aligned to SVG markers in your score.
Each `button()` marker is an SVG element with an ID that encodes a CueDSL expression; at runtime, Oscilla replaces that marker with a positioned HTML button which can trigger any cue.

Basic Syntax

```
id="button(
  trigger:audio(src:noise, amp:0.9, loop:6, toggle:false, uid:noiseA),
  style(size:\"120x45\", color:\"#333\", textcolor:\"#fff\", fontsize:36, active:\"flash\", uid:btnNoise)
)"
```

General form:

```
button(
  trigger: <cue expression>,
  style(...optional style keys...),
  offsetX:<number>,
  offsetY:<number>
)
```

- **`trigger:`** required. Any valid CueDSL expression (e.g.`nav(...)`, `audio(...)`, `page(...)`, `repeat(...)`, etc.).
- **`style(...)`** optional styling for the HTML button.
- **`offsetX`, `offsetY`** optional pixel offsets from the SVG marker position.

The `button()` expression must appear as the **ID** of an SVG object:

```
<rect
  id="button(
    trigger:audio(src:noise, amp:0.9, loop:6, toggle:false, uid:noiseA),
    style(size:\"120x45\", color:\"#333\", textcolor:\"#fff\", fontsize:36, active:\"flash\", uid:btnNoise)
  )"
  x="100" y="40" width="20" height="20"
/>
```

Trigger

The `trigger:` field receives the full cue expression that should run when the button is clicked:

```
button(
  trigger:nav(page:2)
)

button(
  trigger:pause(4)
)

button(
  trigger:audio(src:noise, amp:0.9, loop:6, toggle:false, uid:noiseA)
)
```

At runtime, clicking the button calls `handleCueTrigger(...)` with the parsed trigger AST.

Style Block

The `style(...)` block controls the appearance and behaviour of the HTML button. For example:

```
button(
  trigger:audio(src:noise, amp:0.9, loop:6, toggle:false, uid:noiseA),
```

```
  style(
    label:\"Noise A\",
    size:\"120x45\",
    color:\"#333\",
    textcolor:\"#fff\",
    font:\"Inter\",
    fontsize:36,
    active:\"flash\",
    uid:btnNoise
  )
)
```

Supported style keys

Key	Type	Description
`label`	string	Button text override
`size`	`"WxH"`	Width height in pixels, e.g.`"120x45"`
`color`	string	Background color (CSS color)
`textcolor`	string	Text color
`font`	string	Font-family name
`fontsize`	number	Font size in px
`active`	string	`"flash"` to briefly highlight on click
`uid`	string	Identifier for debugging / CSS hooks

Notes:

- If `style(label:...)` is present, it overrides any auto-generated label.
- If `style(active:\"flash\")` is set, the button gets the `oscilla-button-flashable` class and flashes briefly on click.

Automatic Label Fallback

If no `style(label:...)` is provided, Oscilla derives a label from the trigger:

Priority (roughly):

1. `style(label:\"…\")` override
2. `trigger.uid` (if present)
3. `trigger.action` (e.g.`"nav"`, `"audio"`, `"page"`)
4. `trigger.page` (for page cues)
5. `"button"` as a final fallback

This means even minimal buttons like:

```
button(trigger:nav(page:3))
```

will still get a readable label.

Positioning

Each `button()` marker is an SVG element that defines where the HTML button should appear.

Runtime behaviour:

1. The engine looks for a suitable container element in the DOM:
   - `#singlePage-content`
   - `#pageOverlay`
   - `#scoreInner`

2. It computes the SVG markers bounding box and transform, converts that to screen coordinates, then to container-local coordinates.
3. It creates a `<button>` element with `position:absolute` and sets its `left` and `top` to match the marker position.
4. If the marker is **not** part of a `reuse(...)` clone, the SVG marker is hidden (via `visibility:hidden`) so only the HTML button is visible.

Offsets

You can nudge the button relative to the marker with `offsetX` and `offsetY` in pixels:

```
button(
  trigger:nav(page:4),
  style(label:\"Next Page\"),
  offsetX:10,
  offsetY:-8
)
```

Containers and Page Mode

Buttons are designed primarily for **page mode** overlays:

- In page mode, `button()` elements will be placed inside `singlePage-content` or `pageOverlay`.
- If the chosen `containerEl` accidentally resolves to an SVG element, Oscilla falls back to `#singlePage-content`.

This ensures that buttons remain correctly aligned even when the score scrolls or resizes.

Click Behaviour

On click, a button:

1. Prevents default and stops event propagation.
2. Optionally flashes (`active:\"flash\"`).
3. Invokes `handleCueTrigger(triggerAst, false, true, markerElement)` with the trigger AST.

This means `button()` is essentially a **visual trigger surface** for any existing cue type.

Managing Buttons Programmatically

Destroy all generated buttons

```
destroyAllCueButtons();
```

- Removes all `.oscilla-cue-button` elements from the DOM.
- Calls an internal `_destroyCueButton` if present (for cleanup).
- Useful when unloading or reloading a score.

Hide all `button()` markers in the SVG

```
hideAllButtonPlaceholders(svgRoot);
```

- Finds all markers with `id^="button("` and sets:
  - `opacity: 0`
  - `pointer-events: none`
- This hides the original SVG placeholders (including clones) while leaving the HTML buttons visible.

Summary

- Use `button(trigger:..., style(...))` to add clickable controls into Oscilla scores.
- The SVG marker defines **where** the button appears.
- The `trigger:` field defines **what** happens when the button is clicked.
- The `style(...)` block refines **how** the button looks and responds.
- Buttons integrate seamlessly with existing CueDSL expressions, making them a flexible way to expose navigation, audio, or structural cues as tactile UI widgets.

propagate() Group-Level Parameter Propagation

`propagate(...)` is a pre-parser macro that expands a cue or animation template across all direct children of an SVG `<g>` element. Each child receives a unique instance of the template with independently evaluated parameters.

Purpose

To apply variations of an animation or cue to *multiple similar objects* without manually writing separate IDs. Typical uses:

- evenly distributing random timing or values across many elements
- giving each object slightly different speed, scale, rotation, or other parameters
- generating unique `uid:` values for each item in a group

`propagate(...)` runs **before cue/animation parsing** and rewrites child element IDs into fully expanded DSL strings.

Syntax

```
propagate(
  TEMPLATE,
  ARG1,
  ARG2,
  ...
)
```

`TEMPLATE`

A **new-DSL animation or cue definition**, e.g.:

```
scale(values:[${1}, ${2}], mode:alternate, dur:${3}, uid:circs)
```

`${n}` placeholders

Inside the template, `${1}`, `${2}`, `${3}`, are replaced with the **evaluated values** of `ARG1`, `ARG2`, `ARG3`, etc.

Each **occurrence** is evaluated independently, allowing fresh randomisation.

UID Handling

- If `uid:NAME` appears in the template becomes `uid:NAME_0`, `uid:NAME_1`,
- If no `uid:` is present one is appended automatically:

```
uid:prop_GROUPINDEX_CHILDINDEX
```

Example: `uid:prop_3_2`

Example

Group of 6 circles with unique scale range and timing between **12**:

```
propagate(
  scale(values:[${1}, ${2}], mode:alternate, dur:${3}, uid:circs),
  rnd(1,2),        // ${1}: min scale
  rnd(1,2),        // ${2}: max scale
  rnd(0.4,1.2)     // ${3}: duration
)
```

Possible expanded IDs:

```
scale(values:[1.14,1.92], mode:alternate, dur:0.66, uid:circs_0)
scale(values:[1.83,1.21], mode:alternate, dur:1.07, uid:circs_1)
scale(values:[1.05,1.99], mode:alternate, dur:0.48, uid:circs_2)
...
```

Evaluation Rules

- Arguments (`ARG1`, `ARG2`,) may contain:
  - `rnd(min,max)`
  - `rnd([a,b,c])`
  - numeric literals
  - string literals

- Each `${n}` in the template triggers an independent evaluation of `ARGn`.

This enables effects like:

```
scale(values:[${1},${1}], dur:${2})
```

Where the two `${1}` occurrences receive **different random values**.

Non-Recursive

Only **direct children** of the `<g id="propagate(...)">` are expanded.

Nested groups are left as-is unless you manually apply propagate to them.

Execution Order

`propagate(svgElement)` must run **before any animation or cue parsing**, typically inside `initializeSVG()`:

```
// Expand propagate(...) groups before parsing
propagate(svgElement);
```

If used in page overlays, it must run before `animationAssign(...)` and cue scanning.

Supported Contexts

- Main score SVG
- Subpages / page mode SVG
- Cue-triggered dynamic page loads

When to Use

Use `propagate(...)` when:

- you have many similar shapes needing slight variation
- you want per-object timing/scale/rotation differences
- you want a template-based param generator for groups
- you want unique uids for each instance automatically

When Not to Use

Avoid propagate when:

- objects require coordinated timing (use shared seqdur instead)
- nested group structure must remain untouched
- ordering or indexing inside the SVG is unpredictable

Notes

- `propagate(...)` is **not** parsed by the DSL directly.
- It is a *preprocessor* that rewrites IDs into pure DSL expressions.
- After expansion, each child is parsed as if you wrote it manually.

Version Compatibility

This documentation refers to the **new Oscilla DSL (2025)** with:

- function-style cues
- named parameters
- canonical `uid:` handling
- no legacy `_uid(...)` microsyntax

Reusable Blocks with `use(name)`

`use(name)` allows you to define a visual or interactive structure once and reuse it anywhere in the score or on any page. This avoids copy/paste in Inkscape and keeps layouts consistent.

Defining a Reusable Block

Any SVG group whose ID begins with:

`reuse-`

is treated as a reusable block.

Example:

```
<g id="reuse-mainMenu">
   <!-- Buttons, text, shapes, animations -->
</g>
```

This block is registered automatically when its SVG is loaded.

Using the Block Elsewhere

To insert the reusable block somewhere else, place a placeholder group:

```
<g id="use(mainMenu)"></g>
```

When the score initializes, the placeholder is replaced with a cloned instance of the reusable block, preserving:

- cueButtons
- cueText elements
- animations and object-followers
- internal transforms and visual structure

The placeholder itself is removed.

Placement Modes

There are two placement modes depending on how `use()` is written:

Syntax	Placement Behaviour
`use(name)`	Inserts the block at **its original authored coordinates** (as designed in its source SVG).
`use(name)@self`	Inserts the block **at the location of the placeholder**, inheriting the placeholders transform.

Example (`@self` Placement)

```
<g transform="translate(300,200)">
  <g id="use(mainMenu)@self"></g>
</g>
```

Result: `mainMenu` appears exactly at `(300,200)`.

Behaviour and Guarantees

- The injected block receives **unique IDs** to avoid collisions.
- The placeholder `<g id="use(...)" />` is **removed** automatically.
- Reusable blocks can be defined in **any SVG inside `/pages/`**.
- All internal behaviour is preserved, including:
  - cue triggers
  - animations (rotate, scale, obj2path)
  - cue buttons and overlays

Summary

Define once:

```
<g id="reuse-name">…</g>
```

Use anywhere:

```
<g id="use(name)"></g>
```

Or place at placeholder position:

```
<g id="use(name)@self"></g>
```

This system keeps Oscilla scores modular, clean, and visually authored directly in Inkscapeno handwritten layout code required.

cue:text OscillaScore Text Cue Reference

This document describes all features of the OscillaScore text engine, including **slot-based layout**, **two-layer crossfades**, **duration/gap logic**, **looping**, and **ordering**.

---

1. Basic Syntax

```
text(
  src:<file or inline>,
  order:<seq|rnd>,
  dur:<seconds or range>,
  gap:<seconds or range>,
  cross:<0|1>,
  crossfade:<ms or "%">,
  loop:<0|N>,
  target:<center|self|elementId>,
  offsetX:<px>,
  offsetY:<px>,
  yslots:<N>,
  yoffset:<px>,
  yslotmode:<sequence|singlestep|random|shuffle>,
  autostart:<0|1>,
  style:"CSS..."
)
```

All parameters are optional unless stated.

---

2. Content Source

`src:tzara.txt`

Loads a `.txt` file from the projects text directory.

`src:"Hello world"`

Inline text string.

Lines may be separated with: - newline () - semicolon (;)

---

3. Unit Construction

`mode:line` ***(default)***

Each line is shown as a unit.

`mode:word`

Each word becomes a unit.

`mode:char`

Each character becomes a unit.

---

4. Duration & Gap

`dur:3`

Each unit shows for **3 seconds**.

Ranges

```
dur:2-4
gap:0.5-1.2
```

- Duration is chosen per unit.
- Gaps produce blank time between units **only when `cross:0`**.

---

5. Ordering

`order:seq`

Units shown in natural order.

`order:rnd`

Units shuffled every cycle.

---

6. Looping

The loop behaviour:

Setting	Meaning
`loop:0`	Infinite loop
`loop:N`	Play N cycles
No loop param, `order:seq`	Play once
No loop param, `order:rnd`	Loop infinitely

Example infinite loop:

```
text(src:foo.txt, loop:0)
```

---

7. Crossfade System

OscillaScore now uses a **two-layer crossfade design**:

- The **old line** fades out in its slot
- The **new line** fades in in its own slot
- Both remain visible simultaneously during overlap
- No teleporting, no slot-shifting during fade

`cross:1`

Enables overlapping crossfade.

`cross:0`

Fade-out blank gap fade-in.

`crossfade:"60%"`

Fade time = 60% of unit duration.

`crossfade:800`

Fade time = 800ms.

---

8. Slot-Based Vertical Layout

Slots override vertical placement completely.

This system enables structured multi-line presentation.

`yslots:N`

Enable slot layout, N vertical “lanes”.

Example:

```
yslots:3
```

Creates:

```
top
center
bottom
```

`yoffset:<px>`

Distance between slots. Default: `100px`.

Slot Modes

`yslotmode:sequence`

```
1 → 2 → 3 → 1 → 2 → 3...
```

`yslotmode:singlestep`

Bounces:

```
center → next → center → previous → center...
```

`yslotmode:random`

Each unit goes to a random slot.

`yslotmode:shuffle`

Uses each slot once per cycle:

```
(3,1,2) → (2,3,1) → ...
```

Behavior

- Each line lives in exactly **one** slot.

- Crossfade overlaps use **two different layers**, one per slot.
- With high crossfade %, multiple lines may be visible across slots.

This allows the reader to finish reading an old line while a new one appears.

---

9. Positioning

Slots override Y-position entirely.

Horizontal center with `offsetX`

```
left = 50% + offsetX
```

If `yslots` is not enabled:

- `target:self` anchors to cue element
- `target:<id>` anchors to an SVG element
- `target:center` (default) centers onscreen

---

10. Examples

Basic

```
text(src:foo.txt, dur:3, autostart:1)
```

Infinite random cycling

```
text(src:foo.txt, order:rnd, dur:2, loop:0)
```

Crossfade with 60% overlap

```
text(src:foo.txt, dur:3, cross:1, crossfade:"60%")
```

Slot-based layout (3 lanes)

```
text(
  src:foo.txt,
  dur:3,
  cross:1,
  crossfade:"70%",
  yslots:3,
  yoffset:130,
  yslotmode:singlestep,
  autostart:1
)
```

Sequential slot cycling

```
text(src:foo.txt, yslots:3, yslotmode:sequence)
```

Random slots, no crossfade

```
text(
  src:foo.txt,
  yslots:3,
  yslotmode:random,
  cross:0,
  gap:1,
  dur:2
)
```

---

11. Interaction

Clicking the text overlay cancels the sequence.

`persist:1`

Prevents the overlay from being removed during `stopAllCueTexts()`.

---

12. Stopping All Text

`stopAllCueTexts()`

Removes all **non-persistent** text overlays.

---

13. Summary Table

Param	Description
`src`	File or inline text
`order`	seq or rnd
`dur`	seconds or range
`gap`	seconds or range (non-cross only)
`cross`	1=overlap, 0=fade-out/in
`crossfade`	ms or percent
`loop`	0 infinite, N cycles
`mode`	line / word / char
`yslots`	number of vertical lanes
`yoffset`	px between lanes
`yslotmode`	sequence, singlestep, random, shuffle
`offsetX/Y`	px offsets
`autostart`	begin immediately
`persist`	survive global stop

End of Document

Audio Cues `audio`, `audioPool`, `audioImpulse`

Oscillas audio system provides three related cue types:

- **audio()** play a specific file
- **audioPool()** select one file from a discovered folder
- **audioImpulse()** stochastic repeated triggering from a pool

All cues use generic `key:value` parameter syntax.

---

1. `audio(...)` Play a Single File

Lowest-level audio playback building block.

Example

```
audio(src:kick, amp:0.9, loop:1, fade:0.3, pan:-0.5, pitch:1.2)
```

Parameters

Key	Meaning
`src` (required)	filename/stem, `.wav` auto-added
`amp`	gain 01 (default 1)
`pan`	stereo position -1 (left) to 1 (right), default 0

Key	Meaning
`pitch`	playback rate multiplier (default 1). Values < 1 slow down, > 1 speed up
`loop`	1=once, N>1 repeats, 0=infinite
`fade`	applies to both fadeIn and fadeOut
`fadeIn`	fade-in seconds or percentage (e.g.`"20%"`)
`fadeOut`	fade-out seconds or percentage (e.g.`"50%"`)
`toggle`	second trigger stops instead of restarting
`uid`	playback identity (default = src)

Files load from the project `/audio` folder, falling back to shared `/audio`.

Playback state is tracked so UI and buttons can reflect on/off.

---

2. `audioPool(...)` One-Shot Selection From a Folder

A pool is built dynamically by scanning a folder you never list filenames manually.

Example

```
audioPool(
  path:sfx/birds,
  format:wav,
  mode:shuffle,
  amp:rand(0.2, 0.8),
  pan:rand(-1, 1),
  pitch:rand(0.8, 1.2),
  fadein:0.05,
  fadeout:"30%",
  poly:4,
  uid:birdsA,
  osc:1,
  oscaddr:"/audio/pool/birds"
)
```

Behaviour

- Server enumerates files in `path:`
- Every trigger selects **one** file
- Optional randomisation per trigger
- Overlays show filename and evaluated params (amp, pan, pitch)
- Optional OSC mirror sends each hit to external software

Parameters

Key	Meaning
`path` (required)	folder inside project audio
`glob`	optional filter hint
`format`	extension (default `wav`)
`mode`	`shuffle` (no repeat until exhausted) or `rand` (pure random)
`amp`	number or `rand(a, b)`
`pan`	stereo -1 to 1, or `rand(-1, 1)`
`pitch`	playback rate, or `rand(a, b)`
`fadein`	seconds or percentage string
`fadeout`	seconds or percentage string
`fade`	shorthand for both fadein and fadeout

Key	Meaning
`loop`	loop the selected file
`poly`	overlapping voices (default 1, 0=unlimited)
`uid`	identity of this pool
`osc`	enable OSC mirroring (1=on)
`oscaddr`	custom OSC address

Polyphony applies **per pool**.

---

3. `audioImpulse(...)` Stochastic Repeating Process

Uses the same pool logic, but runs autonomously and keeps firing hits at a configurable rate.

Example

```
audioImpulse(
  path:sfx/rain,
  rate:40,
  jitter:0.4,
  amp:rand(0.1, 0.5),
  pan:rand(-1, 1),
  pitch:rand(0.5, 2),
  fadein:0.1,
  fadeout:rand("10%", "60%"),
  poly:6,
  lifetime:region,
  uid:rainfall,
  osc:1,
  oscaddr:"/audio/impulse/rain"
)
```

Timing Parameters

Key	Meaning
`rate`	events per minute
`jitter`	timing randomisation 01 (0=steady, 1=fully random)
`poly`	overlapping voices (default 6, 0=unlimited)

Sound Parameters

Key	Meaning
`amp`	gain 01, or `rand(a, b)`
`pan`	stereo -1 to 1, or `rand(-1, 1)`
`pitch`	playback rate, or `rand(a, b)`
`fadein`	seconds or percentage
`fadeout`	seconds or percentage
`fade`	shorthand for both

Lifetime Modes

Value	Behaviour
`process`	runs until explicitly stopped

Value	Behaviour
`region`	runs only while playhead is inside the cue element

In region mode, overlays update live and disappear when leaving the region.

OSC Parameters

Key	Meaning
`osc`	enable OSC mirroring (1=on)
`oscaddr`	custom OSC address (default `/oscilla/audio/impulse`)

Fade Values

Fades can be specified as:

Format	Example	Meaning
Seconds	`fadeout:0.5`	0.5 second fade
Percentage	`fadeout:"50%"`	50% of the files duration
Random seconds	`fadeout:rand(0.1, 0.5)`	random between 0.10.5 seconds
Random percentage	`fadeout:rand("10%", "60%")`	random between 10%60% of duration

Percentage-based fades automatically adjust to each files actual duration (accounting for pitch changes).

Random Expressions

Evaluated **per hit**:

```
amp:rand(0.2, 0.9)       // random amplitude
pan:rand(-1, 1)          // random stereo position
pitch:rand(0.5, 2)       // random playback speed
fadeout:rand(0.05, 0.3)  // random fade in seconds
fadeout:rand("10%","50%") // random fade as percentage
```

For integer random values, use `irand(a, b)`.

OSC Output

When `osc:1` is enabled, each audio hit sends an OSC message.

Message Format

```
/oscilla/audio/impulse sfffff filename amp pan pitch fadeIn fadeOut
/oscilla/audio/pool    sfffff filename amp pan pitch fadeIn fadeOut
```

Custom Address

Use `oscaddr` to specify a custom OSC path:

```
audioImpulse(path:sfx, osc:1, oscaddr:"/myapp/texture/rain")
```

This sends to `/oscilla/myapp/texture/rain`.

Example OSC Output

```
/oscilla/audio/impulse/x1 sfffff "sfx/birdgone.wav" 0.45 -0.32 1.2 0.2 0.8
```

Arguments: `filename (string)`, `amp`, `pan`, `pitch`, `fadeIn`, `fadeOut` (all floats)

Stopping

- Internal stop on fade completion or toggle
- Region exit automatically cleans up the process
- Programmatic helpers:
  - `stopAudioImpulse(uid)` stop a specific impulse process
  - `stopAllAudio()` stop all audio playback

Visual Overlays

Audio cues display overlays showing:

- **audioPool**: filename and params, auto-destroys after 1.5s
- **audioImpulse**: persistent overlay showing current file, amp, pan, pitch updates each hit, destroys on region exit

---

Quick Examples

```
// Simple drone with slow fade
audio(src:drone, loop:0, fade:2)

// Percussive one-shots with stereo spread
audioPool(path:sfx/wood, mode:shuffle, pan:rand(-0.8, 0.8), poly:5)

// Pitched-down atmosphere
audio(src:atmosphere, pitch:0.5, loop:0, fadeIn:3, fadeOut:5)

// Rainfall texture inside a passage with OSC output
audioImpulse(
  path:sfx/rain,
  rate:30,
  jitter:0.5,
  amp:rand(0.2, 0.6),
  pan:rand(-1, 1),
  pitch:rand(0.8, 1.3),
  fadeout:"40%",
  lifetime:region,
  osc:1,
  oscaddr:"/texture/rain"
)

// Bird sounds with percentage-based random fades
audioImpulse(
  path:sfx/birds,
  rate:20,
  amp:rand(0.4, 1),
  pan:rand(-1, 1),
  pitch:rand(0.1, 2),
  fadein:0.2,
  fadeout:rand("1%", "60%"),
  lifetime:region,
  uid:birdgroup,
  osc:1,
  oscaddr:"/audio/birds"
```

)

video()

Purpose: Spawn and control HTML5 video overlays during a score.

Required - file: Filename with optional extension (.mp4 default if missing). Supports mp4|webm|ogg.

Placement & Positioning - location: fixed (default) | scroll - fixed pinned to viewport. - scroll follows target/score scroll. - target: <elementId> centers the video on the target (if not fullscreen). - offsetX: / offsetY: pixel offsets applied after centering.

Size - size: - fs or fullscreen covers viewport at (0,0) with 100vw×100vh and **passes clicks through** by default. - <W> (e.g.640) width in px, auto height. - <W>x<H> (e.g.640x360).

Audio (Default Muted) - audio: 0|1|true|false **default is muted**. Set audio:1 (or true) to unmute.

Spawning / Reuse - By default, cues **reuse** an existing video matching the same file + target. - uid: <string> or new:1 **force a new concurrent instance** (even if one exists). - Every instance receives data-uid for tracking; data-key = file_target.

Timing & Playback - in: seconds (seek start) - out: seconds (optional fade-out scheduling; pairs well with fadeOut) - hold: seconds (auto-remove after this duration, regardless of loop) - loop: 0 = infinite; N = loop count; omit = play once - speed: playback rate (e.g.0.5, 1.25) - opacity: 0..1 (default 1) - fadeIn: seconds; fadeOut: seconds

Interaction - **Fullscreen (size:fs)**: uses pointer-events:none so the score behind remains draggable/clickable. - Add clickable:1 to allow clicks **on** the fullscreen video (e.g., to close on click). - Non-fullscreen videos remain clickable; single-click removes them by default.

Removal - Ends when: - playback completes (no loop) **or** - loop count is reached **or** - hold expires **or** - user clicks (non-fs) / double-click overlay (if you implement) / explicit cue cleanup.

Examples

1. **Fullscreen, pass-through clicks, but allow click-to-close**
   `cue:video(file:intro.mp4,size:fs,clickable:1,audio:1,fadeIn:0.5)`
2. **Windowed, anchored to a target, follows scroll, infinite loop**
   `cue:video(file:clip.webm,target:markerA,location:scroll,size:640x360,loop:0,opacity:0.9)`
3. **Spawn a second independent instance via uid**
   `cue:video(file:cam.mp4,uid:stageLeft,in:5,fadeIn:1,fadeOut:1,hold:20)`
4. **Force new instance without specifying uid**
   `cue:video(file:teaser.mp4,new:1,in:2,out:10,fadeIn:0.5,fadeOut:0.5,speed:1.25)`

Note: *size:fs always positions at (0,0) and ignores target geometry; all other sizes center on target (or the cue position) with optional offsetX/offsetY. Default audio is muted; use audio:1 to unmute. By default, same file+target cues reuse the existing element unless uid or new:1 is supplied.*

synth() Web Audio Synth Cue

Oscilla "Native WebAudio Utility Synth"

The synth() cue creates a lightweight Web Audio sound source inside Oscilla. It is intended for **reference tones, drones, textures, chords, and simple patterned sound processes**, integrated directly into the score timeline.

The design prioritises:

- deterministic, cue-scoped behaviour
- readable parameter syntax
- low default amplitudes and click-free envelopes
- optional pattern sequencing
- optional OSC mirroring

The synth is not intended as a full synthesiser environment. For full electroacoustic work, Oscilla is designed to operate in conjunction with external audio systems via OSC (e.g.SuperCollider, Pure Data,

Max). The built-in synth provides a bounded, score-aligned sound source intended for rehearsal contexts away from a complete setup, as well as for simple tone cues, drones, and basic animation- or pattern-driven sound sequences embedded directly in the score.

Basic Usage

```
synth(uid:refA, wave:sine, freq:440, amp:0.1)
```

Pitch may be specified as Hz or note name:

```
synth(uid:tuning, wave:sine, freq:A4, amp:0.08)
```

Wave Types

The following `wave` values are supported:

```
sine
square
saw
triangle
noise
```

Oscillator waveforms map directly to standard Web Audio oscillator types.

`noise` produces a broadband noise source generated from a looping audio buffer. No spectral colouring is applied.

Lifetime and Duration

Region-based lifetime (default)

When `synth()` is attached to a cue element, the synth starts when the playhead enters the cues bounding box and stops when the playhead exits it.

```
synth(uid:regionTone, wave:sine, freq:220, amp:0.06)
```

Explicit duration

```
synth(uid:fixedDur, wave:sine, freq:220, dur:5, amp:0.07)
```

The synth stops automatically after the specified number of seconds.

Persistent process lifetime

```
synth(uid:persist, wave:saw, freq:110, lifetime:process, amp:0.05)
```

The synth continues until explicitly stopped.

Amplitude Envelope (ADSR)

```
synth(
  uid:env1,
  wave:saw,
  freq:220,
  amp:0.12,
  env:{a:0.5, d:0.2, s:0.7, r:1.2}
)
```

Envelope parameters:

- `a` attack (seconds)
- `d` decay (seconds)
- `s` sustain level (01)
- `r` release (seconds)

Chords

If `freq` is an array, multiple oscillators are created and summed as a chord.

```
synth(
  uid:pad3,
  wave:sine,
  freq:[440, 477, 644, 777],
  env:{a:1.5},
  amp:0.12
)
```

All voices share the same envelope and effects chain.

Patterned Parameters

The following parameters may be static values, arrays, or pattern functions:

- freq
- amp
- dur
- filter.freq
- filter.q
- pan

Frequency sequence

```
synth(
  uid:seq1,
  wave:triangle,
  freq:Pseq(220, 330, 440),
  dur:Pseq(1, 1, 2),
  amp:0.08
)
```

Patterned chord progression

```
synth(
  uid:chordSeq,
  wave:saw,
  freq:Pseq(
    [220, 330, 440],
    [247, 370, 494],
    [196, 294, 392]
  ),
  dur:1.5,
  amp:0.1
)
```

Random pitch

```
synth(
  uid:randFreq,
  wave:square,
  freq:Prand(200, 400, 600),
  dur:0.8,
  amp:0.09
)
```

Filters

```
synth(
  uid:filterSeq,
  wave:saw,
  freq:330,
  filter:{type:lp, freq:Pseq(400, 800, 1600, 800), q:0.7},
  dur:0.5,
  amp:0.08
)
```

Supported filter types:

```
lp
hp
bp
notch
```

Delay

```
synth(
  uid:delayLead,
  wave:square,
  freq:550,
  delay:{time:0.25, fb:0.35, mix:0.2},
```

```
  amp:0.12
)
```

Reverb

```
synth(
  uid:revDrone,
  wave:sine,
  freq:110,
  reverb:{mix:0.3, time:2, damp:3000},
  amp:0.07
)
```

Glide (Frequency Only)

```
synth(
  uid:glide1,
  wave:sine,
  freq:Pseq(220, 330, 440),
  glide:0.1,
  dur:1,
  amp:0.1
)
```

`glide` specifies the time (seconds) used to interpolate frequency changes between steps. It applies only to pitch.

Interpolation Mode

```
synth(
  uid:stepSeq,
  wave:triangle,
  freq:Pshuf(300, 450, 600),
  interp:step,
  dur:1,
  amp:0.08
)
```

- `interp:smooth` ramps between values
- `interp:step` immediate value changes

Pan

```
synth(
  uid:panTest,
  wave:sine,
  freq:440,
  pan:-0.6,
  amp:0.09
)
```

Pan may be static or patterned but does not support continuous modulation.

OSC Mirroring

```
synth(
  uid:oscLead,
  osc:1,
  oscAddr:/synth/lead,
  wave:saw,
  freq:440,
  amp:0.1
)
```

Stopping a Synth

```
synthStop(uid:seq1, rel:0.5)
```

The optional `rel` parameter specifies release time in seconds.

Summary

The `synth()` cue provides a bounded, score-aligned sound source suitable for reference tones, drones, chords, and simple patterned textures. All behaviour is deterministic and cue-scoped, and integrates directly with Oscillas timing and animation model.

cue:osc Discrete OSC Event Cue

`osc(...)` sends a **single OSC message** when triggered.

It is **event-based**, not continuous. It turns drawn objects into **discrete control events**, typically consumed by environments like SuperCollider, Pd, or Max.

It works especially well with `propagate()`, because a simple drawing can become many individual OSC triggers.

BASIC FORM

```
osc(addr:<name>, <param>:<source>, ...)
```

Required:

Key	Meaning
`addr`	OSC address name (without leading /)

Example:

```
osc(addr:voice, pitch:y, env:size)
```

EVENT BEHAVIOUR

- Sends **one OSC message per trigger**
- No looping, no animation, no scheduler
- Values are sampled **at the moment of triggering**
- Cue completes immediately

`osc()` is a *logical cue*, not an animation cue.

VISUAL PARAMETER SOURCES (NORMALISED)

All visual sources are normalised to `0–1`.

Source	Meaning
`x`	Object centre X position
`y`	Object centre Y position
`size`	Max(width, height) mapped to viewport
`scale`	Current visual scale value
`rotation`	0360 mapped to 01
`opacity`	Computed opacity
`fill`	Numeric colour hash

Example:

```
osc(addr:voice, pitch:y, amp:size)
```

Here the Y-axis produces a **continuous control pitch**, interpreted downstream.

SEMANTIC PITCH VALUES

Beyond visual values, `osc()` supports **typed pitch declarations**.

Hz (absolute)

```
osc(addr:voice, pitch:hz(440))
```

MIDI note number

```
osc(addr:voice, pitch:midi(60))
```

Scale degree + octave

```
osc(addr:voice, pitch:deg(5,3))
```

Represents:

- scale degree = 5
- absolute octave = 3

The receiver chooses how to interpret the scale.

OPTIONAL ROOT OVERRIDE

You may specify a **root** in the cue:

```
osc(addr:voice, pitch:deg(2,4), root:48)
```

If omitted, the receiver uses its global default (`~oscRoot` in SC).

PITCH PRIORITY

When multiple forms are present, resolution is:

1. `deg(d,o)` (scale-based symbolic pitch)
2. `hz()` / `midi()` (absolute pitch)
3. visual mapping (e.g., `pitch:y`)

Only one pitch representation is used per event.

TRIGGERS

Key	Meaning
`trig:auto`	Send immediately
`trig:playhead`	Fire when playhead intersects
`trig:click`	Fire on user click
`prestate:ghostClickable`	Arm interaction ahead of time

Example:

```
osc(addr:voice, pitch:deg(0,4), trig:playhead)
```

UID (OPTIONAL)

```
osc(addr:voice, pitch:y, uid:v1)
```

UID can be used downstream to associate events with voices/players/etc.

OSC OUTPUT FORMAT (AUTHORITATIVE)

`osc()` currently sends **positional arguments**, not keyed pairs.

GENERAL PACKET FORMAT

```
/oscilla/<addr>
  pitchType, pitchA, pitchB, size, env, density, [optional root]
```

Where:

Field	Meaning
`pitchType`	0=none, 1=deg/oct, 2=hz, 3=midi, 4=y-control
`pitchA`	meaning depends on pitchType
`pitchB`	octave (deg mode only)
`size`	01
`env`	01
`density`	01 (area-derived)
`root`	optional per-event root override

Examples

Control pitch from Y

```
/oscilla/voice
1 5 3 0.41 0.41 0.12
```

Absolute pitch

```
/oscilla/voice
2 440 0 0.18 0.22 0.10
```

Degree + octave + root override

```
/oscilla/voice
1 5 3 0.33 0.20 0.15 48
```

Receivers are expected to decode according to this positional layout.

USE WITH propagate()

```
propagate(
  osc(
    addr:voice,
    pitch:y,
    env:size,
    trig:playhead
  )
)
```

Degree constellation examples

```
propagate(
  osc(
    addr:pontalist,
    pitch:deg(irand(0,11), irand(0,2)),
    env:size
  )
)

propagate(
  osc(
    addr:pontalist,
    pitch:deg(${1}, ${2}),
    env:size,
    root:48
```

```
  ),
  rnd([0,2,4,5,7,9,11]),
  rnd([0,1,2,3,4,5])
)
```

DESIGN NOTES

- stateless
- no DOM ID use
- no continuous OSC streaming
- no scale-logic baked into DSL
- musical interpretation lives downstream
- arguments are positional, compact, and receiver-focused

SUMMARY

*`osc()` converts visual gestures into **one-shot OSC control events**, supporting symbolic, absolute, and control-driven pitch while leaving musical semantics to the instrument.*

oscCtrl Continuous Control Lanes (OSC)

`oscCtrl` lets you draw a **path in the score** and use it as a live control lane for audio / electronics via OSC. As the playhead passes over the path, values are mapped and streamed to the audio engine.

The path acts like a **breakpoint automation curve** tightly aligned to instrumental notation.

Basic Syntax

```
oscCtrl(
  addr:"/fx/ring/freq",
  min:60,
  max:800,
  uid:ringModFreq
)
```

Parameters

Param	Required	Type	Description
`addr`		string	OSC address to control (joined under `/oscilla/control`)
`min`		number	Minimum mapped value (default `0`)
`max`		number	Maximum mapped value (default `1`)
`uid`		string	Unique label (clientside only, debugging)
`mode`		enum	`event` (default) or `continuous`

The cue **must be placed directly on a `<path>` element**.

What gets sent

Given:

`addr:"/fx/ring/freq"`

Oscilla sends:

```
/oscilla/control/fx/ring/freq   <value>   <t>
```

Where:

- **value** mapped Y position between `min` `max`
- **t** normalized X position along the path (01)

Example:

```
/oscilla/control/fx/ring/freq   726.06   0.27
```

Path Semantics

The path defines:

- horizontal = **time alignment** with notation
- vertical = **control value**
- Top of path higher values
- Bottom of path lower values

Zoom, scale, and SVG transforms are handled automatically.

Modes

`mode:event` (default)

Send only when the value **changes meaningfully**.

```
oscCtrl(
  addr:"/fx/pan",
  min:-1,
  max:1
)
```

Reduces controller spam and CPU noise.

`mode:continuous`

Send regularly while the playhead is inside the lane:

```
oscCtrl(
  addr:"/fx/pan",
  min:-1,
  max:1,
  mode:continuous
)
```

Useful for:

- scrubbing
- FM / modulation
- granular playback positions

Messages are still **throttled to ~30/s**.

Recommended Uses

ring modulation frequency
panning automation
filter cutoff / resonance
spatialization
cueing electronics transitions

Not Sent

`oscCtrl` does **not** apply smoothing.
This should happen in the **audio client**, where it belongs.

Future Extensions

- `oscCtrlNode()` breakpoint nodes & interpolation
- custom easing curves (exp / log)
- curve visualization overlays

Troubleshooting

Nothing happens?
Ensure the cue is attached to the **path element itself**, not a group.

OSC not received?
Check the server receives:

```
type: "osc_control"
```

and that `/oscilla/control/*` is routed correctly.

Summary

`oscCtrl` brings automationstyle control inside the score while remaining modular and OSCnative.

Compose gestures visually let the audio engine interpret.

Chapter 4

Animation

Animation cues bring graphic score elements to life during playback. Objects can scale, rotate, follow paths, change colour, and fade in or out all driven by the same embedded cue syntax used for transport and media control.

These transformations are not decorative. In an animated graphic score, movement, colour shift, and visual emphasis carry musical meaning: a rotating arrow might indicate which material a performer should play next; a scaling gesture might suggest dynamic intensity; a colour transition might signal a shift in texture or instrumentation. The animation system allows composers to encode these interpretive signals directly into the visual objects that performers read.

Animations can be layered and combined on a single element, and their parameters can be modulated in real time through the control system described in the next section.

cue:scale Scaling Animation

`scale()` adjusts the size of SVG objects using sequences, patterns, continuous pulsing, or independent X/Y scaling.

BASIC FORMS

Uniform scaling

```
scale(values:[1,1.5,1], dur:2)
```

Non-uniform (XY)

```
scaleXY([1,1.3],[1,0.6], dur:1)
```

Continuous pulse

```
scale(min:1, max:1.3, dur:2)
```

TRIGGERING

- automatic page-mode
- scroll edge
- cue activation

TRIGGER DELAY

`tdelay:<seconds>`

Delays scaling start.

```
scale(values:[1,1.4,1], tdelay:3)
```

PRESTART VISIBILITY

`prestate:<show|hide|ghost>`

```
scale([1,1.5,1], tdelay:4, prestate:ghost)
```

Initial scale is applied immediately.

SEQUENCES

Key	Meaning
`values`	uniform scale list
`x`, `y`	independent XY sequences
`dur`	duration per step
`mode`	loop / once / alternate
`interp`	smooth / step
`hold`	pause (smooth only)

Pattern forms (`Pseq`, `Prand`, etc.) supported for all parameters.

CONTINUOUS SCALING

```
scale(min:1, max:1.4, dur:1.5, loop:0)
```

UID Live Updates

```
scale(values:[1,1.2,1], uid:s1)
scale(uid:s1, dur:0.5)
```

FULL PARAMETER LIST

Key	Description
`values`	scalar sequence
`x`, `y`	axis-specific sequences
`dur`	step duration
`hold`	pause
`ease`	tween curve
`mode`	loop logic
`interp`	smooth/step
`uid`	animation identity
`trig`	trigger
`tdelay`	delay after trigger
`prestate`	pre-start visual state

EXAMPLES

```
scale([1,2,1], dur:2, tdelay:3)
scaleXY([1,1.4],[1,0.6], dur:1)
scale(Pseq([1,1.5,1],inf), dur:Prand([0.5,1],inf))
```

rotate() Rotation Cue

`rotate(...)` rotates an SVG element using either:

- **continuous rotation**, or

- a **sequence / pattern of angle values**

Rotation can be:

- smooth interpolated
- stepped
- looping, alternating, or oneshot
- optionally sending OSC messages (with live overlays).

BASIC FORMS

Continuous

```
rotate(dir:1, dur:1)
```

Sequence

```
rotate(values:[0,120,240], dur:2)
```

Pattern sequences

```
rotate(values:Pseq([0,45,10],inf), dur:Pseq([1,0.2,2],inf))
```

TRIGGERING

- `auto` (page load / mode)
- `edge` (playhead collision)
- cue activation

TRIGGER DELAY `tdelay:<seconds>`

Delays motion after trigger:

```
rotate(values:[0,120,240], tdelay:2)
```

PRESTART VISIBILITY `prestate:<show|hide|ghost>`

Controls appearance before rotation begins.

```
rotate(values:[0,90,180], tdelay:4, prestate:ghost)
```

The element adopts the **initial rotation angle immediately**, then waits.

SEQUENCE PARAMETERS

Key	Meaning
`values`	list of angles or pattern
`dur`	seconds per step (or pattern)
`mode`	`loop`, `once`, `alternate`
`interp`	`smooth` or `step`
`hold`	pause after tween (smooth only)

CONTINUOUS ROTATION

```
rotate(dir:1, dur:2)
rotate(dir:-1, dur:3, ease:"easeInOutSine")
```

- `dir:1` clockwise
- `dir:-1` counterclockwise

- `loop:0` (default) = infinite

UID Live Updates

UID lets multiple cues address the same animation:

```
rotate(values:[0,90,180], uid:r1)
rotate(uid:r1, dur:0.5)
```

OSC OUTPUT (optional)

Rotation can send OSC every frame or perstep.

Enable via:

```
osc:1
```

Specify an OSC address:

```
rotate(values:[0,90,180], osc:1, oscaddr:"/spatialisation/source1")
```

Payload format

Every message includes:

Field	Meaning
`deg`	wrapped degrees **0360**
`rad`	radians (`deg * π/180`)
`norm`	normalized 01 (`deg/360`)
`uid`	animation UID
`timestamp`	ms since epoch

Example conceptual payload:

```
{
  "type": "osc_rotate",
  "uid": "r1",
  "deg": 135.0,
  "rad": 2.356,
  "norm": 0.375,
  "addr": "/spatialisation/source1"
}
```

*NOTE: Internally angles may accumulate, but outgoing OSC **always wraps to 0360**.*

Onscreen overlays

When OSC is enabled, a small overlay shows:

```
/spatialisation/source1 — deg:135.0
```

Overlays can be globally toggled.

FULL PARAMETER LIST

Key	Description
`values`	angle sequence or pattern
`dur`	duration or pattern
`mode`	loop logic
`interp`	step or smooth
`hold`	pause after tween
`dir`	direction (continuous mode)
`ease`	easing curve

Key	Description
`uid`	animation identity
`trig`	trigger mode
`tdelay`	trigger delay
`prestate`	visual prestart state
`osc`	enable OSC (0/1/2)
`oscaddr`	explicit OSC address

EXAMPLES

```
rotate(values:[0,120,240], dur:4, tdelay:2)

rotate(values:Pshuf([0,180],inf),
       dur:1,
       mode:alternate,
       osc:1,
       oscaddr:"/fx/rot1")

rotate(dir:-1, dur:3, prestate:hide, tdelay:1)
```

Summary

- declarative rotation cue
- works in page + scroll modes
- full control of timing + interpolation
- OSCready with wrapped degrees, radians, and normalized values
- optional debug overlays

cue:o2p Object-to-Path Animation

`o2p(...)` animates an SVG object along a named `<path>`.
It supports path traversal, directional modes, rotation modes, looping, OSC output, partial-path motion, trigger-based activation, delayed start, visual pre-start states, **interactive touch/drag control**, and **grouped fader presets with launcher bars**.

BASIC FORM

```
o2p(path:<id>, dur:<seconds>, mode:fwd)
```

Required:

Key	Meaning
`path`	The ID of the `<path>` element to follow

TIMING

`dur:<seconds>`

Duration of one full traversal of the path.

```
o2p(path:orbit, dur:6)
```

In `mode:alt`, the full ABA cycle lasts `dur` seconds.

tdelay:<seconds>

Delays the start of the animation after the cue triggers.

```
o2p(path:orbit, tdelay:3)
```

`tdelay` is real-time and independent of score position.

PRESTART VISIBILITY

prestate:<show|hide|ghost>

Controls how the object looks before the animation begins:

- `show` fully visible immediately
- `hide` invisible until start
- `ghost` semi-transparent until start

The object is positioned at its correct starting location before motion begins.

```
o2p(path:orbit, tdelay:4, prestate:hide)
```

DIRECTION MODES

Mode	Meaning
`forward` / `fwd`	0 1 along path
`reverse` / `rev`	1 0
`alternate` / `alt`	back-and-forth motion

```
o2p(path:curve, mode:alt, dur:8)
```

PATH SEGMENTS

```
o2p(path:ring, start:0.2, end:0.8)
```

The object travels only between normalized positions `start` and `end`.

ROTATION

`rotate:`	Description
`none`	no rotation
`aligned`	align to tangent direction
`locked`	fixed heading using `rotlock`
`spin`	continuous rotation (`rotspeed`, `rotdir`)

Additional keys:

Key	Meaning
`rotoffset`	add angular offset
`rotlock`	fixed heading in degrees
`rotspeed`	seconds per full rotation

Key	Meaning
`rotdir`	1 direction multiplier

LOOPING

```
o2p(path:orbit, loop:3)
o2p(path:orbit, loop:0)   // infinite
```

OSC OUTPUT

`o2p(path:orbit, osc:true)`

Emits:

`/o2p/<uid> <pathT> <normX> <normY> <angle>`

Custom OSC address:

`o2p(path:orbit, osc:true, oscAddr:myController)`

Emits:

`/myController <pathT> <normX> <normY> <angle>`

TRIGGERING

Trigger	Behavior
`trig:auto`	starts automatically in page mode
`trig:edge`	starts when the playhead intersects in scroll mode
`trig:touch`	no auto-animation; object is draggable along path

Cues may also be activated programmatically.
`tdelay` applies after the trigger is detected.

TOUCH MODE (Interactive Control)

The `trig:touch` mode transforms the o2p animation into an **interactive controller**. Instead of automatically animating, the object waits for user interaction.

`o2p(path:fader, trig:touch, osc:true)`

Behavior: - Object is positioned at the start point (or `start:` position) - No automatic animation occurs - User can **grab and drag** the hit-label overlay - Object follows the users finger/mouse along the path - If `osc:true`, every movement emits OSC values

Use Cases: - Browser-based OSC faders/sliders - XY pads with constrained motion - Rotary knobs (using circular paths) - Custom UI controllers that follow any SVG path shape

Example Linear Fader:

`o2p(path:verticalLine, trig:touch, osc:true, oscAddr:fader1)`

Example Circular Knob:

`o2p(path:knobArc, trig:touch, start:0.1, end:0.9, osc:true, oscAddr:knob1)`

Visual Indicator: Touch mode objects display an **orange** hit-label ring (instead of purple) to indicate they are draggable.

FADER GROUPS AND PRESETS

Touch-mode faders can be collected into named **groups**. A group enables saving and recalling all fader positions as named presets, tweening between presets with easing, and sequencing preset recalls over time.

Grouping Faders

Add `group:<name>` and `uid:<id>` to each touch-mode fader:

```
o2p(path:track1, trig:touch, group:mixer1, uid:vol, osc:true)
o2p(path:track2, trig:touch, group:mixer1, uid:pan, osc:true)
o2p(path:track3, trig:touch, group:mixer1, uid:send, osc:true)
```

All three faders register under `window._o2pTouchGroups["mixer1"]` and can be saved/recalled as a unit. The `group` value is freeform any string.

Launcher Bar

When a group registers, a **launcher bar** appears below the groups bounding box. It provides direct access to presets and sequences without opening the full preset panel.

The launcher includes:

- **Preset/Sequence slots** left-click to recall, long-press to store current positions
- **Bank navigation** arrow buttons to page through banks of slots
- **Mode toggle** (P/S) switch between preset and sequence slot modes
- **Tween toggle** (~) smooth interpolation or instant jump on recall
- **Sequence transport** play/stop buttons for sequence playback
- **Settings gear** opens the full o2p Preset Manager panel

An **orange toggle circle** appears to the right of the group. Click it to show or hide the launcher bar.

Preset Manager Panel

The preset panel (keyboard shortcut **Alt+Shift+O**, or gear button on launcher) provides full preset management:

- **Presets tab** save, recall, delete named presets with tween duration and easing
- **Sequences tab** define ordered lists of presets with per-step timing
- **Import/Export** save and load preset collections as JSON files

Console access: `window.o2pPresetUI.toggle()`

Preset Data

Each preset stores the normalized position of every fader in the group:

```
{
  "mixer1": {
    "vol": { "t": 0.75 },
    "pan": { "t": 0.3 },
    "send": { "t": 0.5 }
  }
}
```

If a fader has a rotation handle (`hmode:`), a `p` value is also stored:

```
{ "vol": { "t": 0.75, "p": 0.6 } }
```

Console API

```
window.o2pPresets.save("intro")           // save current positions
window.o2pPresets.recall("intro")         // instant recall
window.o2pPresets.recall("intro", { dur: 2, ease: "easeInOutSine" })  // tween
window.o2pPresets.list()                  // list all preset names
window.o2pPresets.delete("intro")         // delete a preset

// Sequences
window.o2pPresets.defineSequence("drift", [
  { preset: "intro", dur: 3 },
  { preset: "verse", dur: 2 },
  { preset: "chorus", dur: 4 }
```

```
], { loop: true })
window.o2pPresets.playSequence("drift")
window.o2pPresets.stopSequence()
```

ROTATION HANDLES

Touch-mode faders can have a secondary **rotation handle** that adds a second control dimension. This is useful for parameters like pan, filter cutoff, or any value that benefits from a rotary control alongside the linear fader.

`hmode:<continuous|limited>`

Determines the rotation behavior:

- `continuous` full 360-degree rotation with wraparound (suitable for azimuth, phase)
- `limited` constrained arc with hard stops like a physical potentiometer (suitable for volume, gain)

`rotrange:<degrees>`

Sets the arc range for `hmode:limited`. Default is 270 degrees.

```
o2p(path:track1, trig:touch, group:mixer1, uid:vol, hmode:limited, rotrange:270, osc:true)
```

`handle:<elementId>`

Reference a user-drawn SVG element as the rotation handle instead of the auto-generated one:

```
o2p(path:track1, trig:touch, group:mixer1, uid:vol, hmode:limited, handle:knob1, osc:true)
```

Where `knob1` is the ID of an existing SVG element in the score.

LIVE UPDATE (`uid:`)

```
o2p(path:ring, uid:a1)
o2p(uid:a1, mode:alt)
```

Later cues with the same `uid` modify running animations.

FULL PARAMETER LIST

Key	Description
`path`	required path reference
`dur`	motion duration
`mode`	direction mode
`loop`	repeat count (`0` = infinite)
`start`, `end`	normalized path range
`rotate`	rotation behavior
`rotoffset`, `rotlock`, `rotspeed`, `rotdir`	rotation parameters
`ease`	easing preset
`osc`	OSC emission
`oscAddr`	custom OSC address (without leading `/`)
`uid`	animation identity tag
`trig`	trigger mode (`auto`, `edge`, `touch`)
`tdelay`	time delay after trigger
`prestate`	show/hide/ghost before start
`group`	fader group name (touch mode only)
`hmode`	rotation handle mode: `continuous` or `limited` (touch mode only)

Key	Description
`rotrange`	arc range in degrees for `hmode:limited` (default 270)
`handle`	ID of user-drawn SVG element to use as rotation handle

EXAMPLES

```
o2p(path:orbitA, dur:8, tdelay:3, prestate:hide)
o2p(path:spiral, rotate:aligned, rotoffset:-90)
o2p(path:ring, start:0.2, end:0.9, mode:alt)
o2p(path:circle, rotate:spin, rotspeed:2, rotdir:-1)

// Touch mode controllers
o2p(path:faderTrack, trig:touch, osc:true, oscAddr:volume)
o2p(path:knobPath, trig:touch, start:0.15, end:0.85, osc:true)

// Grouped faders with preset system
o2p(path:track1, trig:touch, group:mixer1, uid:vol, osc:true)
o2p(path:track2, trig:touch, group:mixer1, uid:pan, osc:true)
o2p(path:track3, trig:touch, group:mixer1, uid:send, osc:true)

// Grouped fader with rotation handle
o2p(path:track1, trig:touch, group:mixer1, uid:vol, hmode:limited, rotrange:270, osc:true)

// Grouped fader with user-drawn rotation handle
o2p(path:track1, trig:touch, group:mixer1, uid:vol, hmode:continuous, handle:knob1, osc:true)
```

`cueTraverse` Animate Objects Between SVG Points

The `cueTraverse(...)` cue animates an SVG object along a path defined by a list of point elements (`<circle>`, `<rect>`, or `<ellipse>`). Its useful for moving symbols, playheads, or visual elements dynamically across the score.

Syntax

```
cue_traverse_p(p1,p2,p3)_o(objectId)_s(speed)_d(direction)_x(repeats)_e(easeType)_h(holdMs)_next(targetCue)
```

Each parameter is enclosed in its own parenthesis. The cue name and parameters can be written as a single underscore-delimited string.

Parameter Reference

Param	Meaning
`p(...)`	List of point IDs (e.g.`p(p1,p2,p3)`)
`o(...)`	Object ID to move
`s(...)`	Speed in pixels per second (e.g.`s(2.5)`)
`d(...)`	Direction: `0=forward`, `1=reverse`, `2=pingpong`, `3=random`
`x(...)`	Number of repeats (`0 = infinite`)
`e(...)`	Easing type (0 = linear, 19 = anime.js presets)
`h(...)`	Hold time at each point in milliseconds
`_next(...)`	Optional cue to trigger after traversal ends

Examples

Traverse between 3 points once:

```
cue_traverse_p(p1,p2,p3)_o(dot)_s(1.5)_x(1)
```

Infinite pingpong movement:

```
cue_traverse_p(p1,p2,p3)_o(dot)_d(2)_x(0)
```

Traverse with hold, ease, and jump to another cue:

```
cue_traverse_p(A,B,C)_o(cursor)_s(3)_e(2)_h(300)_next(cue_done)
```

Notes

- Objects must be `<circle>`, `<ellipse>`, or `<rect>` to be correctly positioned.
- `p(...)` must contain **at least two** valid point IDs.
- `_next(...)` cue will only be triggered after **final traversal loop** finishes (or never, if infinite).
- Traversal will pause playback if `isPlaying` is false at start.

Triggering Other Objects via Intersections

As the animated object (defined via `o(...)`) moves through the points listed in `p(...)`, the system checks whether any point ID matches the `data-id` of other SVG elements.

When a match is found:

- The corresponding element is **animated using its `data-id` definition**
- This allows `cueTraverse` to function like a **trigger engine**
- Its useful for triggering visual pulses, spins, or dynamic elements connected to spatial locations in the score

Example

```
c-t_p(p1,p2,p3)_o(pulsetrig01)
```

If you have an element like:

```
<circle id="pulse" data-id="pulsetrig01 obj_scale_seq[1,1.5,1]_bounce" />
```

When the animated object reaches a point named `pulsetrig01`, the element above will animate according to its `data-id`.

color() / colour() Colour Animation Cue

Animates the visual colour of SVG elements over time using HSL interpolation.

Both spellings are fully equivalent:

```
color(...)  □  colour(...)
```

Quick Examples

```
// Cycle through three colours
color(vals:[#f00, #0f0, #00f], dur:3)

// Continuous rainbow hue rotation
color(vals:hue, dur:10, loop:0)

// Alternating palette (ping-pong)
color(vals:[#f80, #08f], mode:alt, dur:1.2)

// Pattern-driven sequence
color(vals:Pseq([red, yellow, cyan], 4), dur:0.5)
```

Basic Syntax

```
color(
    vals:[...],      // colour values or hue keyword
    dur:seconds,     // duration per transition
    mode:mode,       // interpolation mode
    loop:n           // repetitions (0 = infinite)
)
```

Parameters

vals: (required)

Defines the colour values or behaviour.

Discrete Colour Values

Supported formats: - **Hex**: `#f00`, `#ff8800` - **RGB**: `rgb(255, 0, 0)` - **HSL**: `hsl(120, 80%, 50%)` - **Named**: `red`, `cyan`, `magenta`

```
color(vals:[#f00, #0f0, #00f], dur:2)
color(vals:[red, yellow, green], dur:3)
```

Hue Cycling (Continuous)

Full 360 hue rotation:

```
color(vals:hue, dur:6)
```

Range-limited cycle:

```
color(vals:hue(120, 240), dur:4)
```

This cycles only through the green-to-blue range.

Pattern Sequences

```
color(vals:Pseq([#f00, #ff0, #0ff], 3), dur:1)
color(vals:Prand([#f80, #08f, #8f0], inf), dur:0.8)
```

---

dur:

Duration in seconds for one transition (or one full hue cycle).

```
color(vals:[#f00, #0f0], dur:2)      // 2 seconds per transition
color(vals:hue, dur:10)               // 10 seconds per full rotation
```

Duration can also be a pattern:

```
color(vals:[#f00, #0f0, #00f], dur:Pseq([1, 0.5, 2], inf))
```

---

mode:

Controls interpolation behaviour.

Mode	Behaviour
`linear`	Smooth interpolation (default)
`alt`	Ping-pong (forward then reverse)
`step`	Hard switching, no interpolation

```
// Smooth transitions
color(vals:[#f00, #0f0], dur:2, mode:linear)

// Bounce back and forth
color(vals:[#f00, #0f0], dur:2, mode:alt)
```

```
// Instant colour changes
color(vals:[#f00, #0f0, #00f], dur:0.5, mode:step)
```

---

loop:

Number of repetitions.

```
loop:3    // repeat three times then stop
loop:0    // infinite (default for hue cycling)
loop:1    // play once
```

---

uid:

Unique identifier for the animation instance.

```
color(uid:myColor, vals:hue, dur:5)
```

---

trig:

Trigger mode.

Value	Behaviour
`auto`	Start immediately on load (default)
`playhead`	Start when playhead reaches element

```
color(vals:[#f00, #00f], dur:2, trig:playhead)
```

---

tdelay:

Delay before animation starts (seconds).

```
color(vals:hue, dur:6, tdelay:2)    // wait 2 seconds then start
```

---

prestate:

Initial visibility state before animation starts.

Value	Effect
`show`	Visible immediately (default)
`hide`	Hidden until triggered
`ghost`	Semi-transparent (30% opacity)
`fadein(ms)`	Fade in over specified milliseconds
`ghostClickable(ms)`	Ghost until clicked

```
color(vals:hue, dur:5, prestate:hide, trig:playhead)
color(vals:[#f00, #0f0], dur:2, prestate:fadein(1000))
```

---

target:

Specifies which SVG property to animate.

Value	Effect
`both`	Animate fill and stroke (default)
`fill`	Animate fill only
`stroke`	Animate stroke only

```
// Animate fill only
color(vals:[#f00, #00f], dur:2, target:fill)

// Animate stroke only (useful for outlines)
color(vals:hue, dur:10, target:stroke)

// Animate both (default)
color(vals:[#f00, #0f0], dur:2, target:both)
```

---

osc:

Enable OSC output.

Value	Behaviour
`0`	Disabled (default)
`1`	Continuous output

```
color(vals:hue, dur:10, osc:1)
```

OSC output includes: - `h` Hue (0360) - `s` Saturation (0100) - `l` Lightness (0100) - `hNorm` Normalised hue (01) - `sNorm` Normalised saturation (01) - `lNorm` Normalised lightness (01)

---

oscaddr:

Custom OSC address.

```
color(vals:hue, dur:8, osc:1, oscaddr:/visuals/bg/hue)
```

Colour Interpolation

All interpolation occurs in **HSL colour space**:

- **Hue**: Circular interpolation (shortest angular path)
- **Saturation**: Linear interpolation
- **Lightness**: Linear interpolation

This means transitions between colours feel natural and avoid muddy intermediate tones.

Shortest Path Hue

When transitioning between hues, Oscilla takes the shortest path around the colour wheel:

```
#f00 → #00f    // Red to Blue: goes via magenta (shorter)
                // NOT via yellow/green (longer)
```

Patterns

Pseq Sequential

Plays values in order, repeating a specified number of times.

```
color(vals:Pseq([#f00, #ff0, #0f0], 3), dur:1)
// Plays: red, yellow, green, red, yellow, green, red, yellow, green
```

Prand Random

Selects randomly, may repeat the same value consecutively.

```
color(vals:Prand([#f00, #0f0, #00f], inf), dur:0.5)
```

Pxrand Exclusive Random

Selects randomly but never repeats the same value twice in a row.

```
color(vals:Pxrand([#f00, #0f0, #00f], inf), dur:0.5)
```

Pshuf Shuffle

Shuffles the values, plays through, then reshuffles.

```
color(vals:Pshuf([#f00, #ff0, #0f0, #0ff], 2), dur:0.8)
```

Examples

Discrete Colour Melody

```
color(vals:[#f00, #ff0, #0ff, #f0f], dur:1.5)
```

Warm to Cool Transition

```
color(vals:[#f80, #fa0, #ff0, #8f0, #0f8, #0ff], dur:6)
```

Breathing Glow

```
color(vals:[hsl(200,80%,30%), hsl(200,80%,60%)], mode:alt, dur:2, loop:0)
```

Continuous Spectrum

```
colour(vals:hue, dur:20, loop:0)
```

Limited Hue Range

```
color(vals:hue(0, 60), dur:4, loop:0)    // Red to yellow only
color(vals:hue(180, 300), dur:5)          // Cyan to magenta
```

Rhythmic Colour Pulses

```
color(vals:[#fff, #000], mode:step, dur:0.25, loop:0)
```

Pattern with Variable Timing

```
color(
    vals:Pseq([#f00, #0f0, #00f], inf),
    dur:Pseq([1, 0.5, 2], inf)
)
```

Playhead-Triggered Fade

```
color(
    vals:[#333, #f80],
    dur:3,
    trig:playhead,
    prestate:fadein(500)
)
```

With OSC Output

```
colour(
    vals:hue,
    dur:15,
    loop:0,
    osc:1,
    oscaddr:/stage/wash/hue
)
```

Named Colours Reference

Oscilla recognises these named colours:

Name	Hue
`red`	0
`orange`	30
`yellow`	60

Name	Hue
`green`	120
`cyan`	180
`blue`	240
`purple`	270
`magenta`	300
`pink`	330
`white`	
`black`	
`gray` / `grey`	

For precise control, use hex, RGB, or HSL notation.

Tips

1. **Use HSL for control**: `hsl(h, s%, l%)` gives direct control over hue, saturation, and lightness.
2. **Hue cycling for ambience**: `vals:hue` creates slow, evolving colour fields without discrete steps.
3. **Step mode for rhythm**: `mode:step` creates hard cuts useful for rhythmic visual accents.
4. **Alt mode for breathing**: `mode:alt` with two colours creates organic pulsing effects.
5. **Combine with other cues**: Colour animations work alongside scale, rotate, and o2p animations on the same element.

See Also

- rotate() Rotation animation
- scale() Scale animation

- o2p() Object-to-path animation
- Patterns Pattern system reference

cue:fade() fade or pulse an SVG element

Fades the opacity of the triggering SVG object (or a specified target) between two opacity values. Can be used for smooth fade-ins, fade-outs, or pulsing effects. The transition speed, easing, and timing can all be controlled.

Syntax

```
cue:fade(mode:<in|out|pulse|blink>, dur:<seconds>,
         from:<0–1>, to:<0–1>, ease:<easing>,
         time:<seconds>, delay:<seconds>, hold:<seconds>,
         target:<id>)
```

Arguments

Argument	Description
mode	type of fade: `"in"`, `"out"`, `"pulse"`, or `"blink"`
dur	duration of the fade in seconds
from	starting opacity (0 = transparent, 1 = opaque)
to	ending opacity
ease	easing type (e.g.`linear`, `easeInOutSine`, etc.)
time	start time offset in seconds (optional, for cue scheduling)
delay	delay before fade starts (seconds)
hold	time to hold at final opacity before reset or removal

Argument	Description
target	optional target element ID; if omitted, applies to the cue element itself

Behavior

- `cue:fade(mode:in)` smoothly fades in from `from` to `to` opacity.
- `cue:fade(mode:out)` fades out in reverse.
- `cue:fade(mode:pulse)` fades in and out continuously while active.
- `cue:fade(mode:blink)` alternates visibility quickly (use short `dur`).
- The cue supports timing offsets and can be combined with OSC or other cue types.
- When `target` is not provided, the fade applies to the object containing the cue.
- The `hold` argument determines how long the element stays visible before reverting or fading away.

Examples

```
cue:fade(mode:in, dur:2, from:0, to:1)
cue:fade(mode:out, dur:3, ease:easeOutSine)
cue:fade(mode:pulse, dur:1.5, from:0.2, to:1)
cue:fade(mode:blink, dur:0.5, time:10)
cue:fade(mode:in, dur:2, from:0, to:1, target:circle1)
cue:fade(mode:pulse, dur:2, hold:5, target:obj_light)
```

ui()

`ui()` applies **performer-facing UI changes** (HTML/CSS) during playback most commonly to **hide or reveal interface elements** such as the playhead, play zone, overlays, or control panels.

It targets **HTML elements (not SVG)** using a CSS selector and applies a controlled set of visual changes, optionally animated over time.

The UI cue system was **primarily designed to manage visibility of the playhead and play zone**, allowing scores to shift between guided, open, or reduced-UI performance states.

Syntax

```
ui("<css selector>", opacity:<number>, scale:<number>, x:<px>, y:<px>, rotate:<deg>, visible:<bool>, color:<css>,
↪  bg:<css>, z:<number>, dur:<seconds>)
```

What it affects

- Works on: HTML UI elements
- Does not affect: SVG score elements
- Multiple matches: All matching elements are updated

Built-in UI toggle button

Oscilla includes a **GUI button in the bottom-right corner** that toggles:

- Playhead
- Play zone

on and off.

This button uses the same internal UI logic as the `ui()` cue and exists to allow fast switching between visible guidance and reduced interface modes during performance.

Parameters

Transition

- `dur` (seconds, default `0`)

Visibility

- `visible:true|false`

Opacity

- `opacity` (number)

Transform

- `x`, `y` (px)
- `scale` (number)
- `rotate` (degrees)

Transforms are stateful and combined.

Color

- `color`
- `bg`

Layering

- `z` (z-index)

Examples

```
ui("#playhead", opacity:0, dur:0.3)
ui("#playhead", visible:false, dur:0.3)
ui("#playhead, #playzone", visible:false, dur:0.25)
ui("#playhead, #playzone", visible:true, dur:0.25)
```

Design notes

- Intended for performance-state control
- Touch- and tablet-friendly
- Reduces visual clutter
- Suitable for audience-facing projections

Chapter 5

Control and Interaction

The control layer allows performers to influence score behaviour in real time. While cues define what happens when the playhead crosses an element, the control system determines how those behaviours can be shaped during performance through live input.

At the centre of this system is a source-agnostic parameter bus. Control signals can originate from the controlXY touchpad interface, from external OSC sources, or from other cues within the score. These signals are routed to cue parameters through a binding system that allows any control source to modulate any parameter animation speed, colour, audio volume, OSC output values without the cue needing to know where the signal comes from.

This architecture supports a range of performance practices, from a single performer adjusting their own score to a conductor modulating parameters across the entire ensemble via a networked control surface.

Oscilla Control Plane Architecture & Usage Guide

Published signals are shared across the system, so animations can control audio parameters, slider-like interfaces can control animations, and multiple cues can influence each other in any combination. Because values are updated continuously, this also allows feedback systems where motion, sound, and interaction form coupled, dynamic behaviours within the score.

synth(uid:pad, freq:“fadeSineFreq.t[90,2000]”, env:{a:4})

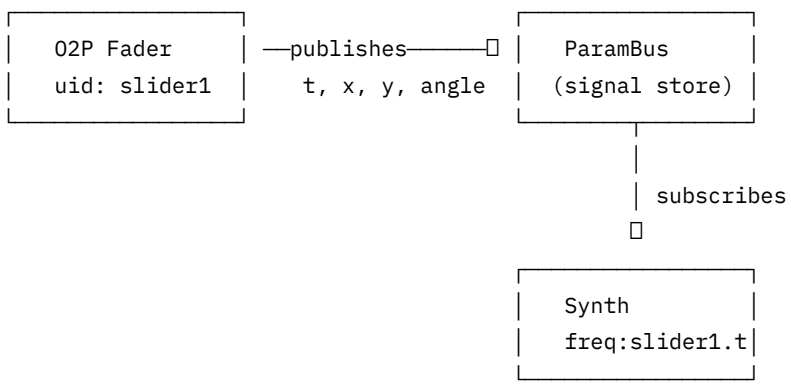
o2p(path:fadeSineFreq, trig:touch, osc:1, uid:fadeSineFreq, oscAddr:fadeSineFreq)

Overview

The Oscilla Control Plane enables **bidirectional signal flow** between cues. Any animation can publish its values as signals, and any synth or audio cue can subscribe to those signals to control its parameters in real-time.

This transforms Oscilla from a trigger-based score system into a **dynamic, signal-driven, executable score environment**.

Core Concept



Quick Start

1. Create a Controller (O2P Fader)

```
o2p(path:faderTrack, trig:touch, uid:myFader)
```

This fader automatically publishes: - `o2p:myFader.t` position along path (0-1) - `o2p:myFader.x` normalized X position (0-1) - `o2p:myFader.y` normalized Y position (0-1) - `o2p:myFader.angle` tangent angle in degrees

2. Bind a Synth Parameter

```
synth(uid:pad, freq:myFader.t[200,800], amp:0.2)
```

The `freq:myFader.t[200,800]` syntax means: - Subscribe to signal `o2p:myFader.t` - Map the 0-1 value to 200-800 Hz

3. Result

Moving the fader changes the synths frequency in real-time!

Signal Reference Syntax

Basic Binding

```
param:source.channel
```

- **param** The parameter to control (freq, amp, pan, etc.)
- **source** The uid of the publishing cue
- **channel** Which signal to subscribe to (t, x, y, angle, etc.)

Binding with Range

```
param:source.channel[min,max]
```

Maps the signal (assumed 0-1) to the specified output range.

Examples

DSL	Meaning
`freq:slider.t`	Freq follows slider position (0-1)
`freq:slider.t[200,2000]`	Freq mapped to 200-2000 Hz
`amp:fader.y[0,0.5]`	Amplitude mapped to 0-0.5
`pan:knob.x[-1,1]`	Pan mapped to full left-right
`cutoff:wheel.t[200,8000]`	Filter cutoff mapped to range

Published Signals by Cue Type

O2P Animation

Signal	Description	Range
`o2p:{uid}.t`	Position along path	0-1
`o2p:{uid}.x`	Normalized X in frame	0-1
`o2p:{uid}.y`	Normalized Y in frame	0-1
`o2p:{uid}.angle`	Tangent angle	degrees

Rotate Animation

Signal	Description	Range
`rotate:{uid}.angle`	Current angle	0-360
`rotate:{uid}.rad`	Angle in radians	0-2
`rotate:{uid}.norm`	Normalized angle	0-1

Scale Animation

Signal	Description	Range
`scale:{uid}.sx`	Scale X factor	varies
`scale:{uid}.sy`	Scale Y factor	varies
`scale:{uid}.uniform`	Average scale	varies

Bindable Parameters

Synth

Parameter	Default Range	Description
`freq` / `frequency`	20-20000	Oscillator frequency (Hz)
`amp` / `amplitude`	0-1	Output amplitude
`pan`	-1 to 1	Stereo position
`cutoff` / `filterFreq`	20-20000	Filter cutoff (Hz)
`q` / `resonance`	0.1-30	Filter resonance
`detune`	-1200 to 1200	Detune in cents

Audio / AudioPool / AudioImpulse

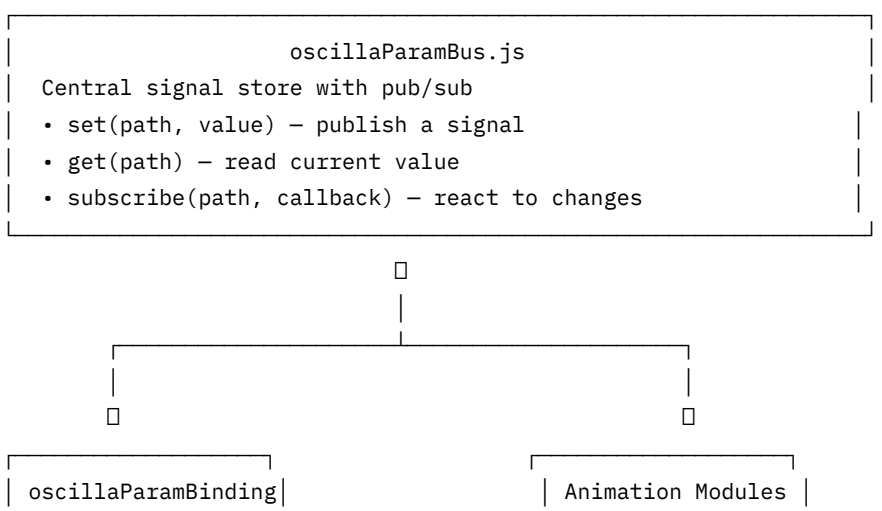
Parameter	Default Range	Description
`amp`	0-1	Playback amplitude
`pan`	-1 to 1	Stereo position
`pitch` / `rate`	0.25-4	Playback rate

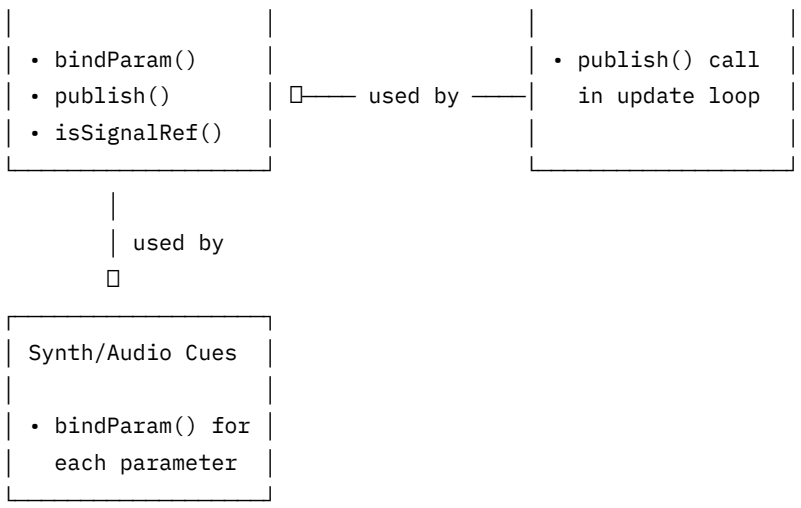
Effects (within synth)

Parameter	Default Range	Description
`delayTime`	0-2	Delay time (seconds)
`feedback` / `fb`	0-0.99	Delay feedback
`mix` / `wet`	0-1	Effect mix level

Architecture

Module Overview





File Descriptions

File	Purpose
`oscillaParamBus.js`	Central signal store with pub/sub
`oscillaParamBinding.js`	`bindParam()` and `publish()` helpers
`oscillaParserSignalRef.js`	Parser extension for signal ref syntax

Integration Guide

Adding Signal Publishing to a Cue

To make any animation publish signals, add **one line** to its update callback:

```
import { publish } from './oscillaParamBinding.js';

// In your animation's update callback:
update: () => {
    // ... existing animation logic ...

    // ADD THIS LINE:
    publish("o2p", cfg.uid, { t: pathT, x: normX, y: normY, angle });
}
```

Example: O2P Animation

Location: `oscillaAnimationO2p.js`, in `startContinuousO2P()`, around line 492

```
// BEFORE (existing code):
emitO2POsc({ cfg, uid: cfg.uid, path, point, pathT });

// AFTER (add this line):
import { publish } from './oscillaParamBinding.js';
// ... then in update callback:
emitO2POsc({ cfg, uid: cfg.uid, path, point, pathT });
publish("o2p", cfg.uid, { t: globalT, x: normX, y: normY, angle });
```

Note: Youll need to calculate `normX` and `normY` from the path bbox:

```
const bbox = path.getBBox();
const normX = (point.x - bbox.x) / bbox.width;
const normY = (point.y - bbox.y) / bbox.height;
```

Example: Rotate Animation

Location: `oscillaAnimationRotate.js`, in `handleRotateContinuous()`, around line 548

```
import { publish } from './oscillaParamBinding.js';

update: () => {
    // ... existing code ...
    const angle = getCurrentAngle(animEl, 0);

    // ADD THIS:
```

```
    publish("rotate", cfg.uid, { angle: angle });
}
```

Example: Scale Animation

Location: `oscillaAnimationScale.js`, in `handleScaleContinuous()`, around line 586

```
import { publish } from './oscillaParamBinding.js';

update: () => {
    // ... existing code ...
    const sx = parseFloat(tr.match(/scale\(([^,]+),/)?.[1] || 1);
    const sy = parseFloat(tr.match(/,\s*([^)]+)\)/)?.[1] || sx);

    // ADD THIS:
    publish("scale", cfg.uid, { sx, sy, uniform: (sx + sy) / 2 });
}
```

Adding Signal Binding to a Cue

To make parameters bindable in a cue, use `bindParam()`:

```
import { bindParam, isSignalRef } from './oscillaParamBinding.js';

function startMyCue(params) {
    const unbinders = [];

    // Bind frequency - works with both static and signal ref
    const freqBinding = bindParam(
        params.freq,
        (hz) => oscillator.frequency.setTargetAtTime(hz, 0, 0.02),
        { min: 20, max: 20000, default: 440 }
    );
    unbinders.push(freqBinding.unbind);

    // Use initial value
    oscillator.frequency.value = freqBinding.value;

    // ... later, on cleanup:
    unbinders.forEach(fn => fn());
}
```

Example: Synth Integration

Location: `oscillaSynth.js`, in `startSynthVoice()`, around line 673

```
import { bindParam } from './oscillaParamBinding.js';

function startSynthVoice(uid, ast, cueElement, opts) {
    const ctx = sharedAudioCtx;
    const params = extractParams(ast);
    const unbinders = [];

    // Frequency binding
    const freqBinding = bindParam(
        params.freq,
        (hz) => {
            if (voice.source?.kind === 'osc') {
                voice.source.node.frequency.setTargetAtTime(hz, ctx.currentTime, 0.02);
            }
        },
        { min: 20, max: 20000, default: 440 }
    );
    unbinders.push(freqBinding.unbind);

    // Amplitude binding
    const ampBinding = bindParam(
        params.amp,
        (amp) => {
            voice.graph.gain.gain.setTargetAtTime(amp, ctx.currentTime, 0.02);
        },
        { min: 0, max: 0.5, default: 0.1 }
    );
```

```
    unbinders.push(ampBinding.unbind);

    // ... create voice with initial values ...
    const freqHz = freqBinding.value;
    const amp = ampBinding.value;

    // Store unbinders on voice for cleanup
    voice._unbinders = unbinders;
}

function cleanupVoice(uid, voice) {
    // Unbind all signal subscriptions
    voice._unbinders?.forEach(fn => fn());
    // ... rest of cleanup ...
}
```

Parser Integration

Modifying the Parser

The parser needs to recognize signal references in parameter values. Add this to the value extraction logic:

Location: `oscillaParser.js`, in `cstToAst()` synth/audio sections

```
import { maybeConvertToSignalRef } from './oscillaParserSignalRef.js';

// When extracting a parameter value:
let val = extractRawValue(valueNode);
val = maybeConvertToSignalRef(val, paramName);
```

What the Parser Outputs

Input DSL:

```
synth(uid:pad, freq:fader1.t[200,800], amp:0.2)
```

Output AST:

```
{
    type: "cueSynth",
    args: [
        { type: "uid", value: "pad" },
        {
            type: "freq",
            value: {
                type: "signalRef",
                source: "fader1",
                channel: "t",
                range: [200, 800]
            }
        },
        { type: "amp", value: 0.2 }
    ]
}
```

Debugging

Enable Debug Mode

```
// In browser console:
oscillaParamBus.setDebugMode(true);
```

This logs all signal changes.

Inspect Current Signals

```
// List all signals:
oscillaParamBus.list();

// List signals from a specific source:
oscillaParamBus.list("o2p:");

// Get current value:
oscillaParamBus.get("o2p:fader1.t");
```

```
// Get snapshot of all values:
oscillaParamBus.snapshot();
```

Test Signal Publishing

```
// Manually publish a signal:
oscillaParamBus.set("o2p:test.t", 0.5);

// Watch a signal:
const unsub = oscillaParamBus.subscribe("o2p:test.t", (value, path) => {
    console.log(`${path} = ${value}`);
});

// Later: unsub() to stop watching
```

Common Patterns

XY Pad Synth

```
o2p(path:xyPad, trig:touch, uid:xy)
synth(uid:pad, freq:xy.x[200,2000], amp:xy.y[0,0.5])
```

Rotation Filter

```
rotate(dur:4, loop:0, uid:wheel)
synth(uid:drone, freq:220, cutoff:wheel.norm[200,4000])
```

Multiple Controllers One Synth

```
o2p(path:freqSlider, trig:touch, uid:fSlider)
o2p(path:ampSlider, trig:touch, uid:aSlider)
synth(uid:lead, freq:fSlider.t[100,1000], amp:aSlider.t[0,0.3])
```

One Controller Multiple Synths

```
o2p(path:masterFader, trig:touch, uid:master)
synth(uid:bass, freq:master.t[50,200], amp:0.2)
synth(uid:mid, freq:master.t[200,800], amp:0.15)
synth(uid:high, freq:master.t[800,4000], amp:0.1)
```

Limitations & Notes

1. **Signal Publishing Rate:** Signals are published at ~60fps to avoid overwhelming the system.
2. **Binding Lifecycle:** Bindings are automatically cleaned up when the cue stops (if you call the unbind functions).
3. **No Circular Dependencies:** The system doesnt prevent circular signal routing. Avoid creating feedback loops unless intentional.
4. **Static Parameters:** Some parameters cant be changed after cue start (e.g., `wave` type, audio `src`). These will use the initial value even if bound.
5. **Signal Range:** All signals are assumed to be in the 0-1 range. Use the `[min,max]` syntax to map to your desired output range.

Summary Checklist

To Make an Animation Publish Signals:

1. Import `publish` from `oscillaParamBinding.js`
2. Add `publish("type", uid, { channel: value })` to update callback
3. Done!

To Make a Cue Accept Signal Bindings:

1. Import `bindParam` from `oscillaParamBinding.js`
2. Wrap parameter initialization with `bindParam()`
3. Store unbind functions for cleanup

4. Call unbind functions when cue stops
5. Update parser to recognize signal refs (one-time)

Version History

- **v1.0** Initial control plane implementation
  - ParamBus signal store
  - bindParam/publish helpers
  - Signal reference syntax support

controlXY Multitouch XY Control & Score Animation System

`controlXY` transforms your score into an **interactive animation canvas**. It defines persistent, multi-touch XY control surfaces that enable composers to create **complex choreographed animations** directly within the score the **score-as-instrument, instrument-as-score**.

Unlike traditional time-based cues, `controlXY` provides **continuous spatial control** that can be: - **Pre-programmed** as animated sequences (choreography) - **Performed live** as a tactile control surface - **Hybrid** switching between automation and manual control

As a bonus, all movements can simultaneously **transmit OSC** for external synthesis and media control.

Oscilla OSC controller design in Inkscape

Your browser does not support the video tag.

The controlXY multitouch OSC controller created in Inkscape. Each control object can be dragged freely across the canvas, sending X, Y, and rotation (twist) values via OSC. The orange rings indicate touch mode (trig:touch) is active. Multiple simultaneous touches are supported, with each object independently sending its position and rotation state. This example shows the page view with preset management and parameter display.

Core Concept: Score Animation Through Spatial Control

Think of `controlXY` as a **spatial modulation source** embedded in your score:

1. **Define control pads** with draggable handles
2. **Bind handle positions** to visual/sonic parameters
3. **Animate via presets & sequences** OR **perform live**
4. **Record/playback** spatial gestures as part of the composition

This inverts the traditional "playhead reads static notation" model instead, **notation becomes dynamic**, responding to spatial control in real-time.

Syntax

```
controlXY(
  uid: <string>,
  touchpt: <element-id> | [<id1>, <id2>, ...],
  handle: <element-id> | [<id1>, <id2>, ...],  // optional: rotation handles
  hmode: continuous | limited | [mode1, mode2, ...],  // optional: rotation behavior
  rotRange: <degrees>,  // optional: rotation range for limited mode
  bounds: <element-id> | "self",
  label: <bool>,
  osc: <bool|number>,
  oscAddr: <string>
)
```

Note: For backwards compatibility, if only `handle:` is specified (without `touchpt:`), it refers to XY control elements with no rotation.

Parameters

Parameter	Type	Required	Default	Description
`uid`	string	yes		Unique identifier for the control pad. Used for publishing and parameter binding.
`touchpt`	element id or array	yes*		ID(s) of SVG element(s) for XY position control. Use array syntax for multitouch: `[dot1, dot2, dot3]`
`handle`	element id or array	no		ID(s) of SVG element(s) for rotation control. Can be single shared handle or parallel array. Omit for XY-only control.
`hmode`	`continuous` | `limited` | array	no	`continuous`	Rotation behavior: `continuous` wraps at 360, `limited` stops at min/max like a potentiometer. Use array for per-handle modes.
`rotRange`	number	no	`270` (limited)`360` (continuous)	Rotation range in degrees for `limited` mode. Default 270 provides 7 oclock to 4 oclock range.
`bounds`	element id or “self”	no	`self`	ID of the SVG element that defines the XY constraint area. Defaults to the element the DSL is attached to.
`label`	boolean	no	`false`	Show live value labels above each handle. Useful for debugging and performance.
`osc`	`true` | `false` | number	no	`false`	Enable OSC output. If a number is given, it specifies throttle interval in ms. Default throttle 30 ms.
`oscAddr`	string	no	`controlXY/<uid>`	Custom OSC address (without leading `/`).

*For backwards compatibility, `handle:` can be used instead of `touchpt:` when not using rotation.

Coordinate System

- **X axis**

  - Left = `0.0`
  - Right = `1.0`
- **Y axis** (musical convention)
  - Bottom = `0.0`
  - Top = `1.0`

All values are **normalized** to the bounding box, making animations resolution-independent.

Rotation Control (Optional)

In addition to XY positioning, handles can have **rotation control** - either as a shared twist handle or individual knobs per touchpoint.

Rotation Modes: `hmode` Parameter

`continuous` (default) - Full 360 rotation with wraparound - Rotating past 360 wraps to 0 - Natural for: azimuth, phase, circular parameters - Provides endless rotation without hard stops

`limited` - **Hard stops at minimum and maximum** - like a real potentiometer - Default range: 270 (7 oclock to 4 oclock) - **Cannot wrap** from maximum to minimum or vice versa - Safe for: volume, gain, pan - prevents accidental jumps - Custom range via `rotRange:` parameter (e.g., `rotRange:180` for half rotation)

Rotation Coordinate System

- **0** = 7 oclock position (minimum)
- **90** = 10 oclock position
- **180** = 1 oclock position

- **270** = 4 oclock position (maximum for default `limited` mode)
- Rotation proceeds **clockwise**

Example Behaviors

Continuous mode (default):

```
Rotate clockwise: 350° → 360° → 0° → 10° (wraps smoothly)
Perfect for spatial direction, phase rotation, etc.
```

Limited mode:

```
At 0°: Try rotating counterclockwise → STOPS (hard stop at minimum)
At 270°: Try rotating clockwise → STOPS (hard stop at maximum)
Perfect for volume faders, mix levels, etc.
```

Published Signals

XY Only (No Rotation)

```
controlXY:<uid>.x        // 0.0 – 1.0
controlXY:<uid>.y        // 0.0 – 1.0
controlXY:<uid>.handle   // handle element id
```

XY + Rotation

```
controlXY:<uid>.x        // 0.0 – 1.0
controlXY:<uid>.y        // 0.0 – 1.0
controlXY:<uid>.p        // 0.0 – 1.0 (rotation, normalized)
controlXY:<uid>.handle   // handle element id
```

Multiple Handles

```
controlXY:<uid>.<handleId>.x   // 0.0 – 1.0
controlXY:<uid>.<handleId>.y   // 0.0 – 1.0
controlXY:<uid>.<handleId>.p   // 0.0 – 1.0 (rotation, if handle has rotation)
```

Rotation normalization: - **Continuous mode:** `p = angle / 360` (full circle = 0.0 to 1.0) - **Limited mode:** `p = angle / rotRange` (range = 0.0 to 1.0)

These signals can be bound to **any parameter** in the system, creating direct connections between spatial control and visual/sonic outcomes.

Setup Examples

Basic Single Handle (XY Only)

```
<rect id="pad1" x="100" y="100" width="400" height="300"
      fill="#222" stroke="#666"
      cue="controlXY(uid:pad1, touchpt:dot1)"/>

<circle id="dot1" cx="0" cy="0" r="12" fill="#ff4444"/>
```

XY + Shared Rotation Handle

```
<rect id="synthPad" x="100" y="100" width="400" height="300"
      fill="#222" stroke="#666"
      cue="controlXY(uid:synth, touchpt:[dot1, dot2], handle:knob, label:true)"/>

<circle id="dot1" cx="0" cy="0" r="12" fill="#ff4444"/>
<circle id="dot2" cx="0" cy="0" r="12" fill="#44ff44"/>
<circle id="knob" cx="0" cy="20" r="6" fill="#ffaa00"/>
```

One rotation handle shared by both touchpoints.

Volume Mixer with Hard Stops

```
<rect id="mixer" x="100" y="100" width="600" height="400"
      fill="#111"
      cue="controlXY(uid:mixer,
                     touchpt:[vol1, vol2, vol3, vol4],
                     handle:fader,
                     hmode:limited,
                     label:true)"/>

<circle id="vol1" cx="0" cy="0" r="10" fill="#ff4444"/>
<circle id="vol2" cx="0" cy="0" r="10" fill="#44ff44"/>
<circle id="vol3" cx="0" cy="0" r="10" fill="#4444ff"/>
<circle id="vol4" cx="0" cy="0" r="10" fill="#ffff44"/>
<circle id="fader" cx="0" cy="15" r="5" fill="#fff"/>
```

Rotation stops at min/max - safe for volume control, no wraparound.

Mixed Rotation Modes (Array Syntax)

```
<rect id="hybrid" x="100" y="100" width="600" height="300"
      cue="controlXY(uid:hybrid,
                     touchpt:[vol, phase, pan],
                     handle:knob,
                     hmode:[limited, continuous, limited],
                     label:true)"/>
```

Volume and pan use `limited` (hard stops), phase uses `continuous` (wraps).

Multi-Handle (Multitouch, XY Only)

```
<rect id="mixer" x="100" y="100" width="600" height="400"
      fill="#111"
      cue="controlXY(uid:mixer, touchpt:[fader1,fader2,fader3,fader4], label:true)"/>
```

```
<circle id="fader1" cx="0" cy="0" r="10" fill="#ff4444"/>
<circle id="fader2" cx="0" cy="0" r="10" fill="#44ff44"/>
<circle id="fader3" cx="0" cy="0" r="10" fill="#4444ff"/>
<circle id="fader4" cx="0" cy="0" r="10" fill="#ffff44"/>
```

With OSC Output (XY + Rotation)

```
<rect id="synthControl" x="50" y="50" width="300" height="300"
      cue="controlXY(uid:synth, touchpt:dot1, handle:knob, osc:true, oscAddr:synth/xy)"/>
```

Sends X, Y, and rotation values via OSC.

Animation Workflow: Creating Score Choreography

The true power of `controlXY` emerges when you **pre-program spatial animations** as part of your composition.

Step 1: Save Spatial States as Presets

Using the Preset UI: 1. Press **Alt+Shift+P** to open the preset manager (or click on the pad) 2. Move handles to desired positions 3. Type a preset name (e.g., "intro_position") 4. Click Save

Via DSL (triggered by playhead or buttons):

```
<!-- Save current state when playhead passes -->
<rect x="100" y="0" width="2" height="600"
      cue="ui(action:'controlXYSave', preset:'stateA')"
      fill="red" opacity="0.3"/>

<!-- Button to save state -->
<g cue="button(
        trigger:ui(action:'controlXYSave', preset:'verse1'),
        style(label:'Save Verse', x:10, y:10)
      )"/>
```

Via Console (for experimentation):

```
// Save all controlXY instances
window.controlXYPresets.save('stateA');

// Save specific pad only
window.controlXYPresets.save('stateB', 'pad1');
```

Step 2: Recall States with Animation

Instant recall (no tween):

```
<rect x="500" y="0" width="2" height="600"
      cue="ui(action:'controlXYRecall', preset:'stateA')"
      fill="blue" opacity="0.3"/>
```

Smooth tween (2 seconds, easing):

```
<rect x="1000" y="0" width="2" height="600"
      cue="ui(action:'controlXYRecall', preset:'stateB', dur:2, ease:'easeInOutSine')"
      fill="green" opacity="0.3"/>
```

Via button:

```
<g cue="button(
        trigger:ui(action:'controlXYRecall', preset:'chorus', dur:3, ease:'easeOutElastic'),
        style(label:' Chorus', x:10, y:50)
      )"/>
```

Step 3: Define Sequences (Choreographed Animation)

Sequences are **playlists of presets** that play automatically.

Define sequence via DSL:

```
<!-- Define a 3-state sequence -->
<rect x="100" y="0" width="2" height="600"
      cue="ui(action:'controlXYDefineSequence',
              name:'intro_dance',
              steps:'stateA,stateB,stateC')"
      fill="yellow" opacity="0.3"/>
```

Play the sequence:

```
<!-- Auto-play when playhead reaches this point -->
<rect x="200" y="0" width="2" height="600"
      cue="ui(action:'controlXYSequence',
```

```
              seq:'intro_dance',
              dur:2,
              ease:'easeInOutSine',
              loop:false)"
      fill="cyan" opacity="0.3"/>
```

Stop sequence:

```
<rect x="1500" y="0" width="2" height="600"
      cue="ui(action:'controlXYSequenceStop')"
      fill="red" opacity="0.5"/>
```

Via buttons (interactive control):

```
<!-- Define sequence button -->
<g cue="button(
        trigger:ui(action:'controlXYDefineSequence',
                   name:'verse_pattern',
                   steps:'v1,v2,v3,v1'),
        style(label:'Define Pattern', x:10, y:10)
      )"/>

<!-- Play button -->
<g cue="button(
        trigger:ui(action:'controlXYSequence',
                   seq:'verse_pattern',
                   dur:1.5,
                   loop:true),
        style(label:' Play', x:10, y:50)
      )"/>

<!-- Stop button -->
<g cue="button(
        trigger:ui(action:'controlXYSequenceStop'),
        style(label:' Stop', x:10, y:90)
      )"/>
```

Rotation Control Use Cases

When to Use `hmode:limited` (Hard Stops)

Perfect for: - **Volume/Gain controls** - prevents accidental jump from loud to silent - **Mix levels** - fader-style controls - **Pan controls** - left-center-right with hard stops - **Envelope parameters** - attack, decay, sustain, release - **Filter cutoff** - when you need precise min/max boundaries

Example:

```
<rect cue="controlXY(uid:mixer,
                   touchpt:[ch1, ch2, ch3],
                   handle:volKnob,
                   hmode:limited,
                   rotRange:270)"/>
```

Provides 270 of rotation (7 oclock to 4 oclock) with hard stops at both ends.

When to Use `hmode:continuous` (Wrapping)

Perfect for: - **Spatial positioning** - azimuth, direction, angle - **Phase controls** - where 0 = 360 naturally - **LFO rate/frequency** - continuous spinning - **Cyclical parameters** - anything that wraps by nature - **Visual rotation** - spinning objects, wheels, orbits

Example:

```
<rect cue="controlXY(uid:spatial,
                   touchpt:[src1, src2],
                   handle:direction,
                   hmode:continuous)"/>
```

Full 360 rotation with smooth wraparound at the 0/360 boundary.

Mixed Modes for Hybrid Control

Different parameters on the same pad can have different rotation behaviors:

```
<rect cue="controlXY(uid:synth,
                   touchpt:[cutoff, resonance, phase, gain],
                   handle:knob,
                   hmode:[limited, limited, continuous, limited])"/>
```

- **cutoff, resonance, gain:** Hard stops (safe for audio levels)
- **phase:** Wraps naturally (0 = 360)

Complete Animation Example: Verse-Chorus-Bridge

Heres a **full composition workflow** showing how to choreograph spatial animation:

```
<!-- ===== SETUP: Define the control pad ===== -->
<rect id="controlPad" x="100" y="100" width="600" height="400"
      fill="#111" stroke="#333"
      cue="controlXY(uid:mixer, handle:[dot1,dot2,dot3], label:true)"/>

<circle id="dot1" cx="0" cy="0" r="12" fill="#ff4444"/>
<circle id="dot2" cx="0" cy="0" r="12" fill="#44ff44"/>
<circle id="dot3" cx="0" cy="0" r="12" fill="#4444ff"/>

<!-- ===== PLAYHEAD TRIGGERS: Auto-save states ===== -->
<!-- Measure 1: Save intro position -->
<rect x="100" y="0" width="1" height="600"
      cue="ui(action:'controlXYSave', preset:'intro')"
      fill="transparent"/>

<!-- Measure 5: Save verse position -->
<rect x="500" y="0" width="1" height="600"
      cue="ui(action:'controlXYSave', preset:'verse')"
      fill="transparent"/>

<!-- Measure 13: Save chorus position -->
<rect x="1300" y="0" width="1" height="600"
      cue="ui(action:'controlXYSave', preset:'chorus')"
      fill="transparent"/>

<!-- Measure 21: Save bridge position -->
<rect x="2100" y="0" width="1" height="600"
      cue="ui(action:'controlXYSave', preset:'bridge')"
      fill="transparent"/>

<!-- ===== PLAYHEAD AUTOMATION: Recall with tweens ===== -->
<!-- M9: Tween to verse over 2 seconds -->
<rect x="900" y="0" width="2" height="600"
      cue="ui(action:'controlXYRecall', preset:'verse', dur:2, ease:'easeInOutSine')"
      fill="blue" opacity="0.4"/>

<!-- M17: Quick snap to chorus -->
<rect x="1700" y="0" width="2" height="600"
      cue="ui(action:'controlXYRecall', preset:'chorus', dur:0.5, ease:'easeOutQuad')"
      fill="green" opacity="0.4"/>

<!-- M25: Elastic bounce to bridge -->
<rect x="2500" y="0" width="2" height="600"
      cue="ui(action:'controlXYRecall', preset:'bridge', dur:3, ease:'easeOutElastic')"
      fill="purple" opacity="0.4"/>

<!-- ===== SEQUENCES: Complex multi-step animations ===== -->
<!-- M29: Define and play verse pattern -->
<rect x="2900" y="0" width="1" height="600"
      cue="ui(action:'controlXYDefineSequence',
              name:'verse_pattern',
              steps:'verse,intro,verse')"
      fill="transparent"/>

<rect x="2950" y="0" width="2" height="600"
      cue="ui(action:'controlXYSequence',
              seq:'verse_pattern',
              dur:1,
              loop:false)"
```

```
      fill="yellow" opacity="0.4"/>

<!-- M40: Stop all automation, return to intro -->
<rect x="4000" y="0" width="2" height="600"
      cue="ui(action:'controlXYSequenceStop')"
      fill="red" opacity="0.5"/>

<rect x="4020" y="0" width="2" height="600"
      cue="ui(action:'controlXYRecall', preset:'intro', dur:4, ease:'easeInOutSine')"
      fill="cyan" opacity="0.4"/>

<!-- ===== MANUAL OVERRIDE: Buttons for live performance ===== -->
<!-- Performers can override automation at any time -->
<g cue="button(
        trigger:ui(action:'controlXYRecall', preset:'intro', dur:2),
        style(label:' Intro', x:10, y:500, width:80)
      )"/>

<g cue="button(
        trigger:ui(action:'controlXYRecall', preset:'verse', dur:1.5),
        style(label:'V Verse', x:100, y:500, width:80)
      )"/>

<g cue="button(
        trigger:ui(action:'controlXYRecall', preset:'chorus', dur:1),
        style(label:'C Chorus', x:190, y:500, width:80)
      )"/>

<g cue="button(
        trigger:ui(action:'controlXYSequenceStop'),
        style(label:' Stop Auto', x:280, y:500, width:100)
      )"/>
```

Binding to Score Elements: Creating Visual Animations

This is where **spatial control becomes visual transformation**.

Position Control

```
<!-- Dot X position controls rectangle X position -->
<rect id="box1"
      cue="scale(uid:box1, tx:mixer.dot1.x[-200,200])"/>
```

Size/Scale Control

```
<!-- Dot Y controls vertical scale -->
<rect id="box2"
      cue="scale(uid:box2, sy:mixer.dot1.y[0.5,2.0])"/>
```

Rotation Control

```
<!-- Map X position to rotation angle -->
<g id="spinner"
   cue="rotate(uid:spinner, values:mixer.dot1.x[0,360])"/>
```

Binding Handle Rotation (p)

```
<!-- Map the handle's twist (p) to element rotation -->
<g id="dial"
   cue="rotate(uid:dial, values:mixer.ch1.p[0,360])"/>

<!-- Use p for filter cutoff via OSC binding -->
<rect id="filterPad"
      cue="controlXY(uid:filter, touchpt:dot1, handle:knob, hmode:limited, osc:true)"/>
```

When handles have rotation, the `p` value (0.01.0) is saved in presets, tweened between presets, and published alongside `x` and `y`.

Color Control

```
<!-- Y position controls hue -->
<circle id="colorDot"
        cue="color(uid:colorDot, hue:mixer.dot1.y[0,360])"/>
```

Opacity Control

```
<!-- X position fades element -->
<rect id="fader"
      cue="fade(uid:fader, opacity:mixer.dot1.x)"/>
```

Multi-Parameter Complex Animation

```
<!-- Dot1 controls rotation, Dot2 controls scale, Dot3 controls opacity -->
<g id="complexShape">
  <rect cue="rotate(uid:r1, values:mixer.dot1.x[0,360])
             scale(uid:s1, sx:mixer.dot2.x[0.5,2], sy:mixer.dot2.y[0.5,2])
             fade(uid:f1, opacity:mixer.dot3.y)"/>
</g>
```

Spatial Navigation

```
<!-- Two handles define start/end points of a loop region -->
<g cue="o2p(path:loopA, start:mixer.dot1.x, end:mixer.dot2.x)"/>
```

Advanced: Programmatic Animation via Console

For complex choreography, you can script animations directly:

Tweening Without Presets

```
// Tween specific handles to target positions over 2 seconds
window.controlXYPresets.tweenTo({
  pad1: {
    dot1: { x: 0.5, y: 0.5 },
    dot2: { x: 0.2, y: 0.8 }
  }
}, { dur: 2, ease: 'easeInOutSine' });

// Include rotation (p) — tweens smoothly alongside x/y
window.controlXYPresets.tweenTo({
  mixer: {
    ch1: { x: 0.3, y: 0.7, p: 0.75 },
    ch2: { x: 0.8, y: 0.2, p: 0.25 }
  }
}, { dur: 3, ease: 'easeOutElastic' });
```

Complex Sequences with Per-Step Timing

```
// Define sequence with variable timing
window.controlXYPresets.defineSequence('complex_dance', [
  { preset: 'position1', dur: 2 },
  { preset: 'position2', dur: 1 },
  { preset: 'position3', dur: 3 },
  { preset: 'position1', dur: 2 }
]);

// Play with custom options
window.controlXYPresets.playSequence('complex_dance', {
  loop: true,
  onStep: (step, preset) => console.log(`Now: ${preset}`)
});
```

Per-Handle Timing Control

```
// Staggered animation - each handle moves independently
window.controlXYPresets.recall('myPreset', {
  handles: {
    dot1: { dur: 2, delay: 0, ease: 'easeInOutSine' },
    dot2: { dur: 1.5, delay: 0.5, ease: 'easeOutQuad' },
    dot3: { dur: 1, delay: 1, ease: 'easeOutElastic' }
  }
});
```

Easing Functions

Choose from 15 easing functions to shape your animations:

Number	Name	Character
0	`linear`	Constant speed
1	`easeInSine`	Slow start
2	`easeOutSine`	Slow end
3	`easeInOutSine`	Smooth both ends

Number	Name	Character
4	`easeInQuad`	Accelerate
5	`easeOutQuad`	Decelerate
6	`easeInOutQuad`	Smooth acceleration
7	`easeInCubic`	Strong acceleration
8	`easeOutCubic`	Strong deceleration
9	`easeInOutCubic`	Powerful smooth
10	`easeInBack`	Anticipation (goes backward first)
11	`easeOutBack`	Overshoot
12	`easeInOutBack`	Anticipation + overshoot
13	`easeInElastic`	Elastic snap-in
14	`easeOutElastic`	Bouncy arrival

Use by name or number in DSL:

```
<!-- By name -->
cue="ui(action:'controlXYRecall', preset:'state1', dur:2, ease:'easeOutElastic')"

<!-- By number -->
cue="ui(action:'controlXYRecall', preset:'state1', dur:2, ease:14)"
```

Complete DSL Action Reference

All `controlXY` preset actions work through `ui(action:...)` syntax.

1. Save Preset

```
<!-- Save all pads -->
ui(action:"controlXYSave", preset:"stateName")

<!-- Save specific pad -->
ui(action:"controlXYSave", preset:"stateName", uid:"pad1")
```

Examples:

```
<!-- Playhead trigger -->
<rect x="500" cue="ui(action:'controlXYSave', preset:'verse1')"/>

<!-- Button -->
<g cue="button(
        trigger:ui(action:'controlXYSave', preset:'myState'),
        style(label:'Save State', x:10, y:10)
      )"/>
```

2. Recall Preset

```
<!-- Instant -->
ui(action:"controlXYRecall", preset:"stateName")

<!-- With tween -->
ui(action:"controlXYRecall", preset:"stateName", dur:2, ease:"easeInOutSine")
```

Examples:

```
<!-- Playhead trigger with animation -->
<rect x="1000"
      cue="ui(action:'controlXYRecall', preset:'chorus', dur:3, ease:'easeOutElastic')"/>

<!-- Button with quick snap -->
<g cue="button(
        trigger:ui(action:'controlXYRecall', preset:'breakdown', dur:0.5),
        style(label:' Breakdown', x:10, y:50)
      )"/>
```

3. Define Sequence

```
ui(action:"controlXYDefineSequence", name:"seqName", steps:"preset1,preset2,preset3")
```

Examples:

```
<!-- Define on load -->
<rect x="100"
      cue="ui(action:'controlXYDefineSequence',
              name:'intro_pattern',
```

```
              steps:'a,b,c,a')"/>

 <!-- Button to create sequence -->
 <g cue="button(
        trigger:ui(action:'controlXYDefineSequence',
                   name:'verse_loop',
                   steps:'verse1,verse2,verse1'),
        style(label:'Create Loop', x:10, y:90)
      )"/>
```

4. Play Sequence

```
ui(action:"controlXYSequence", seq:"seqName", dur:2, ease:"easeInOutSine", loop:true)
```

Examples:

```
<!-- Auto-play when playhead hits -->
<rect x="2000"
      cue="ui(action:'controlXYSequence',
              seq:'intro_pattern',
              dur:1.5,
              loop:false)"/>

<!-- Button with looping -->
<g cue="button(
        trigger:ui(action:'controlXYSequence',
                   seq:'verse_loop',
                   dur:2,
                   loop:true),
        style(label:' Loop Verse', x:10, y:130)
      )"/>
```

5. Stop Sequence

```
ui(action:"controlXYSequenceStop")
```

Examples:

```
<!-- Playhead trigger -->
<rect x="4000" cue="ui(action:'controlXYSequenceStop')"/>

<!-- Button -->
<g cue="button(
        trigger:ui(action:'controlXYSequenceStop'),
        style(label:' Stop', x:10, y:170)
      )"/>
```

OSC Output (Bonus Feature)

While spatial animation is the primary use case, all handle movements can **simultaneously control external software**.

Enable OSC

```
<rect id="pad1"
      cue="controlXY(uid:synth, touchpt:dot1, osc:true)"/>
```

Custom Address & Throttle

```
<rect id="pad1"
      cue="controlXY(uid:synth, touchpt:dot1, osc:50, oscAddr:'max/xy')"/>
```

OSC Messages Sent

XY only (no rotation):

```
/controlXY/synth 0.42 0.87
```

XY + Rotation:

```
/controlXY/synth 0.42 0.87 0.65
```

Third value is rotation (0.0-1.0, normalized by `hmode`).

Multiple handles:

```
/controlXY/synth/dot1 0.42 0.87 0.33
/controlXY/synth/dot2 0.15 0.63 0.78
```

Custom address:

```
/max/xy 0.42 0.87 0.65
```

Works with Max/MSP, Pure Data, SuperCollider, TouchOSC, and any OSC-compatible software.

Preset Panel UI

Opening the Panel

1. **Click button** in top-right of any controlXY pad
2. **Keyboard: Alt+Shift+P**
3. **Console:** `window.controlXYPresetUI.toggle()`

Panel Features

- **Save Section:** Type name, click
- **Presets List:** Click to recall, to delete
- **Recall Options:** Set duration (seconds) and easing
- **Sequences:** Play/stop defined sequences
- **Import/Export:** Save/load preset files

Keyboard Shortcut

Alt+Shift+P toggles the panel (changed from Ctrl+Shift+P to avoid Chrome conflict).

Storage & Persistence

Presets auto-save to `scores/<project>/controlxy-presets.json`:

```
{
  "presets": {
    "intro": {
      "pad1": {
        "dot1": { "x": 0.25, "y": 0.75, "p": 0.33 },
        "dot2": { "x": 0.80, "y": 0.30, "p": 0.67 }
      }
    }
  },
  "sequences": {
    "verse_pattern": ["verse1", "verse2", "verse1"]
  },
  "updatedAt": 1704067200000,
  "projectId": "myProject"
}
```

The `p` value stores rotation (0.0-1.0) if the handle has rotation control. It is saved, recalled, and **smoothly tweened** between presets just like `x` and `y`. Loaded automatically when project loads.

CSS Styling

Handle Classes

```
.controlxy-handle {
  cursor: grab;
  transition: opacity 0.15s;
}

.controlxy-handle:hover {
  opacity: 0.8;
}

.controlxy-handle--active {
  cursor: grabbing;
  filter: drop-shadow(0 0 10px rgba(100, 200, 255, 0.9));
}
```

Label Styling

```
.controlxy-label {
  font-family: 'SF Mono', 'Monaco', monospace;
  font-size: 11px;
  fill: #fff;
  filter: drop-shadow(0 1px 2px rgba(0, 0, 0, 0.8));
}
```

Toggle Button

The button is automatically added to each pads top-right corner. Style via:

```
.controlxy-toggle-btn {
  background: rgba(30, 30, 30, 0.9);
  border: 1px solid rgba(255, 255, 255, 0.3);
}

.controlxy-toggle-btn:hover {
  background: rgba(30, 30, 30, 1);
  transform: scale(1.1);
}
```

Complete API Reference

JavaScript Console API

```
// ===== PRESETS =====
controlXYPresets.save(name, uidFilter?)
controlXYPresets.recall(name, options?)
controlXYPresets.delete(name)
controlXYPresets.list()                    // Returns array of preset names
controlXYPresets.get(name)                 // Returns preset data

// ===== TWEENING =====
controlXYPresets.tweenTo(positions, options?)  // positions: { uid: { handleId: { x, y, p? } } }
controlXYPresets.stopAllTweens()

// ===== SEQUENCES =====
controlXYPresets.defineSequence(name, steps)
controlXYPresets.playSequence(name, options)
controlXYPresets.stopSequence()
controlXYPresets.getActiveSequence()       // Returns current sequence info

// ===== SCENES =====
controlXYPresets.saveScene(name)           // Save all handle positions + launcher state
controlXYPresets.recallScene(name, options?)  // Recall scene (supports dur/ease tweening)
controlXYPresets.deleteScene(name)
controlXYPresets.listScenes()
controlXYPresets.getScene(name)

// ===== PERSISTENCE =====
controlXYPresets.init(projectId)           // Called automatically
controlXYPresets.export()                  // Returns JSON string
controlXYPresets.import(json, merge?)
controlXYPresets.importFromProject(projectId, merge?)

// ===== UI =====
controlXYPresetUI.show()
controlXYPresetUI.hide()
controlXYPresetUI.toggle()
controlXYPresetUI.refresh()
```

Compositional Patterns

Pattern 1: Verse-Chorus Automation

1. Save state at verse start
2. Save state at chorus start
3. Tween between them at transitions
4. Add button overrides for live performance

Pattern 2: Looping Textures

1. Define 3-4 related states
2. Create sequence with varied timing
3. Loop sequence with `loop:true`
4. Bind to visual parameters for evolving texture

Pattern 3: Build-Tension-Release

```
1. Start at calm state (center, low values)
2. Sequence through increasingly tense positions
3. Climax: rapid sequence or elastic bounce
4. Release: slow tween back to calm
```

Pattern 4: Call-Response

```
1. Manual control for "call" phrase
2. Automated sequence for "response"
3. Alternate between live and programmed
```

Pattern 5: Polytemporal Layers

```
1. Multiple pads, each with own sequence
2. Different loop lengths (3, 5, 7 steps)
3. Creates phasing/evolving relationships
4. Each pad controls different visual layer
```

Technical Notes

- `controlXY` is a **control-plane cue**, not temporal
- Registered during `assignCues()`, not via playhead
- Handles initialized at center of bounds (XY) and 0 rotation
- Uses Pointer Events API (works with touch, mouse, stylus)
- All tweening uses `requestAnimationFrame` for smooth 60fps
- Event propagation stopped to prevent score dragging
- Multi-touch: each pointer controls nearest available handle
- **Rotation handles:**
  - Can be shared (one handle for all touchpoints) or individual
  - Coordinate system: 0 = 7 oclock, clockwise positive
  - `limited` mode applies hard stops using angle clamping
  - `continuous` mode normalizes angles to 0-360 with wraparound

Summary

`controlXY` fundamentally transforms the score from **static notation** into **dynamic canvas**. By combining:

1. **Spatial control surfaces** (the pads)
2. **Parameter binding** (connecting space to parameters)
3. **Preset/sequence system** (programming choreography)
4. **Live performance** (manual override)

composers gain the ability to create **complex, evolving visual/sonic animations** directly within the score itself. The score becomes both **instrument and notation**, collapsing the traditional divide between composition and performance.

As a bonus, OSC output allows the same spatial gestures to control external synthesis and media systems, making `controlXY` a **unified interface** for all aspects of a multimedia performance.

Score-as-instrument. Instrument-as-score.

Quick Start Checklist

1. Add CSS: `<link rel="stylesheet" href="controlxy-preset-ui.css">`
2. Define pad: `<rect cue="controlXY(uid:pad1, touchpt:dot1)"/>`
3. *Optional:* Add rotation: `handle:knob, hmode:limited` for volume-style controls
4. Open UI: Press **Alt+Shift+P** or click
5. Save states: Move handles, name them, click
6. Animate: Use `ui(action:'controlXYRecall', ...)` on playhead triggers

7. Choreograph: Define sequences, play/loop them
8. Bind: Connect handle positions to visual parameters
9. Perform: Override automation with buttons or manual control

Now go create some animated score-instruments!

Chapter 6

Tools

This section covers external tools that extend the Oscilla authoring workflow. While scores can be created entirely by editing SVG element IDs in Inkscapes Object Properties dialog, dedicated tooling can streamline the process of building and applying cues.

OSCILLA Inkscape Extension

A smart cue editor for creating OSCILLA interactive graphic notation within Inkscape.

Overview

The OSCILLA Inkscape Extension provides a graphical interface for applying OSCILLA DSL cues to SVG elements. Instead of manually typing cue syntax into element IDs, you select cue types from organized categories, configure parameters using appropriate widgets, and apply them directly to selected elements.

Installation

Automatic Installation

```
cd oscilla-inkscape-extension
./install.sh
```

The installer detects your operating system and copies files to the correct Inkscape extensions folder.

Manual Installation

Copy all files to your Inkscape extensions folder:

OS	Extensions Path
Linux	`~/.config/inkscape/extensions/`
macOS	`~/Library/Application Support/org.inkscape.Inkscape/config/inkscape/extensions/`
Windows	`%APPDATA%\inkscape\extensions\`

Restart Inkscape after installation.

First-Time Setup: Keyboard Shortcut

For efficient workflow, bind “Apply Queued Cue” to a keyboard shortcut:

1. In Inkscape: **Edit Preferences Interface Keyboard**
2. In the search box, type `Apply Queued`
3. Click on the “Apply Queued Cue” entry
4. Press your desired shortcut (recommended: `Ctrl+Shift+Q`)
5. Click **Close**

This shortcut applies cues configured in the Smart Cues Editor to your selected elements.

For detailed keyboard shortcut documentation, see the Inkscape Keyboard Shortcut Guide.

Components

The extension consists of two parts under **Extensions OSCILLA**:

OSCILLA Smart Cues Editor

The main interface for building cues. Opens as a floating window that stays on top of Inkscape while you work.

Note: The editor cannot dock inside Inkscape - it remains a separate floating window. However, it stays on top so you can see both windows simultaneously.

Apply Queued Cue

Applies the configured cue to selected elements in Inkscape. Bind this to a keyboard shortcut for the fastest workflow.

Workflow

Standard Workflow

1. **Open the Smart Cues Editor:** Extensions OSCILLA OSCILLA Smart Cues Editor
2. **Keep the editor open** - it floats above Inkscape
3. **Configure your cue:**
   - Select a category (Timing, Animation, Visual, etc.)
   - Select a cue type
   - Optionally select a preset
   - Adjust parameters as needed
4. **Click "Apply to Selection"** - this queues the cue
5. **In Inkscape:** Select one or more elements
6. **Press your keyboard shortcut** (e.g., `Ctrl+Shift+Q`) - cue is applied!

Quick Workflow with Presets

Once youve saved presets for your commonly used cues:

1. Open Smart Cues Editor (keep it open)
2. Select preset from dropdown
3. Click "Apply to Selection"
4. Select element in Inkscape Press `Ctrl+Shift+Q`

Stacking Multiple Cues

To add multiple cues to one element:

1. Apply the first cue normally
2. Check **Append to existing ID**
3. Configure and apply additional cues

Result: `pause(dur:4) scale(values:[1,1.3,1], dur:2)`

Using the Smart Cues Editor

Interface Elements

Category Dropdown

Select from seven cue categories:

Category	Cue Types
Timing & Navigation	stop, pause, speed, nav, page, stopwatch, metro
Animation	scale, scaleXY, rotate, o2p
Visual	color, fade, text
Audio / Video	audio, audioPool, audioImpulse, video
Synth	synth, synthStop
OSC	osc, oscCtrl
Interaction	button, reuse, use

Cue Type Dropdown

Shows available cues for the selected category. When you select a cue type, only the relevant parameters appear below.

Preset Dropdown

Load pre-configured parameter combinations. Select " Custom " to configure parameters manually.

Parameters Area

Dynamic parameter widgets appropriate to each parameter type:

Widget	Used For
Text entry	Free text, file paths, color values
Spin button	Integer values with increment/decrement
Float spinner	Decimal values with precision
Dropdown	Fixed option selection
Checkbox	Boolean on/off values
Color button	Color picker (alongside text entry)

Preview

Shows the cue string as you configure parameters. Updates in real-time. Wraps long cues across multiple lines.

Append Checkbox

When checked, the cue is added to the elements existing ID rather than replacing it. Use this to stack multiple cues on one element.

Buttons

Button	Action
Save Preset	Save current parameters as a reusable preset
Copy to Clipboard	Copies the cue string for manual pasting
Apply to Selection	Queues the cue for application via keyboard shortcut

Presets

Using Presets

Presets provide quick access to common cue configurations. Select from the Preset dropdown to auto-fill parameters, then adjust as needed.

Saving Presets

1. Configure parameters as desired
2. Click **Save Preset**
3. Enter a name for the preset
4. Click **Save**

The preset is immediately available in the Smart Cues Editor dropdown.

Customizing Presets Manually

Presets are stored in `oscilla_presets.json` in your extensions folder. You can edit this file directly.

File Location

OS	Path
Linux	`~/.config/inkscape/extensions/oscilla_presets.json`
macOS	`~/Library/Application Support/org.inkscape.Inkscape/config/inkscape/extensions/oscilla_presets`
Windows	`%APPDATA%\inkscape\extensions\oscilla_presets.json`

Structure

```
{
  "category_id": {
    "cue_name": [
      {
        "name": "Preset Display Name",
        "params": {
          "param_name": value
        }
      }
    ]
  }
}
```

Example: Adding Pause Presets

```
{
  "timing": {
    "pause": [
      {
        "name": "Short Pause (4s)",
        "params": {
          "dur": 4
        }
      },
      {
        "name": "Long Pause with Countdown",
        "params": {
          "dur": 12,
          "count": true
        }
      }
    ]
  }
}
```

Category IDs

Use these exact category IDs in the JSON:

Display Name	Category ID
Timing & Navigation	`timing`
Animation	`animation`
Visual	`visual`
Audio / Video	`audio`
Synth	`synth`
OSC	`osc`
Interaction	`interaction`

Cue Names

Use the exact cue name as shown (case-sensitive):

`stop`, `pause`, `speed`, `nav`, `page`, `stopwatch`, `metro`, `scale`, `scaleXY`, `rotate`, `o2p`, `color`, `fade`, `text`, `audio`, `audioPool`, `audioImpulse`, `video`, `synth`, `synthStop`, `osc`, `oscCtrl`, `button`, `reuse`, `use`

Validating Your Presets

After editing, verify the JSON is valid:

```
python3 -m json.tool oscilla_presets.json > /dev/null && echo "Valid JSON"
```

Restart the Smart Cues Editor to load updated presets.

Troubleshooting

Extension not appearing in menu

1. Restart Inkscape completely
2. Verify files are in the correct extensions folder
3. Check for Python errors: `python3 -c "import oscilla_smart_cues_gtk"`

Editor wont open

Check the launcher script is executable and Python 3 with GTK is available:

```
python3 -c "import gi; gi.require_version('Gtk', '3.0'); from gi.repository import Gtk; print('GTK OK')"
```

Cue not applying to element

1. Ensure an element is selected in Inkscape
2. Ensure youve clicked “Apply to Selection” in the Smart Cues Editor
3. Press your keyboard shortcut (e.g., `Ctrl+Shift+Q`)
4. If still not working, check `/tmp/oscilla_cue.txt` exists after clicking Apply

Keyboard shortcut not working

1. Verify the shortcut is bound: Edit Preferences Interface Keyboard search “Apply Queued”
2. Make sure Inkscape window is focused when pressing the shortcut
3. Try a different shortcut combination if theres a conflict

Presets not loading

1. Validate JSON syntax: `python3 -m json.tool oscilla_presets.json`
2. Check category and cue names match exactly (case-sensitive)
3. Restart the Smart Cues Editor

Editor not staying on top

On some systems/window managers, “always on top” may not work. Try: - Right-click the window title bar “Always on Top” (if available) - Use your window managers settings to pin the window

Files

File	Purpose
`oscilla_smart_cues_gtk.py`	Main GTK editor application
`oscilla_smart_cues_launcher.py`	Inkscape extension that launches editor
`oscilla_smart_cues_launcher.inx`	Menu entry definition
`oscilla_apply_cue.py`	Extension that applies cue to selection
`oscilla_apply_cue.inx`	Menu entry definition
`oscilla_presets.json`	User-editable preset configurations